

Unit 9

ALGORITHM DESIGN AND PROBLEM-SOLVING

145 items · 848 marks

Syllabus sections in this unit

- 9.1 Computational Thinking Skills
- 9.2 Algorithms

Unit 9: Algorithm Design and Problem-solving

1. M/J 26/P21/Q2/a(ii,iii)

2 A washing machine contains:

- two trays for the washing powder
- a user interface display
- five sensors.

The washing machine control program includes the following variables:

variable	description
MainsOn	TRUE when the mains power is on
WaterOn	TRUE when the water supply to the machine is turned on
DisplayOn	TRUE when the user interface display is active
Tray1	TRUE when it detects washing powder in tray 1
Tray2	TRUE when it detects washing powder in tray 2

- (ii) The display is only active when both the mains power is on and the water supply is turned on.

Complete the pseudocode statement.

DisplayOn ←

[1]

- (iii) The washing machine control program will start when a new variable `WashStart` is assigned the value `TRUE`

`WashStart` is assigned the value `TRUE` when the mains power is switched on and there is washing powder in tray 1 or tray 2 or both trays.

Write the pseudocode statement for the given description.

[3]

2. M/J 26/P21/Q4

4 An algorithm is expressed as:

- A user inputs a sequence of integer values.
- The process will calculate the number of positive integers.
- The user will input value 0 to indicate there are no more numbers to be input.
- Output the number of positive integers after the terminating 0 is input.

Write a structured English description for this algorithm as a sequence of steps.

Do **not** include pseudocode statements in your description.

Your answer might **not** require all **six** steps shown.

step 1

step 2

step 3

step 4

step 5

step 6

[4]

3. M/J 26/P22/Q2/a,b,c

2 A new computer system is being developed to help manage a tool hire business. A customer can specify if they want to hire a tool for either half a day or a full day.

An abstract model for the new system is required.

(a) Explain the purpose of abstraction.

[1]

(b) State **four** items of data that must be stored each time a tool is hired.

item 1

item 2

item 3

item 4

[2]

(c) The computer system will have to process the tool hire data. One process is to cancel a tool hire booking.

State **two** other processes that would be required.

process 1

process 2

[2]

4. M/J 26/P22/Q3

- 3 A computer game uses a text file named `WordList.txt` containing 10 000 words, all in lower case. Each word is unique and is stored on a separate line of the text file.

An efficient algorithm is needed to check that a word input by the user is stored in `WordList.txt`. If the word is found, the line number where the word is stored must be output. If the word is **not** found, the message "Not Found" must be output.

Complete the steps needed to describe the algorithm.

Do **not** include pseudocode statements in your answer.

The first **three** steps have been completed for you.

step 1: set line counter to 0

step 2: input a word

step 3: convert the word to lower case

[6]

5. M/J 26/P23/Q2/b,c

- 2 A control program is being developed for a robot floor vacuum cleaner. The settings for the robot can be changed and include:

- the strength of the suction (numerical values: 0, 1, 2, 3, 4 or 5)
- the total vacuum cleaning time (a whole number of minutes).

The robot has a number of sensors.

The control program includes the variables:

variable	description
MainsOn	TRUE when the mains power is on
DisplayOn	TRUE when the user display is shown
Suction	TRUE when the program detects that the user has selected a non-zero value for the strength of the suction

- (b) Write a pseudocode statement to set the `DisplayOn` variable to TRUE when the mains power is on.

[1]

- (c) When the robot is ready to move, the program assigns the value TRUE to a variable, `Move`

The control program contains the pseudocode logic expression:

`Move ← MainsOn AND Suction`

Explain the pseudocode statement.

[2]

6. M/J 26/P23/Q4

4 An algorithm is expressed as:

- A user inputs a sequence of integers.
- The user will input value -1 to indicate there are no more integers to be input.
- The program will calculate the total number of integers greater than 120.
- Output the total number of integers greater than 120 after the -1 terminator is input.

Write a structured English description for this algorithm as a sequence of steps.

Do **not** include pseudocode statements in your description.

Your answer might **not** require all **six** steps shown.

step 1

step 2

step 3

step 4

step 5

step 6

[4]

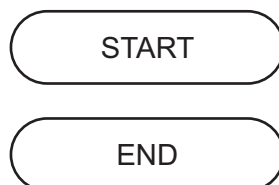
7. O/N 25/P21/Q2

2 `Data` is a global 1D array containing 30 elements of type `STRING`

An algorithm will output:

- all non-blank elements (elements that do **not** contain an empty string)
- the final total of the number of elements output.

Complete the program flowchart to represent the algorithm:



[5]

8. O/N 25/P22/Q1/a

1 (a) The table contains pseudocode examples.

Each example may contain statements that relate to one or more of:

- selection
- iteration (repetition)
- subroutines (procedures or functions).

Complete the table by placing **one or more** ticks (✓) in each row.

Pseudocode example	Selection	Iteration	Subroutine
IF Status = FALSE THEN FOR Count ← 1 TO 20 CALL Reset (Count) NEXT Count ENDIF			
OTHERWISE : Status ← TRUE			
WHILE AllDone() = TRUE			
30 : NextChar ← 'X'			

[4]

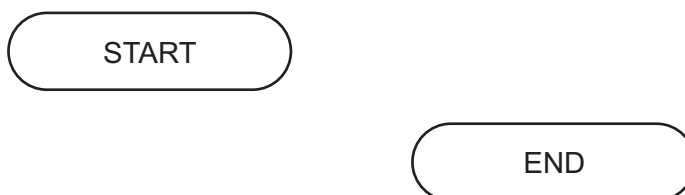
9. O/N 25/P22/Q2

2 Data is a global 1D array containing 20 elements of type REAL

An algorithm will:

- input a sequence of real values, one at a time
- assign each value to consecutive array elements, starting from index 1
- end when the value 99.9 is input, or all 20 elements have been assigned (the value 99.9 must **not** be stored in the array).

Complete the program flowchart to represent the algorithm:



[5]

10. O/N 25/P22/Q3/a

- 3 A text file `OldFile.txt` contains IDs, names and email addresses for members of a club. Three information items are stored for each member and each item is stored on a separate line.

The example shows the information for the first two members in the first six lines of the file:

Line in file	Information item	Example data
1	member 1 ID	"AB1234"
2	member 1 name	"Freddie Jones"
3	member 1 email address	"FreddieJ909@Cambridge.org"
4	member 2 ID	"BC2345"
5	member 2 name	"Sue Smith"
6	member 2 email address	"Sue1024@Cambridge.org"

The member ID string is always two letters followed by four digits.

The file design is to be changed so that information for each user is stored as a single line of the file, with the character '`\`' used as a separator between data items.

For example, the single line for member 1 will be:

"AB1234\Freddie Jones\FreddieJ909@Cambridge.org"

An algorithm will produce a new file `NewFile.txt` from the contents of `OldFile.txt`

Assume:

- `LineX`, `LineY` and `LineZ` are of type `STRING` and are used to store the three items of information for each member
- `NewString` is a temporary variable of type `STRING`
- `OldFile.txt` exists and contains valid data.

- (a) The algorithm to create the new file is expressed in steps.

Complete the following numbered steps:

- Step 1 : open the file in
- Step 2 : open the file in
- Step 3 : read a line from and store in
- Step 4 : read a line from and store in
- Step 5 : read a line from and store in
- Step 6 : set `NewString` to
- Step 7 : write `NewString` to
- Step 8 : repeat from step until
- Step 9 : close both files.

11. O/N 25/P22/Q4/c

4 A quiz has nine questions. There are:

- five easy questions each worth 3 points
- four hard questions each worth 5 points.

At the end of the quiz the points for each correctly answered question are added to give a total score.

A check is made to test that the total score is valid using a 1D array `CheckTotal` of type `BOOLEAN`. Each index value of the array represents a possible total score. The corresponding element value is `TRUE` if the index value represents a valid total score and `FALSE` otherwise.

The first nine rows of the array are:

Index value	Element value	Comment
0	TRUE	valid total score (no correct answers)
1	FALSE	invalid total score
2	FALSE	invalid total score
3	TRUE	valid total score (one 3-point question correct)
4	FALSE	invalid total score
5	TRUE	valid total score (one 5-point question correct)
6	TRUE	valid total score (two 3-point questions correct)
7	FALSE	invalid total score
8	TRUE	valid total score (one 3-point question and one 5-point question correct)

For example, a total score of 6 points is valid; the value of the array element at index value 6 is `TRUE`

(c) The array elements have been assigned the required values.

A module `ValidateScore()` will take an integer value representing a total score as a parameter. It will return `TRUE` if the total score is valid, or `FALSE` if it is **not** valid.

Describe the algorithm for `ValidateScore()` using **four** steps.

Do **not** use pseudocode in your answer.

Step 1

Step 2

Step 3

Step 4

[4]

12. O/N 25/P23/Q2/a,b

- 2 (a)** A program contains a global 1D array of type `STRING` containing 65 elements.

An existing text file is used to store the data in the array. Only non-blank elements (those that do **not** contain an empty string) are written to the text file.

An algorithm will:

- write the first element of the array as a new line in the text file
- continue until all elements have been written, each to a new line of the text file.

Stepwise refinement is applied to the algorithm.

Describe up to **six** steps for this algorithm that could be used to produce pseudocode.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

Step 5

Step 6

[6]

- (b)** Iteration is one programming construct.

Identify **one** other programming construct that will be required when the algorithm from part **(a)** is converted into pseudocode and explain how it is used.

Construct

Use

[2]

13. M/J 25/P21/Q1/a,b,c

1 A program is being developed to help the manager of a shop control the stock.

(a) An identifier table has been used during the design stage.

Complete the identifier table:

Example value	Explanation	Variable name	Data type
"Fruit"	a category of stock that is sold in the shop		
20/02/2025	when an item was sold		
12.67	the cost of an item		
TRUE	to indicate if an item is in stock		

[4]

(b) A module `Sales()` is part of the stock control program.

The table contains pseudocode extracts from the module `Sales()`

Each extract may include all or part of:

- assignment
- selection
- iteration (repetition).

Complete the table by placing one or more ticks (✓) in each row:

Pseudocode extract	Assignment	Selection	Iteration
<code>Result ← CalculateTotal()</code>			
<code>WHILE IsClosed</code>			
<code>REPEAT INPUT Value UNTIL Sales[4] > Value</code>			
<code>IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF</code>			
<code>CASE OF Option</code>			

[5]

(c) Decomposition has been used to design the program to help the shop manager control the stock.

Describe decomposition.

[3]

14. M/J 25/P21/Q3

- 3** A student has been asked to create a simple guessing game program. This program will generate a random integer value between 1 and 100. It will then repeatedly prompt the user to input an integer value until they input the randomly generated value.

The student has written a structured English description:

step 1 – randomly generate an integer value between 1 and 100 inclusive

step 2 – prompt the user to input an integer value

step 3 – output an appropriate message if the value input was too high; then repeat from **step 2**

step 4 – output an appropriate message if the value input was too low; then repeat from **step 2**

step 5 – output an appropriate message if the value input was the same value that was randomly generated; then end the program.

Write a pseudocode algorithm from this structured English description.

Assume no input validation is needed.

[8]

15. M/J 25/P22/Q2/a

- 2** A program to calculate the pay of employees working for a company is being designed.

(a) Stepwise refinement has been used in the design of the program.

Describe stepwise refinement.

[2]

16. M/J 25/P22/Q3

- 3** An algorithm is designed to generate and output two unique random integers. Each integer value is between –10 and 10 inclusive.

If both integers output are negative, a third random integer between 30 and 35 inclusive will be generated and output.

Write pseudocode for this algorithm.

[8]

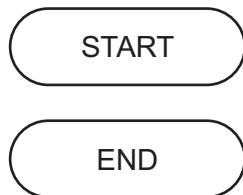
17. M/J 25/P22/Q4

4 An algorithm will:

- input 100 integer values, one value at a time
- store the first value input into the first location of the array `Number`
- store the next input value in the next unused location of the array `Number`
- output the contents of `Number` array in the opposite sequence to that in which the values were input.

Complete the program flowchart to represent the algorithm.

Variable declarations are **not** required.



[6]

18. M/J 25/P23/Q3

3 A programmer has been asked to create a module `RollDice()` to simulate multiple rolls of a dice. This module will be used as part of a program for a game.

The module will:

- step 1 – take a positive integer parameter representing the number of times the dice will be rolled
- step 2 – simulate **one** roll of the dice by generating a random integer between 1 and 6 inclusive
- step 3 – output the integer generated
- step 4 – repeat, as required from **step 2**
- step 5 – calculate the average value of the random integers generated
- step 6 – return the average value.

Write pseudocode for the module `RollDice()`

[7]

19. M/J 25/P23/Q4/a

- 4 A programmer is developing a new stock control program for a shop owner to produce sales reports.

(a) A text file `Sales.txt` stores strings representing the number of items sold.

The file contains 52 lines, each representing the number of items sold in each week of the year.

For example:

File line number	File data
1	"350"
2	"502"
3	"434"
	
49	"492"
50	"872"
51	"772"
52	"201"

The example shows that 502 items were sold in week 2.

The owner requires a module to output all week numbers where the number of items sold is greater than 500.

Describe an algorithm that produces the required module.

Do **not** include pseudocode statements in your answer.

[5]

20. O/N 24/P21/Q3/a,b

- 3** An algorithm will output the **last** three lines from a text file `Result.txt`

The lines need to be output in the same order as they appear in the file.

Assume:

- Three variables `LineX`, `LineY` and `LineZ` will store the three lines. These are of type string and all three variables have been initialised to an empty string.
- The file exists and contains **at least** three lines.

- (a)** The algorithm to output the lines is expressed in eight steps.

Complete the steps.

1. Open the file
2. Loop until
3. and store in `ThisLine`
4. Assign `LineY` to `LineX`
5. Assign `LineZ` to `LineY`
6. Assign `ThisLine` to `LineZ`
7. After the loop,
8. Output `LineX`, `LineY`, `LineZ`

[4]

- (b)** Explain the purpose of steps 4, 5 and 6 in the algorithm from part **(a)**.

[1]

21. O/N 24/P22/Q7/a,b

- 7** A coffee shop runs a computerised loyalty card system.

Customers are issued with a loyalty card with their name together with a unique customer ID.

Loyalty points are added to their card each time they spend money at the shop.

The following information is stored for each customer: ID, name, home address, email address, mobile phone number, date of birth, number of points, date of last visit and amount of money spent at last visit.

A new module will generate a personalised email message to each loyalty card customer who has not visited the coffee shop in the last three months. The message will include a unique voucher code which can be used to authorise a discount if the customer goes to the shop within the next two weeks.

- (a)** Identify **three** items of customer information that will be used by the new module **and** justify your choices.

Item 1

Justification

Item 2

Justification

Item 3

Justification

[3]

- (b)** Identify the computational thinking skill that you needed to use to answer part **(a)**.

[1]

22. O/N 24/P23/Q1/a

- 1 (a)** The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:

- selection
- iteration (repetition)
- subroutine (procedure or function).

Complete the table by placing **one or more** ticks ('✓') in each row.

Pseudocode example	Selection	Iteration	Subroutine
FOR Index ← 1 TO 3 IF Safe[Index] = TRUE THEN Flap[Index] ← 0 ENDIF NEXT Index			
CASE OF Compound(3)			
REPEAT UNTIL AllDone() = TRUE			
WHILE Result[3] <> FALSE			

[4]

23. O/N 24/P23/Q2/a,b

2 An algorithm will:

1. prompt and input a sequence of 100 integer values, one at a time
2. sum the positive integers
3. output the result of the sum.

(a) Write pseudocode for the algorithm.

Assume the value zero is neither positive nor negative.

You must declare all variables used in the algorithm.

[5]

(b) The algorithm requires the use of basic constructs. One of these is sequence.

Identify **one other** basic construct required by the algorithm **and** describe how it is used.

Construct

Use

[2]

24. O/N 24/P23/Q3/b,c

- 3** The implementation of a linked list uses an integer variable and a 1D array `List` of type `Node`.

Record type `Node` is declared in pseudocode as follows:

```
TYPE Node
  DECLARE Data : STRING
  DECLARE Pointer : INTEGER
ENDTYPE
```

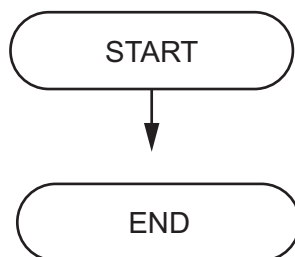
The array `List` is declared in pseudocode as follows:

```
DECLARE List : ARRAY[1:200] OF Node
```

The linked list will operate as follows:

- Integer variable `HeadPointer` will store the array index for the first node in the linked list.
 - The `Pointer` field of a node contains the index value of the next array element (the next node) in the linked list.
 - The value 0 is used as a null pointer.
- (b)** The array `List` will be initialised so that each node points to the following node. The last node will contain a null pointer.

Complete the program flowchart to represent the algorithm for this operation.



[4]

- (c)** An algorithm outputs the `Data` field from all nodes in the array `List`. The order the `Data` is output should be the same order it is stored in the linked list.

Describe the algorithm in **four** steps.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

[4]

25. O/N 24/P23/Q7/b

- 7** A coffee shop owner wants to introduce a computerised loyalty card system.

A programmer discusses the details of the system with the shop owner.

- (b)** The shop will give each customer a loyalty card that displays a unique customer ID as a bar code. A customer will be able to present their card each time they make a purchase. The system will scan the bar code, calculate points, and add them to the customer's total. When the customer next makes a purchase and presents their card, they will have the option to exchange points for a discount.

The designer decides that this activity will be handled by a new module. Decomposition will be used to break the problem of designing the new module down into sub-problems (sub-modules).

Identify **four** sub-modules that could be used in the design of the new module **and** describe their use.

Sub-module 1

Use

Sub-module 2

Use

Sub-module 3

Use

Sub-module 4

Use

[4]

26. M/J 24/P21/Q1/c

- 1 An algorithm is developed in pseudocode before being coded in a programming language.

(a) The following table shows four valid pseudocode assignment statements.

Complete the table by giving an appropriate data type to declare each of the variables A, B, C and D.

Assignment statement	Data type
A ← LEFT(MyName, 1)	
B ← Total * 2	
C ← INT(ItemCost) / 3	
D ← "Odd OR Even"	

[4]

(c) The variable names A, B, C and D in part (a) are **not** good programming practice.

(i) State why these variable names are **not** suitable.

[1]

(ii) Identify **one** problem that these variable names might cause.

[1]

(iii) The choice of suitable variable names is one example of good programming practice.

Give **one other** example.

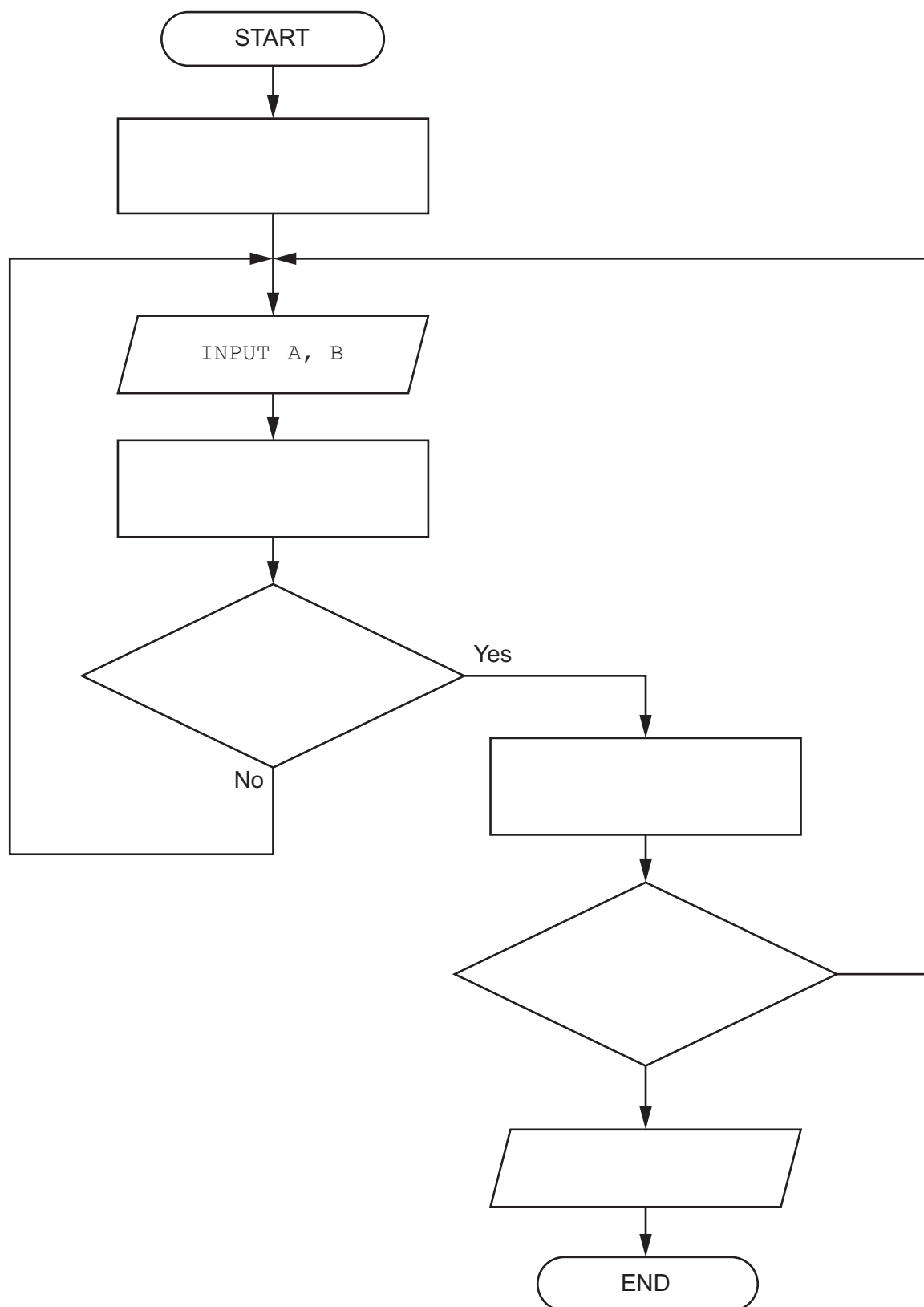
[1]

27. M/J 24/P21/Q2/a

- 2 An algorithm has three steps. It will:

- repeatedly input a pair of numeric values A and B
- count the number of pairs that are input until A has been greater than B 10 times
- output the number of pairs that were input.

(a) Complete the program flowchart.



28. M/J 24/P21/Q7/a

- 7 A fitness club has a computerised membership system. The fitness club offers a number of different exercise classes.

The following information is stored for each club member: name, home address, email address, mobile phone number, date of birth and the exercise(s) they are interested in.

- (a) When an exercise class is planned, a new module will send personalised text messages to each member who has expressed an interest in that exercise. Members wishing to join the class send a text message back. Members may decide **not** to receive future text messages by replying with the message 'STOP'.

The process of abstraction is used to filter out unnecessary information.

- (i) State **one** advantage of applying abstraction to this problem.

[1]

- (ii) Identify **three** items of information that will be required by the new module. Justify your choices with reference to the given scenario.

Item 1 required

Justification

Item 2 required

Justification

Item 3 required

Justification

[3]

- (iii) Identify **two** operations that would be required to process data when the new module receives a text message back from a member.

Operation 1

Operation 2

[2]

29. M/J 24/P22/Q1/a,c

- 1 (a) The following table contains pseudocode examples.

Each example may contain statements that relate to one or more of the following:

- selection
- iteration (repetition)
- input/output.

Complete the table by placing **one or more** ticks (✓) in each row.

Pseudocode example	Selection	Iteration	Input/Output
FOR Index $\leftarrow$ 1 TO 10 Data[Index] $\leftarrow$ 0 NEXT Index			
WRITEFILE ThisFile, "*****"			
UNTIL Level > 25			
IF Mark > 74 THEN READFILE OldFile, Data ENDIF			

[4]

(b) Program variables have data types as follows:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER

Complete the table by filling in each gap with a function (from the **insert**) so that each expression is valid.

Expression
MyInt $\leftarrow$ (3.1415926)
MyChar $\leftarrow$ ("Elwood", 3, 1)
MyString $\leftarrow$ (..... (27.509))
MyInt $\leftarrow$ (..... ("ABC123", 3))

[4]

(c) The variables given in part (b) are chosen during the design stage of the program development life cycle.

The choices are to be documented to simplify program maintenance.

State a suitable way of documenting the variables **and** give **one** piece of information that should be recorded, in addition to the data type.

[2]

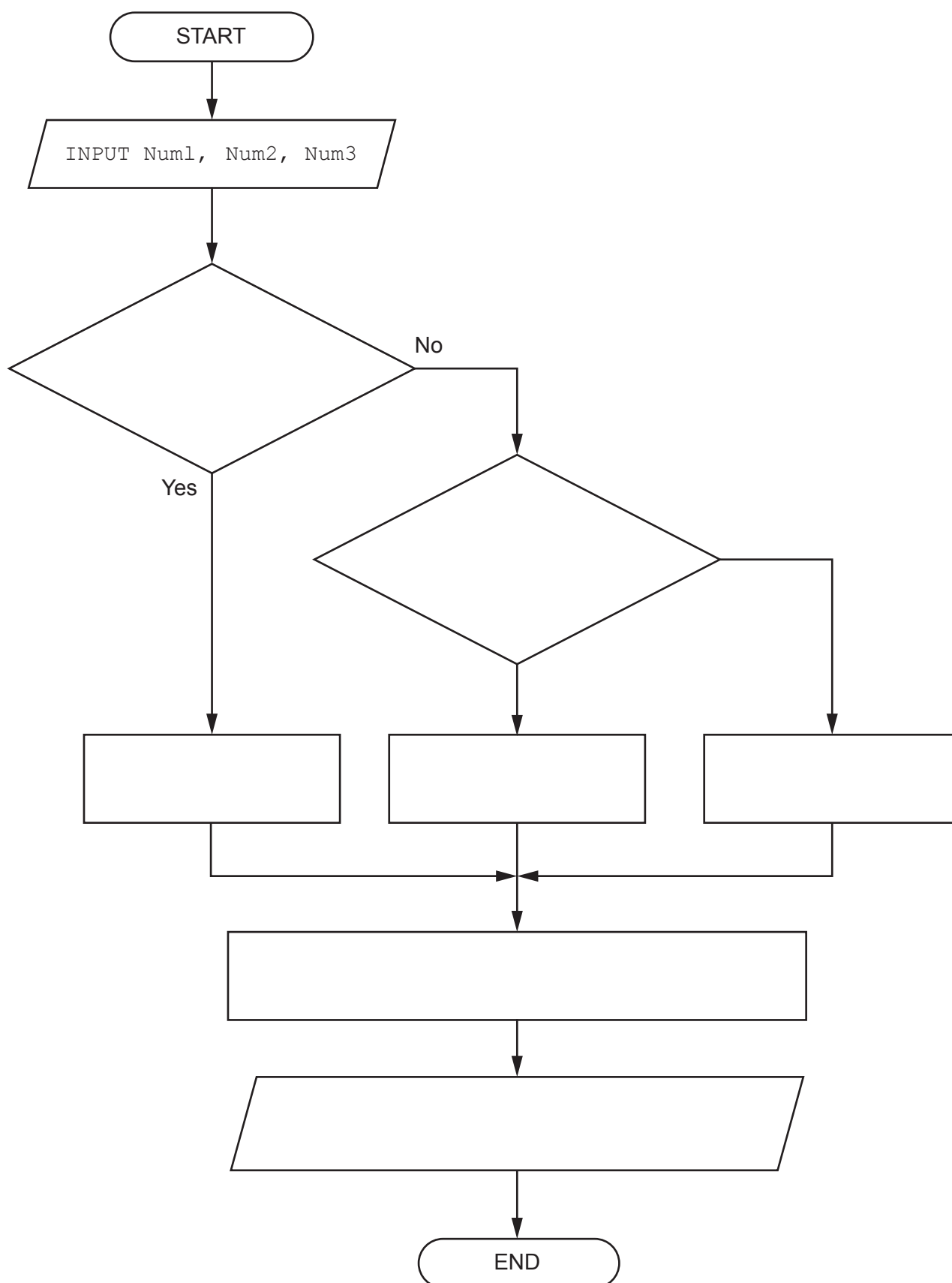
30. M/J 24/P22/Q2/a,b

2 A program is being developed.

- (a) An algorithm for part of the program will:
- input three numeric values and assign them to identifiers `Num1`, `Num2` and `Num3`
 - assign the largest value to variable `Ans`
 - output a message giving the largest value and the average of the three numeric values.

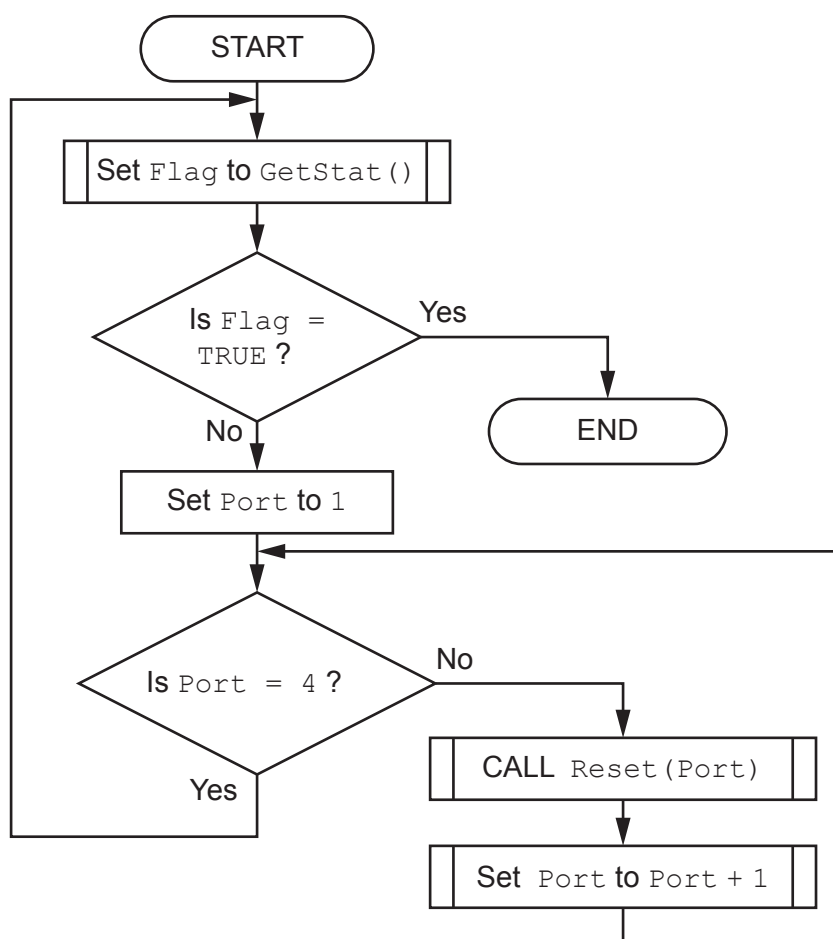
Assume the values are all different and are input in no particular order.

Complete the program flowchart on page 5 to represent the algorithm.



[5]

(b) A different part of the program contains an algorithm represented by the following program flowchart:



Write pseudocode for the algorithm.

[5]

31. M/J 24/P22/Q7/a

- 7** A fitness club has a computerised membership system.

The system stores information for each club member: name, home address, email address, mobile phone number, date of birth and exercise preferences.

Many classes are full, and the club creates a waiting list for each class. The club adds details of members who want to join a class that is full to the waiting list for that class.

When the system identifies that a space is available in one of the classes, a new module will send a text message to each member who is on the waiting list.

- (a)** Decomposition will be used to break the new module into sub-modules (sub-problems).

Identify **three** sub-modules that could be used in the design **and** describe their use.

Sub-module 1

Use

Sub-module 2

Use

Sub-module 3

Use

[3]

32. M/J 24/P22/Q8/a

- 8** A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
MichaelAday_src.txt	Student project file produced by student Michael Aday
MichaelAday_S1.txt	File produced by stage 1
MichaelAday_S2.txt	File produced by stage 2

- (a)** Suggest a reason why the teacher's program has been split into a number of stages **and** give the benefit of producing a different file from each stage.

Reason

Benefit

[2]

33. M/J 24/P23/Q2/a,b

- 2 (a) A program uses a global 1D array of type string and a text file.

An algorithm that forms part of the program is expressed as follows:

- copy the first line from the file into the first element of the array
- copy the second line from the file into the second element of the array
- continue until all lines in the file have been copied into the array.

Stepwise refinement is applied to the algorithm.

Outline **five** steps for this algorithm that could be used to produce pseudocode.

Assume there are more elements in the array than lines in the file.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

Step 5

[5]

- (b) Sequence is one programming construct.

Identify **one other** programming construct that will be required when the algorithm from part (a) is converted into pseudocode **and** explain its use.

Construct

Use

[2]

**34. M/J 24/P23/Q3/b**

- 3 A record structure is declared to hold data relating to components being produced in a factory:

```
TYPE Component
  DECLARE Item_ID : STRING
  DECLARE Reject : BOOLEAN
  DECLARE Weight : REAL
ENDTYPE
```

The factory normally produces a batch (or set) of 1000 components at a time. A global array is declared to store 1000 records for a batch:

```
DECLARE Batch : ARRAY [1:1000] OF Component
```

Two global variables contain the minimum and maximum acceptable weight for each component. The values represent an inclusive range and are declared as:

```
DECLARE Min, Max : REAL
```

(b) A module `InRange()` will:

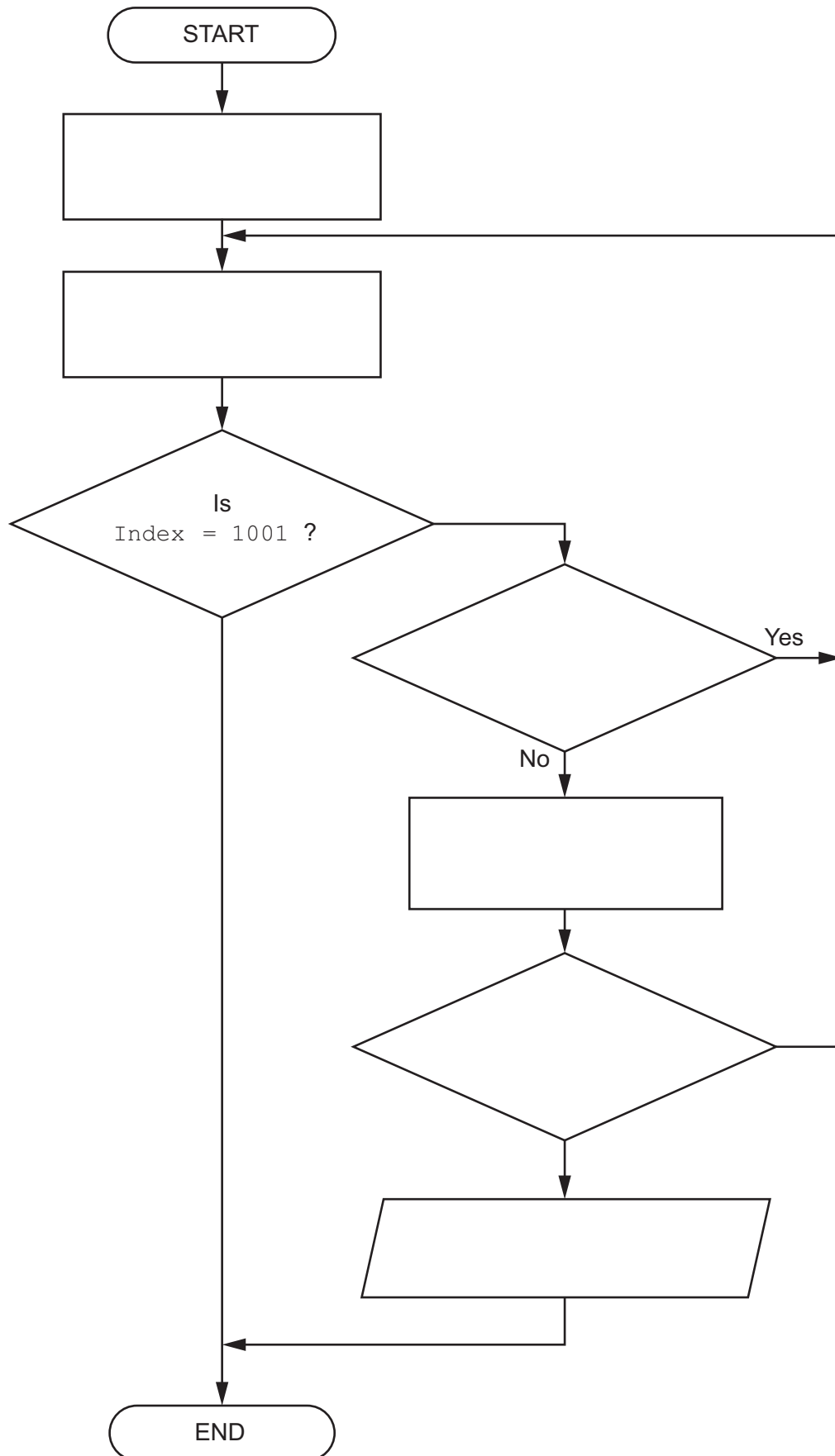
- be called with an integer parameter representing an index value of a record in the `Batch` array
- check if the weight of the indexed component is within the acceptable range
- return `TRUE` if the weight is in the range and `FALSE` if it is **not**.

A module `BatchCheck()` will:

- iterate through a batch of 1000 component records
- call module `InRange()` to check each individual component record
- keep a count of the number of components that fail
- output a suitable warning message and immediately stop if the number of failed components exceeds 5.



Complete the program flowchart to represent the algorithm for module `BatchCheck()`.



35. O/N 23/P21/Q2/a,b

2 (a) An algorithm is expressed as follows:

- input 100 numbers, one at a time
- keep a total of all numbers input that have a value between 30 and 70 inclusive and output this total after the last number has been input.

Outline, using stepwise refinement, the five steps for this algorithm which could be used to produce pseudocode.

Do **not** use pseudocode statements in your answer.

Step 1

Step 2

Step 3

Step 4

Step 5

[5]

(b) Sequence is one programming construct.

Identify **two other** programming constructs that will be required when the algorithm is converted into pseudocode.

Construct 1

Construct 2

[2]

36. O/N 23/P22/Q1/a

- 1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(a) The following identifier table shows some of the data that will be stored for each order.

Complete the identifier table by adding meaningful variable names and appropriate data types.

Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer		
3	The number of items in the order		
TRUE	To indicate whether this is a new customer		
15.75	The deposit paid by the customer		

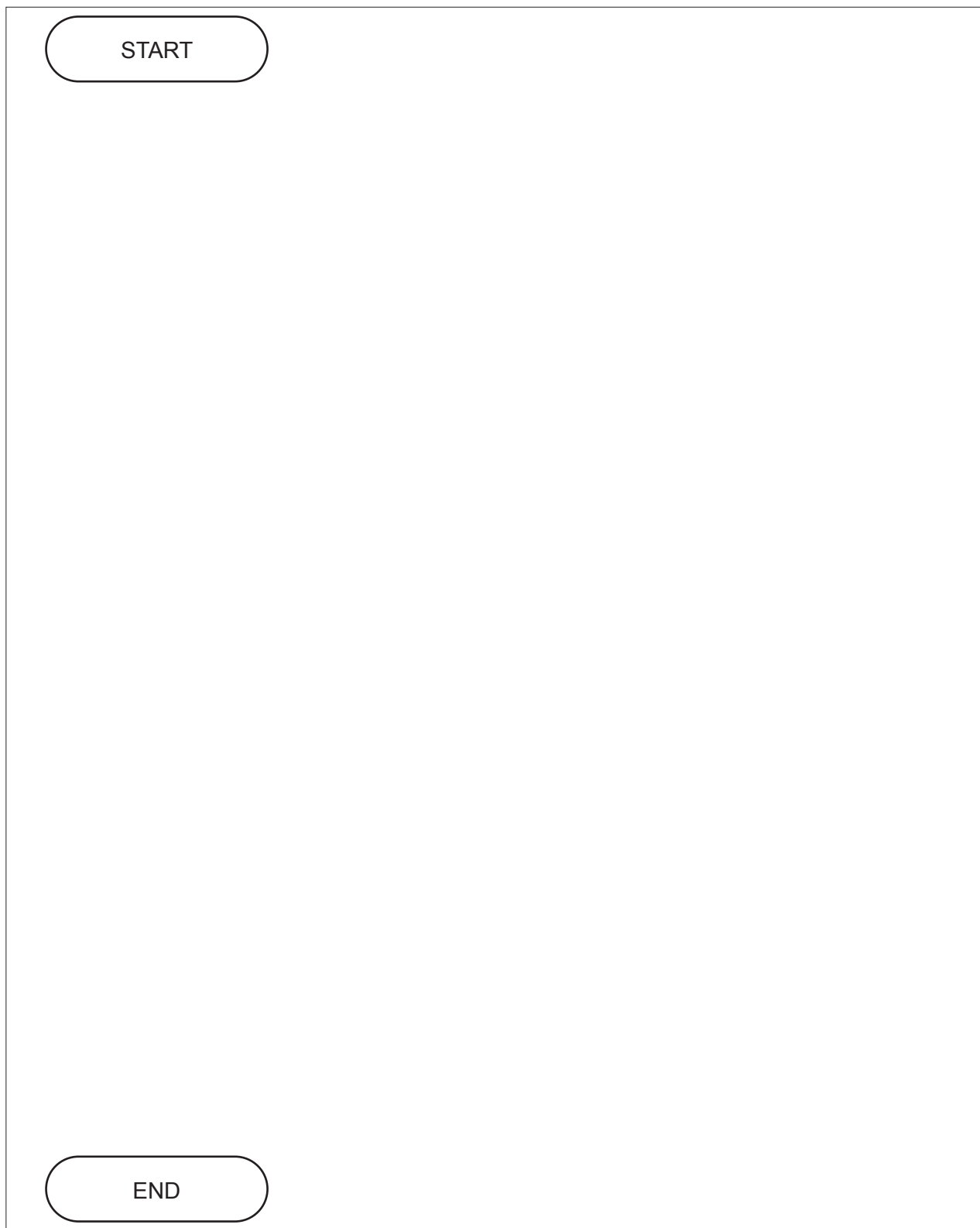
[4]

37. O/N 23/P22/Q2/a

- 2 An algorithm will:

1. input a sequence of integer values, one at a time
2. ignore all values until the value 27 is input, then sum the remaining values in the sequence
3. stop summing values when the value 0 is input and then output the sum of the values.

(a) Draw a program flowchart to represent the algorithm.



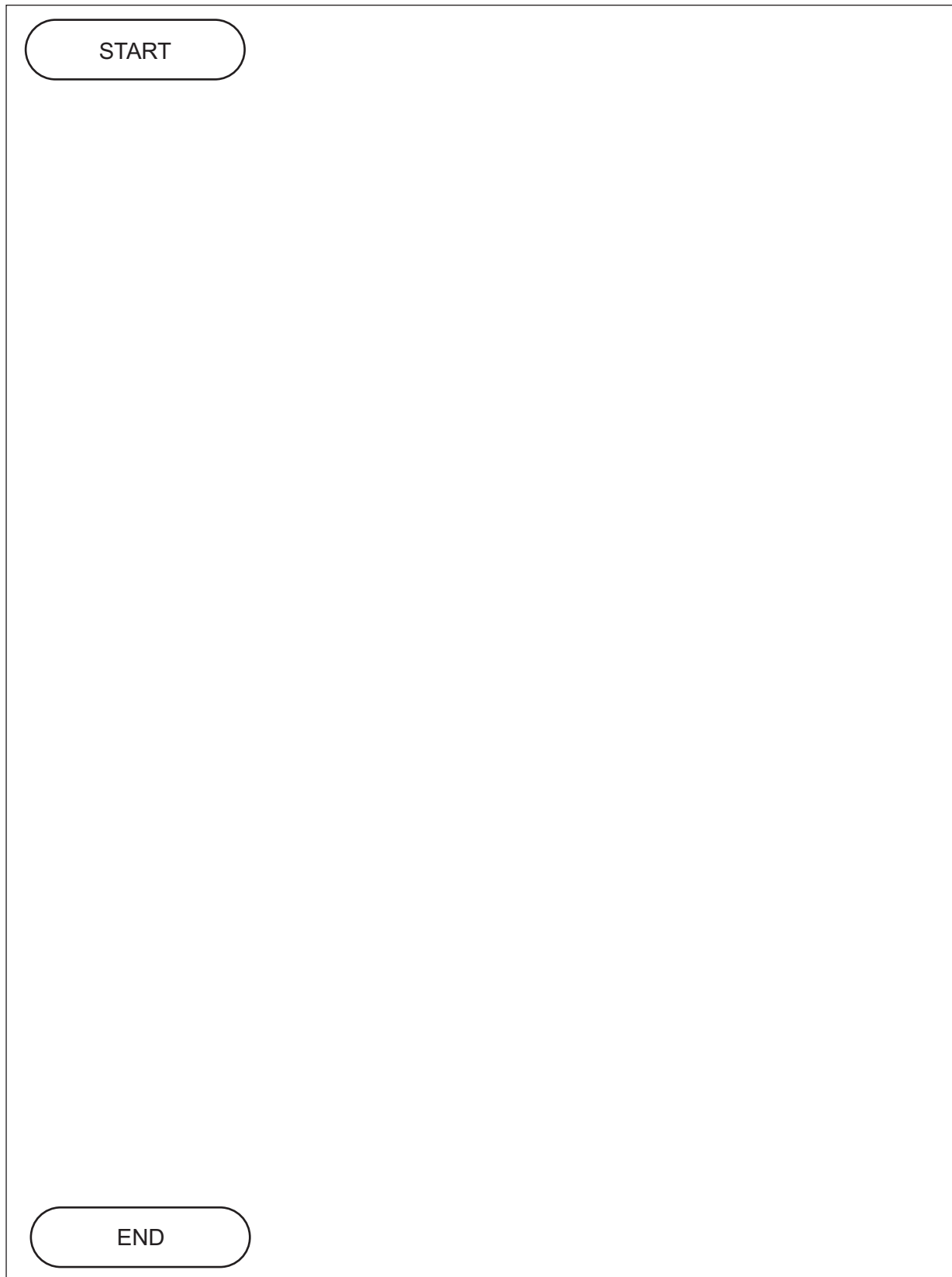
[5]

38. O/N 23/P23/Q2/a

2 Data is a 1D array of integers, containing 30 elements. All element values are unique.

(a) An algorithm will output the index of the element with the smallest value.

Draw a program flowchart to represent the algorithm.



[5]

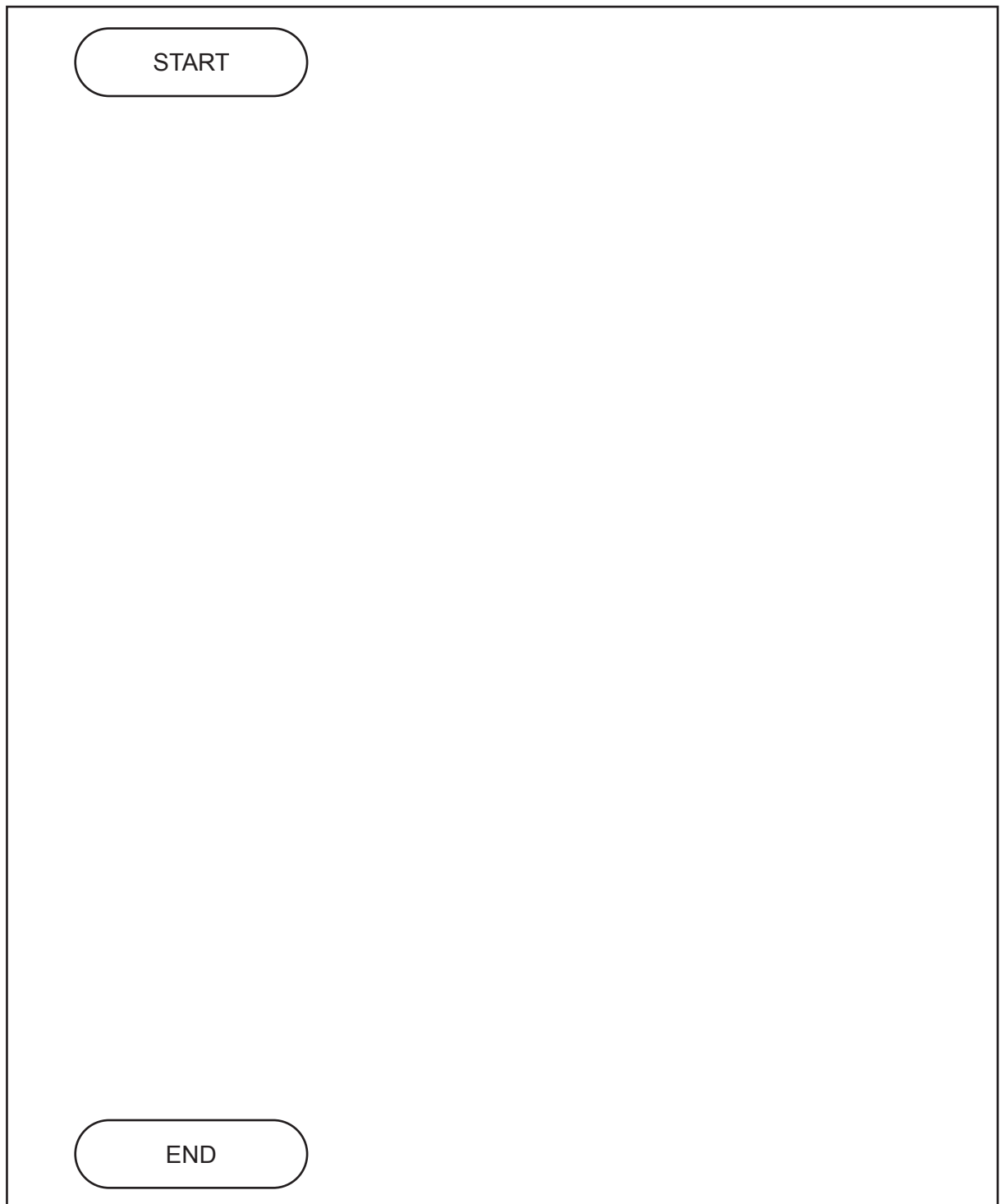
39. M/J 23/P21/Q2/a

- 2 A program contains an algorithm to output a string of a specified length containing identical characters.

(a) The algorithm is described as follows:

1. prompt and input a character and store in `MyChar`
2. prompt and input an integer and store in `MyCount`
3. generate a string consisting of `MyChar` repeated `MyCount` times
4. output the string.

Draw a program flowchart to represent the algorithm.



40. M/J 23/P21/Q3/b

- 3** Customers collect points every time they make a purchase at a store.

A program is used to manage the points system and the table lists some of the information stored for one customer.

Information	Data type required
Name	String
Number of points collected	Integer
Date of birth	Date

- (b)** Customers receive points depending on the amount they spend. The number of points depends on the band that the amount falls into:

Band	Amount	Points
1	Less than \$10	5 per whole dollar (\$)
2	Between \$10 and \$100 inclusive	7 per whole dollar (\$)
3	Over \$100	10 per whole dollar (\$)

For example, if the amount is \$99.77, this amount is in band 2 and therefore the number of points is 7×99 , which is 693 points.

The algorithm to calculate the points from a given amount is expressed as follows:

- work out the appropriate band
- calculate and output the number of points.

Apply the process of stepwise refinement to increase the detail of the algorithm. Structure your algorithm into a sequence of five steps that could be used to produce pseudocode.

Write the **five** steps.

- 1
- 2
- 3
- 4
- 5

41. M/J 23/P21/Q7/a

- 7 A school has a computerised library system that allows students to borrow books for a fixed length of time. The system uses text files to store details of students, books and loans.

A new module is to be written which will generate emails to each student who has an overdue book.

- (a) Decomposition will be used to break down the problem of designing the new module into sub-problems.

Identify **three** program modules that could be used in the design **and** describe their use.

Module 1

Use

Module 2

Use

Module 3

Use

[3]

42. M/J 23/P22/Q1/b

- 1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of \$3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
    ItemPostalCost ← 3.75      // set postal cost for item to $3.75
    ItemStatus ← "Valid"      // item can be sent
ENDIF
```

- (b) The lines of pseudocode contain features that make them easier to understand.

State **three** of these features.

1

2

3

[3]

43. M/J 23/P22/Q2/c

2 A program stores a user's date of birth using a variable `MyDOB` of type `DATE`.

(c) The program will output the day of the week corresponding to `MyDOB`.

For example, given the date 22/06/2023, the program will output "Thursday".

An algorithm is required. An array will be used to store the names of the days of the week.

Define the array **and** describe the algorithm in **four** steps.

Do **not** use pseudocode statements in your answer.

Array definition

Step 1

Step 2

Step 3

Step 4

[6]

44. M/J 23/P22/Q3/b

3 A program stores data in a text file. When data is read from the file, it is placed in a queue.

(b) The design of the queue is completed and the number of locations is increased.

A function `AddToQueue()` has been written. It takes a string as a parameter and adds this to the queue. The function will return `TRUE` if the string was added successfully.

A procedure `FileToQueue()` will add each line from the file to the queue. This procedure will end when all lines have been added or when the queue is full.

Describe the algorithm for procedure `FileToQueue()`.

Do **not** use pseudocode in your answer.

[5]

45. M/J 23/P22/Q7/a

- 7** A computer system for a shop stores information about each customer. The items of information include name and address (both postal and email) together with payment details and order history. The system also stores the product categories they are interested in and how they would like to be contacted.

- (a)** The shop wants to add a program module that will generate emails to be sent to customers who may be interested in receiving details of new products.

- (i)** State **three** items of information that the new module would need. Justify your choice in each case.

Information

Justification

Information

Justification

Information

Justification

[3]

- (ii)** Identify **two** items of customer information that would **not** be required by the new module. Justify your choice in each case.

Information

Justification

Information

Justification

[2]

46. M/J 23/P23/Q1/d

- 1 The following pseudocode represents part of the algorithm for a program.

Line numbers are for reference only.

```

10  DECLARE Sheet4 : ARRAY[1:2, 1:50] OF INTEGER
...
100 FOR PCount ← 0 TO 49
101     Sheet4[1, PCount] ← 0
102     Sheet4[2, PCount] ← 47
103 NEXT PCount

```

- (d) The pseudocode statements given in the following table are used in other parts of the algorithm.

Complete the table by placing **one or more** ticks (✓) in each row.

The first row has already been completed.

Pseudocode statement	Input	Process	Output
INPUT MyChoice	✓		
OUTPUT FirstName & LastName			
WRITEFILE YourFile, TextLine			
READFILE MyFile, TextLine			
Result ← SQRT (NextNum)			

[4]

47. M/J 23/P23/Q3/b(ii)

- 3 A program processes data using a stack. The data is copied to a text file before the program ends.

- (b) The data is copied to a text file before the program ends.

- (ii) A module `SaveStack()` will write the data from the stack to a text file.

Express an algorithm for `SaveStack()` as five steps that could be used to produce pseudocode.

Write the **five** steps.

Step 1

Step 2

Step 3

Step 4

Step 5

[5]

48. M/J 23/P23/Q7/a

- 7 A school has a library system which allows students to borrow books for a length of time. Information relating to students and books is stored in text files. Student information includes name, home address, email address, date of birth, tutor and subject choices. Book information includes author, title, subject category, library location and the date that the book was borrowed.

A program helps the staff to manage the borrowing of books.

- (a) A new module needs to be written to generate emails to send to students who have an overdue book. Students who are sent an email are prevented from borrowing any more books until the overdue book is returned.

The process of abstraction has been used when designing the new module.

- (i) State the purpose of applying abstraction to this problem.

[1]

- (ii) Identify **one** item of information that is required and **one** item that is **not** required in the new module. Justify your choices.

Item required

Justification

Item not required

Justification

[2]

- (iii) Identify **two** operations that would be required to process data when an overdue book is returned.

Operation 1

Operation 2

[2]

49. O/N 22/P21/Q2/a,b

- 2** A system is being developed to help manage a car hire business. A customer may hire a car for a number of days.

An abstract model needs to be produced.

- (a)** Explain the process of abstraction **and** state **four** items of data that should be stored each time a car is hired.

Explanation

Item 1

Item 2

Item 3

Item 4

[3]

- (b)** Identify **two** operations that would be required to process the car hire data.

Operation 1

Operation 2

[2]

50. O/N 22/P21/Q3

- 3** A 1D array `Data` of type integer contains 200 elements. Each element has a unique value.

An algorithm is required to search for the largest value and output it.

Describe the steps that the algorithm should perform.

Do **not** include pseudocode statements in your answer.

[5]

51. O/N 22/P22/Q2/a

also Unit 10 (2 marks); this unit 5 marks

- 2 (a)** An algorithm will process data from a test taken by a group of students. The algorithm will prompt and input the name and test mark for each of the 35 students.

The algorithm will add the names of all the students with a test mark of less than 20 to an existing text file `Support_List.txt`, which already contains data from other group tests.

- (i) Describe the steps that the algorithm should perform.

Do **not** include pseudocode statements in your answer.

[5]

- (ii) Explain why it may be better to store the names of the students in a file rather than in an array.

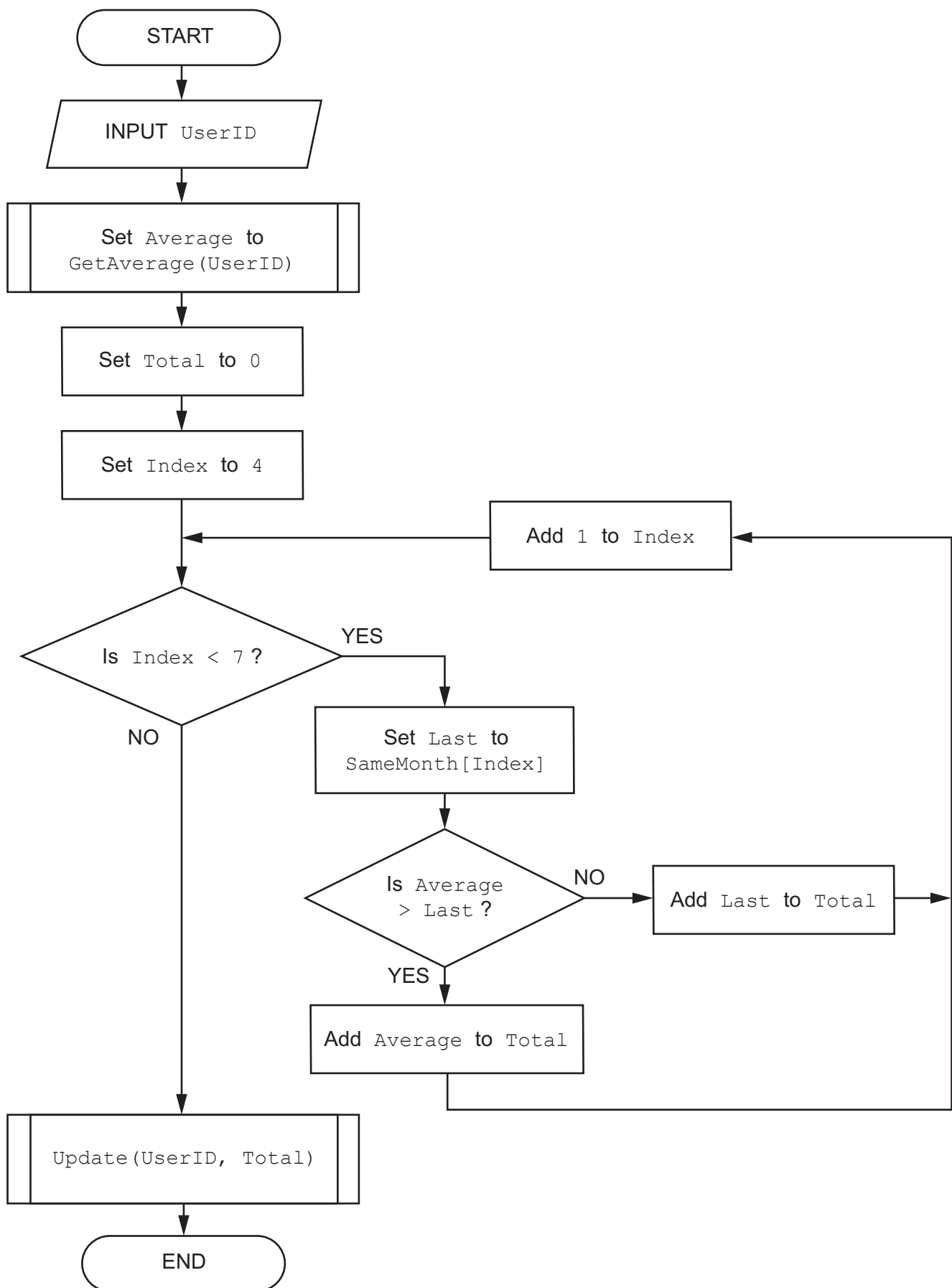
[1]

- (iii) Explain why `WRITE` mode cannot be used in the answer to **part 2(a)(i)**.

[1]

52. O/N 22/P22/Q4/a

- 4** The program flowchart represents a simple algorithm.



(a) Write the equivalent pseudocode for the algorithm represented by the flowchart.

53. O/N 22/P22/Q5

5 Examine the following pseudocode.

```
IF A = TRUE THEN
  IF B = TRUE THEN
    IF C = TRUE THEN
      CALL Sub1 ()
    ELSE
      CALL Sub2 ()
    ENDIF
  ENDIF
ELSE
  IF B = TRUE THEN
    IF C = TRUE THEN
      CALL Sub4 ()
    ELSE
      CALL Sub3 ()
    ENDIF
  ELSE
    IF C = FALSE THEN
      CALL Sub3 ()
    ELSE
      CALL Sub4 ()
    ENDIF
  ENDIF
ENDIF
```

A programmer wants to re-write the pseudocode as **four** separate IF...THEN...ENDIF statements, each containing a single CALL statement. This involves writing a single, simplified logic expression as the condition in each statement.

Write the amended pseudocode.

- 1
- 2
- 3
- 4

[4]

54. O/N 22/P23/Q2

- 2 A program is being designed for a smartphone to allow users to send money to the charity of their choice.

Decomposition will be used to break the problem down into sub-problems.

Identify **three** program modules that could be used in the design **and** describe their use.

Module 1

Use

Module 2

Use

Module 3

Use

[3]

55. O/N 22/P23/Q3

- 3 An algorithm is needed to process a sequence of numbers. Numbers may be positive or negative and may be integer or decimal.

The algorithm will:

- prompt and input one number at a time until the value zero is input
- sum the negative numbers
- sum the positive numbers
- when zero is input, output the two sum values and then end.

Describe the algorithm needed. Do **not** include pseudocode statements in your answer.

[5]

56. O/N 22/P23/Q5/c

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (c) `Unpack()` is part of a program made up of several modules. During the design stage, it is important to follow good programming practice. One example of good practice is the use of meaningful identifier names.

Give the reason why this is good practice. Give **two other** examples of good practice.

Reason

Example 1

Example 2

[3]

57. O/N 22/P23/Q7/b(ii)

- 7 A teacher is designing a program to perform simple syntax checks on programs written by students.

Two global 1D arrays are used to store the syntax error data. Both arrays contain 500 elements.

- Array `ErrCode` contains integer values that represent an error number in the range 1 to 800.
- Array `ErrMsgText` contains string values that represent an error description.

The following diagram shows an example of the arrays.

Index	ErrCode	ErrMsgText
1	10	"Invalid identifier name"
2	20	"Bracket mismatch"
3	50	""
4	60	"Type mismatch in assignment"
...		
500	999	<Undefined>

Note:

- There are less than 500 error codes so corresponding elements in both arrays may be unused. Unused elements in `ErrCode` have the value 999. These will occur at the end of the array. The value of unused elements in `ErrText` is undefined.
- Values in the `ErrCode` array are stored in ascending order but not all values may be present. For example, there may be no error code 31.
- Some error numbers are undefined. In these instances, the `ErrCode` array will contain a valid error number but the corresponding `ErrText` element will contain an empty string.

The teacher has defined one program module as follows:

Module	Description
<code>OutputRange()</code>	• Prompts for input of two error numbers • Outputs a list of error numbers between the two numbers input (inclusive) together with the corresponding error description • Outputs a warning message when the error description is missing as for error number 50 in the example • Outputs a suitable header and a final count of error numbers found Output based on the example array data above: <pre>List of error numbers from 1 to 60 10 : Invalid identifier name 20 : Bracket mismatch 50 : Error Text Missing 60 : Type mismatch in assignment 4 error numbers output</pre>

- (ii) A new Module `RemoveError()` will remove a given error number from the array.

Describe the algorithm that would be required. Do **not** include pseudocode statements in your answer.

58. O/N 21/P21/Q1/c

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.
- (c) An airline wants to provide passengers with information about individual flights and allow them to book their flight using an online booking system.
- (i) Tick (✓) **one** box in each row of the table to indicate whether each item of information would be essential for the customer when making the booking.

Information	Essential	Not essential
Departure time		
Flight number		
Departure airport		
Aircraft type		
Ticket price		
Number of seats in aircraft		

[3]

- (ii) Identify the technique used to filter out information that is not essential when designing the booking system **and** state one benefit of this technique.

Technique

Benefit

[2]

- (iii) Identify **two additional** pieces of essential information that a passenger might need when booking a flight.

1

2

[2]

59. O/N 21/P21/Q2/a,b

- 2 (a) An algorithm to sort a 1D array into ascending order is described as follows:

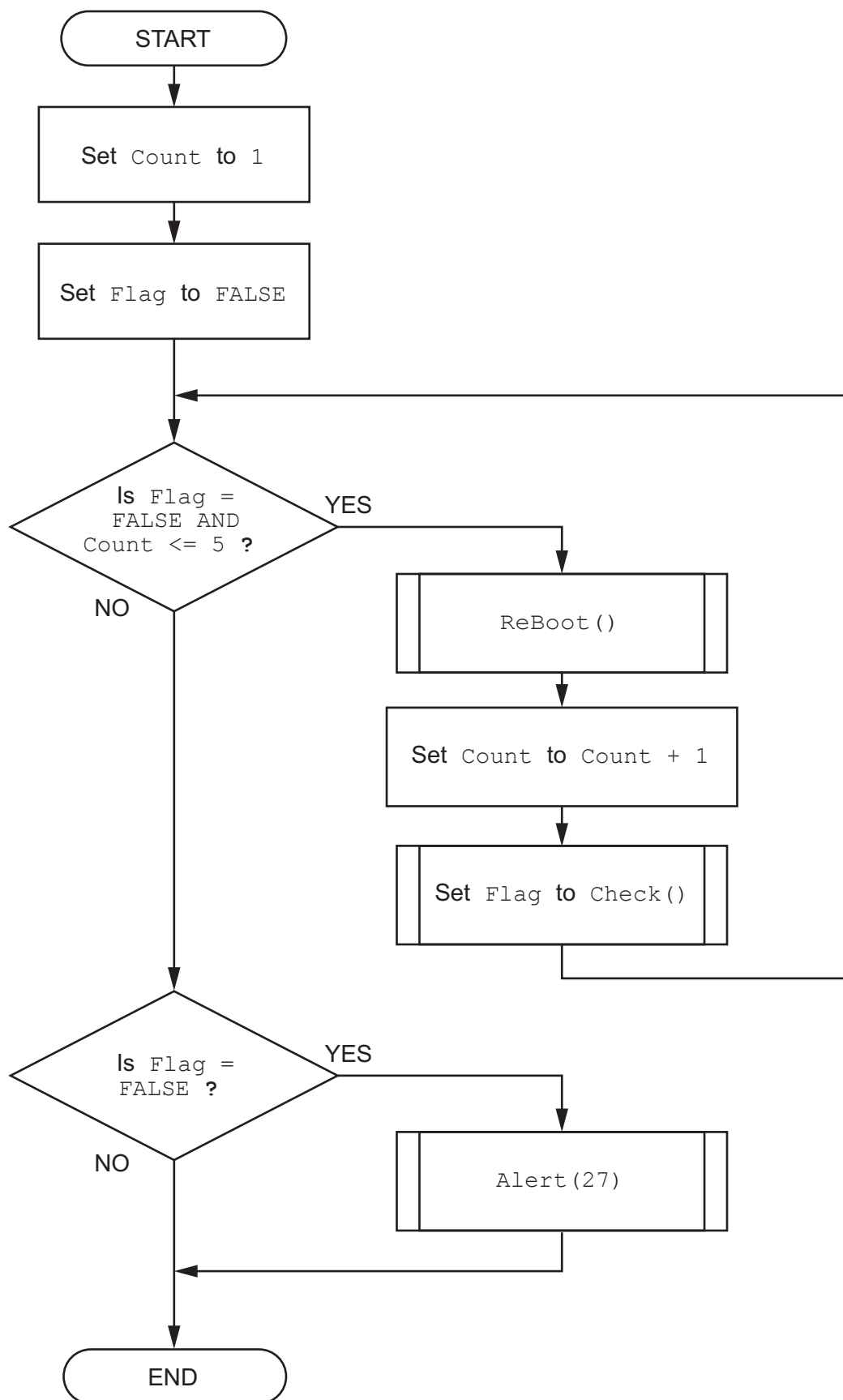
- move the largest value to the end
- keep repeating until the array is sorted.

Apply the process of stepwise refinement to this algorithm in order to produce a more detailed description.

Write the more detailed description using **structured English**. Your explanation of the algorithm should not include pseudocode statements.

[6]

- (b) The program flowchart shown describes a simple algorithm.



Write pseudocode for the simple algorithm shown on page 6.

60. O/N 21/P22/Q1/a

- 1 (a) A programmer applies decomposition to a problem that she has been asked to solve.

Describe decomposition.

[2]

61. O/N 21/P22/Q2/a

- 2 (a) An algorithm will:

1. input an integer value
2. jump to step 6 if the value is zero
3. sum and count the positive values
4. sum and count the negative values
5. repeat from step 1
6. output the two sum values and the two count values.

Draw a program flowchart on the following page to represent the algorithm.

Note that variable declarations are not required in program flowcharts.

[5]

62. O/N 21/P22/Q4/b

- 4** The following is a procedure design in pseudocode.

Line numbers are given for reference only.

```
10  PROCEDURE Check(InString : STRING)
11      DECLARE Odds, Evens, Index : INTEGER
12
13      Odds ← 0
14      Evens ← 0
15      Index ← 1
16
17      WHILE Index <= LENGTH(InString)
18          IF STR_TO_NUM(MID(InString, Index, 1)) MOD 2 <> 0 THEN
19              Odds ← Odds + 1
20          ELSE
21              Evens ← Evens + 1
22          ENDIF
23          Index ← Index + 1
24      ENDWHILE
25
26      CALL Result(Odds, Evens)
27  ENDPROCEDURE
```

- (b)** The pseudocode includes several features that make it easier to read and understand.

Identify **three** of these features.

1

2

3

[3]

63. O/N 21/P23/Q1/c

- 1 Sylvia is testing a program that has been written by her colleague. Her colleague tells her that the program does not contain any syntax errors.
- (c) An airline wants to provide passengers with information about individual flights and allow them to book their flight using an online booking system.
- (i) Tick (✓) **one** box in each row of the table to indicate whether each item of information would be essential for the customer when making the booking.

Information	Essential	Not essential
Departure time		
Flight number		
Departure airport		
Aircraft type		
Ticket price		
Number of seats in aircraft		

[3]

- (ii) Identify the technique used to filter out information that is not essential when designing the booking system **and** state one benefit of this technique.

Technique

Benefit

[2]

- (iii) Identify **two additional** pieces of essential information that a passenger might need when booking a flight.

1

2

[2]

64. O/N 21/P23/Q2/a,b

- 2 (a) An algorithm to sort a 1D array into ascending order is described as follows:

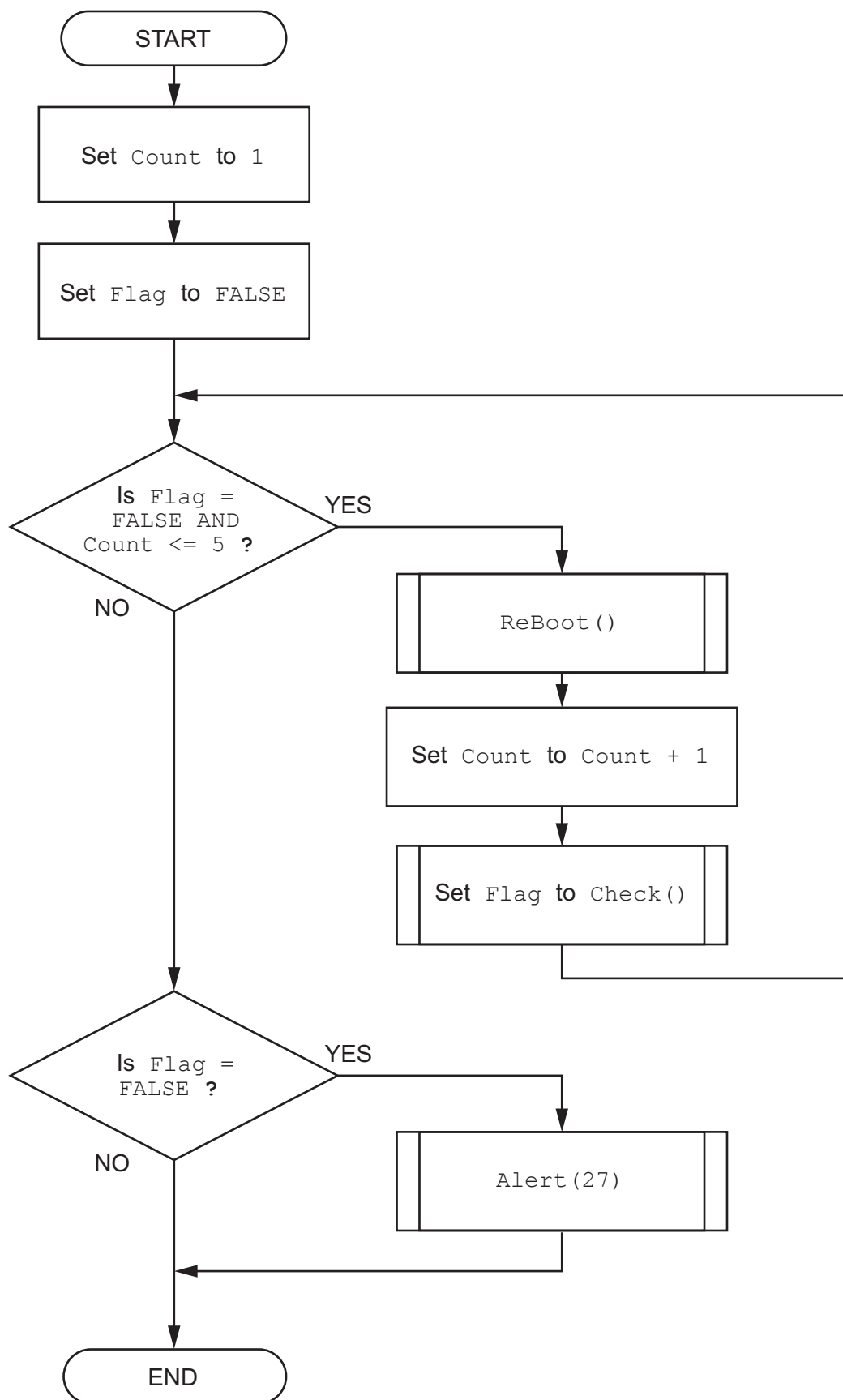
- move the largest value to the end
- keep repeating until the array is sorted.

Apply the process of stepwise refinement to this algorithm in order to produce a more detailed description.

Write the more detailed description using **structured English**. Your explanation of the algorithm should not include pseudocode statements.

[6]

- (b) The program flowchart shown describes a simple algorithm.



Write pseudocode for the simple algorithm shown on page 6.

65. M/J 21/P21/Q1/a

- 1 (a) A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
"Wong"	The preferred name of the member joining the football club		
FALSE	A value to indicate whether an existing member of the club lives at the same address		
19/02/1983	When the member joined the football club		
1345	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

66. M/J 21/P21/Q2/c

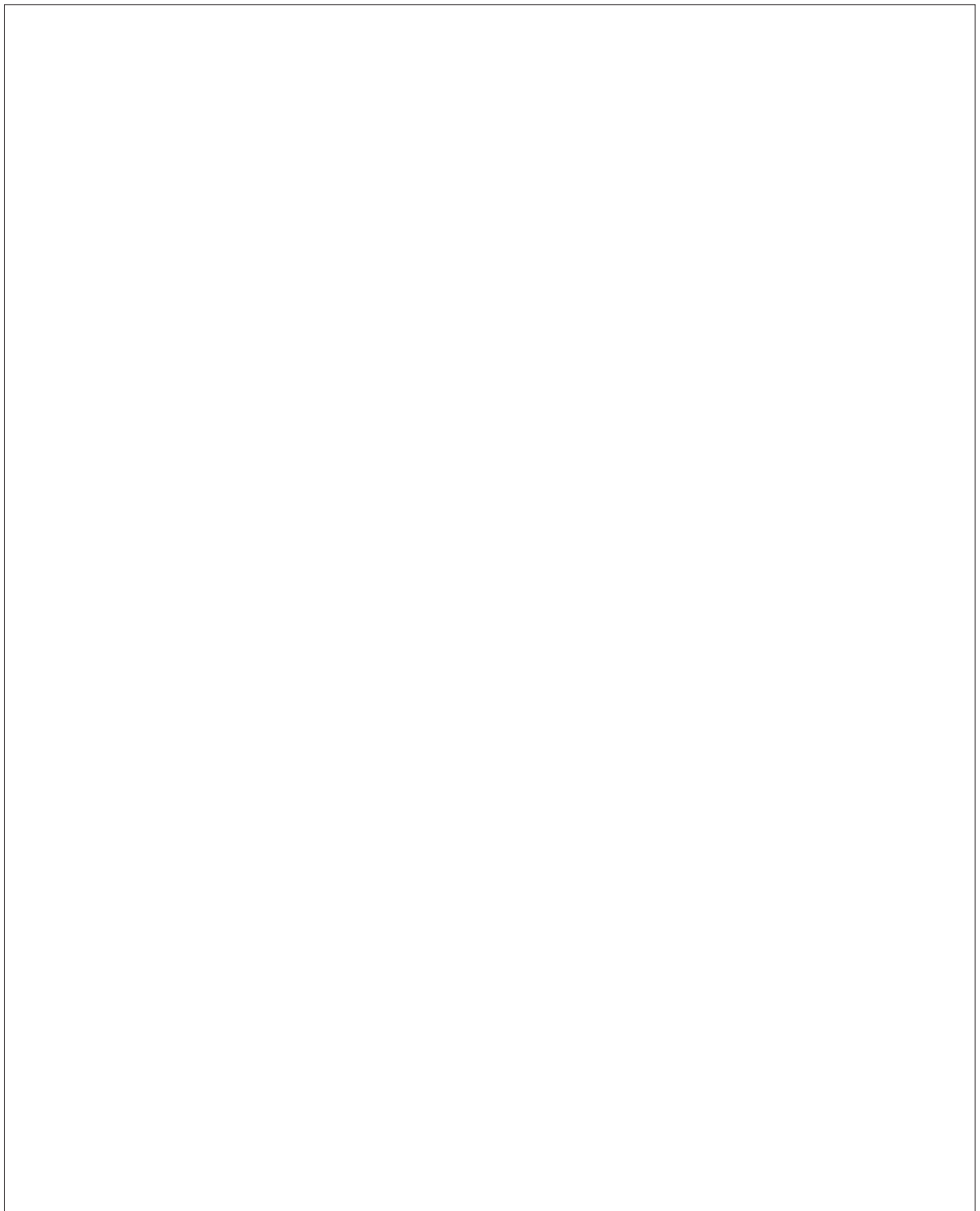
- (c) A program will:

- input 50 unique integer values
- output the largest value
- output the average of the values **excluding** the largest value.

Draw a program flowchart to represent the algorithm.

Variable declarations are **not** required.

It is not necessary to check that each input value is unique.



[6]

67. M/J 21/P21/Q7/b

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

- (b) The programmer has decided to use a global ten-element 1D array `IgnoreList` of type `STRING` to store the ignored words. Unused elements contain the empty string ("") and may occur anywhere in the array.

A new module `AddWord()` is needed as follows:

Module	Description
<code>AddWord()</code>	• Called with a parameter representing a word • Stores the word in an unused element of the <code>IgnoreList</code> array and returns <code>TRUE</code> • Returns <code>FALSE</code> if the array was already full or if the word was already in the array

Write a detailed description of the algorithm for `AddWord()`. Do **not** include pseudocode statements in your answer.

68. M/J 21/P22/Q1/b

- 1 (a) (i) Complete the following table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Name	"Catherine"	
Index	100	
Modified	FALSE	
Holiday	25/12/2020	

[4]

- (ii) Evaluate each expression in the following table by using the initial data values shown in part (a)(i).

Expression	Evaluates to
Modified OR Index > 100	
LENGTH("Student: " & Name)	
INT(Index + 2.9)	
MID(Name, 1, 3)	

[4]

- (b) Each pseudocode statement in the following table contains an example of selection, assignment or iteration.

Put **one** tick (✓) in the appropriate column for each statement.

Statement	Selection	Assignment	Iteration
Index ← Index + 1			
IF Modified = TRUE THEN			
ENDWHILE			

[3]

69. M/J 21/P22/Q2/b

(b) A system is being developed to help manage book loans in a library.

Registered users may borrow books from the library for a period of time.

(i) State **three** items of data that must be stored for each loan.

1

2

3

[2]

(ii) State **one** item of data that will be required in the library system but does not need to be stored for each loan.

[1]

(iii) One operation that manipulates the data stored for each loan, would produce a list of all overdue books.

Identify **two other** operations.

Operation 1

Operation 2

[2]

70. M/J 21/P22/Q4

4 A teacher uses a paper-based system to store marks for a class test. The teacher requires a program to assign grades based on these results.

The program will output the grades together with the average mark.

Write a detailed description of the algorithm that will be needed.

[6]

71. M/J 21/P23/Q1/a

- 1 (a) A program is being developed to help manage the membership of a football club.

Complete the following identifier table.

Example value	Explanation	Variable name	Data type
"Wong"	The preferred name of the member joining the football club		
FALSE	A value to indicate whether an existing member of the club lives at the same address		
19/02/1983	When the member joined the football club		
1345	The number of points a member has earned. Members of the club earn points for different activities.		

[4]

72. M/J 21/P23/Q2/c

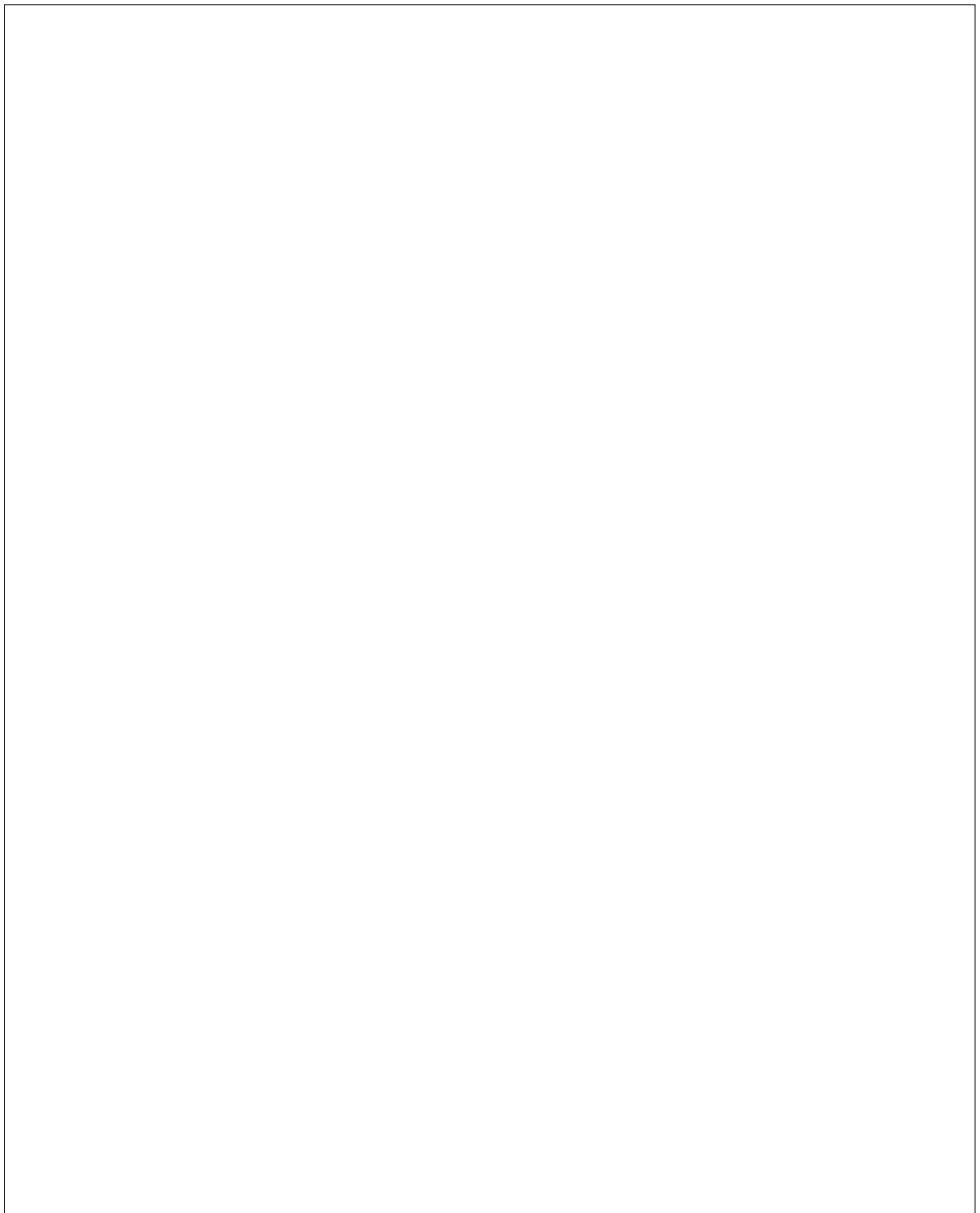
- (c) A program will:

- input 50 unique integer values
- output the largest value
- output the average of the values **excluding** the largest value.

Draw a program flowchart to represent the algorithm.

Variable declarations are **not** required.

It is not necessary to check that each input value is unique.



[6]

73. M/J 21/P23/Q7/b

7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

- (b) The programmer has decided to use a global ten-element 1D array `IgnoreList` of type `STRING` to store the ignored words. Unused elements contain the empty string ("") and may occur anywhere in the array.

A new module `AddWord()` is needed as follows:

Module	Description
<code>AddWord()</code>	• Called with a parameter representing a word • Stores the word in an unused element of the <code>IgnoreList</code> array and returns <code>TRUE</code> • Returns <code>FALSE</code> if the array was already full or if the word was already in the array

Write a detailed description of the algorithm for `AddWord()`. Do **not** include pseudocode statements in your answer.

74. 9608 M/J 21/P21/Q2/a,b

- 2 (a)** The following pseudocode algorithm counts the number of each alphabetic character in the string `Msg`. The character count values are stored in an array `CharCount`.

Variable declarations are not shown.

```
FOR Index ← 1 TO 26
    CharCount[Index] ← 0
ENDFOR

FOR Index ← 1 TO LENGTH(Msg)
    ThisChar ← MID(Msg, Index, 1)
    ThisChar ← LCASE(ThisChar)
    IF ThisChar >= 'a' AND ThisChar <= 'z'
        THEN
            ThisIndex ← ASC(ThisChar) - 96 // value from 1 to 26
            CharCount[ThisIndex] ← CharCount[ThisIndex] + 1
        ENDIF
    ENDFOR
```

Draw a program flowchart to represent the algorithm.

Variable declarations are not required in program flowcharts.

[5]

- (b)** The character count values have been assigned to the array in **part (a)**.

Use **structured English** to describe an algorithm to:

- search the array
- output the alphabetic character that occurs most often (the highest character count value)
- output a suitable message if more than one alphabetic character has the same highest count value.

[6]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost `x` characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`INT(x : REAL)` RETURNS INTEGER
returns the integer part of `x`

Example: `INT(27.5415)` returns 27

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
If `ThisChar` is not an upper case alphabetic character, it is returned unchanged.

Example: `LCASE('W')` returns 'w'

`ASC(ThisChar : CHAR)` RETURNS INTEGER
returns the ASCII value of character `ThisChar`

Example: `ASC('A')` returns 65

`RAND(x : INTEGER)` RETURNS REAL
returns a real number in the range 0 to `x` (not inclusive of `x`).

Example: `RAND(87)` could return 35.43

`NUM_TO_STRING(x : REAL)` RETURNS STRING
returns a string representation of a numeric value.
Note: This function will also work if `x` is of type INTEGER

Example: `NUM_TO_STRING(87.5)` returns "87.5"

75. 9608 M/J 21/P22/Q2/b

2 Study the following pseudocode.

```
DECLARE Overload : BOOLEAN
```



```
PROCEDURE LEM()
```

```
    DECLARE Status : BOOLEAN
```

```
    DECLARE Landed : INTEGER
```

```
    Overload ← FALSE
```

```
    Landed ← FALSE
```

```
    WHILE Landed = FALSE
```

```
        Status ← Sample()
```

```
        IF Status = TRUE
```

```
            THEN
```

```
                Landed ← SubA(42)
```

```
            ELSE
```

```
                Overload ← SubB(37)
```

```
                IF Overload = TRUE
```

```
                    THEN
```

```
                        CALL Display("Alarm 1202")
```

```
                    ENDIF
```

```
            ENDIF
```

```
    ENDWHILE
```

```
ENDPROCEDURE
```

(b) Draw a program flowchart to represent the pseudocode algorithm.

Variable declarations are not required in program flowcharts.

76. 9608 M/J 21/P23/Q1/a

- 1 (a) Algorithms usually consist of three different types of activity.

Complete the following table.

Write each example statement in **program code** and state the programming language used.

The third activity has already been given.

Activity	Example statement in program code	Programming language
OUTPUT		

[5]

77. 9608 M/J 21/P23/Q2/b

- (b) Good programming practice may help to avoid faults. The use of sensible identifier names is one example of good practice.

- (i) Explain the reason for using sensible identifier names.

[1]

- (ii) State **three other** examples of good programming practice.

1

2

3

[3]

78. 9608 M/J 21/P23/Q3/a,b,c

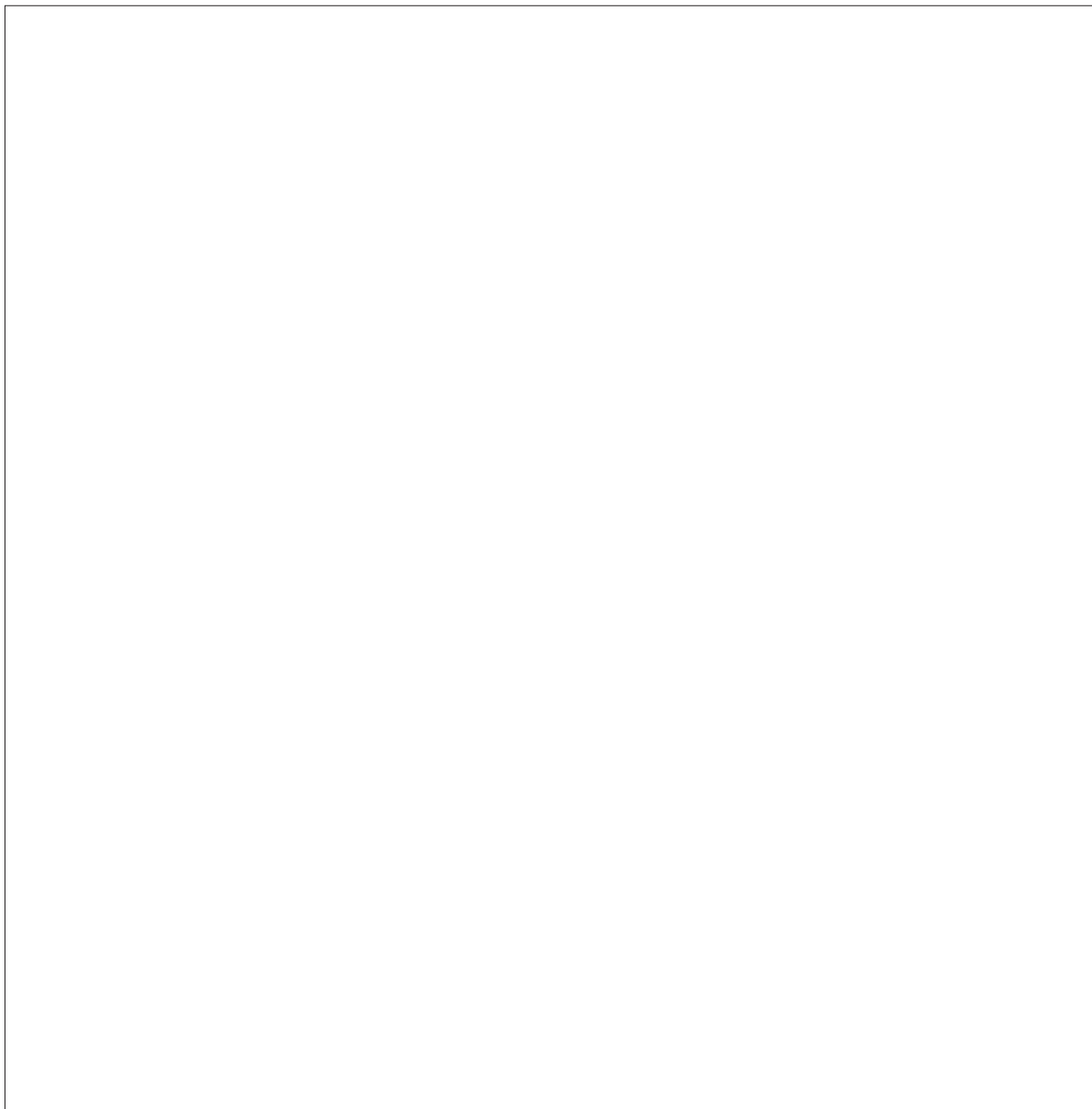
- 3 (a) The process of decomposition is often applied to a programming problem.

Describe the process of decomposition.

[2]

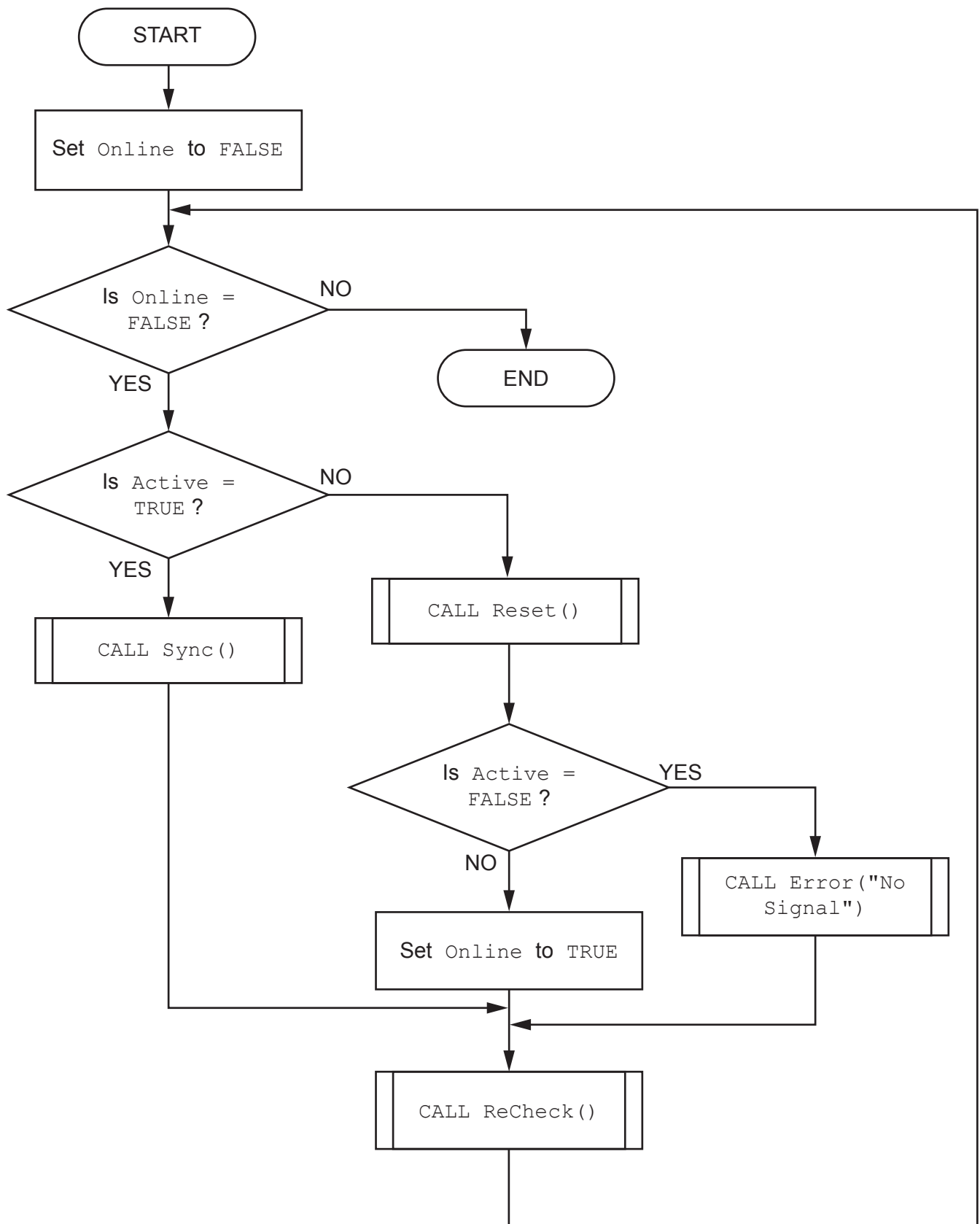
- (b) `Result` is a 1D array of type `STRING`. It contains 100 elements.

Draw a program flowchart for an algorithm that will output each element in the array.



[4]

- (c) The program flowchart for part of an algorithm from a mobile phone program is shown. Identifier `Active` is a global variable of type `BOOLEAN`.



Write **pseudocode** to implement the algorithm represented by the flowchart.

79. M/J 20/P21/Q1/a,c

- 1 (a) Complete this definition of the term **algorithm**.

An algorithm is a solution to a problem expressed as

[2]

- (c) Draw lines on the following diagram to connect each computing term with the appropriate description.

Term	Description
Selection	Checking that a program performs as expected
Black-box testing	A method for increasing the level of detail of an algorithm
Stepwise refinement	To test a condition to determine the path of program execution
Iteration	A method of executing certain lines of code more than once

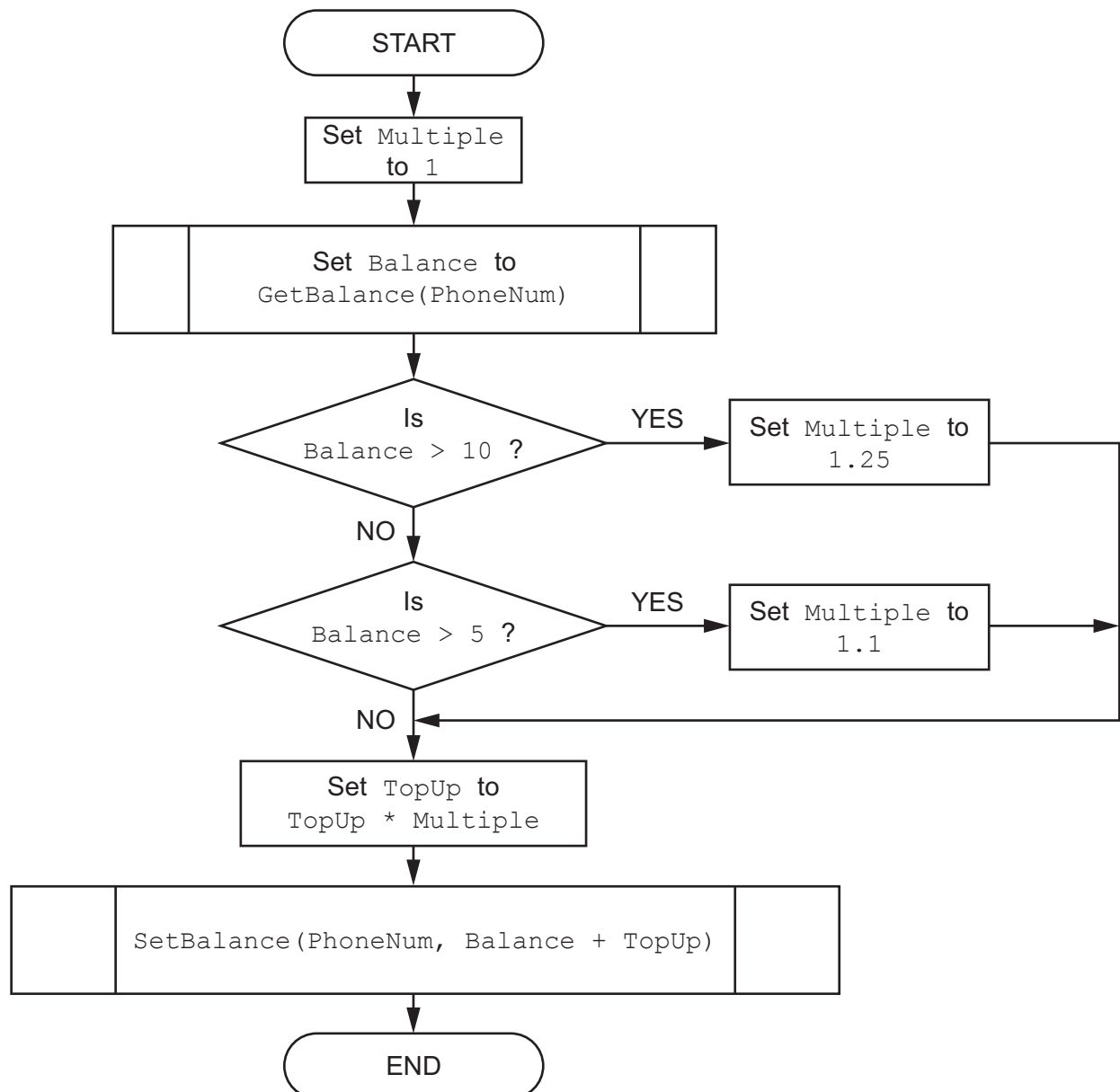
[3]

80. M/J 20/P21/Q3/a

- 3 (a) A mobile phone provider has developed an account management program. The program includes a procedure, `AddCredit()`. The procedure is called with two parameters, `TopUp` and `PhoneNum`.

The relevant part of the identifier table and the program flowchart for the procedure are as shown:

Identifier	Type	Description
<code>TopUp</code>	REAL	The amount of credit to be added
<code>PhoneNum</code>	STRING	The unique customer phone number
<code>Balance</code>	REAL	The current amount of credit
<code>Multiple</code>	REAL	The amount of credit bonus
<code>GetBalance()</code>	FUNCTION	Takes the phone number as a parameter and returns the current balance
<code>SetBalance()</code>	PROCEDURE	Takes the phone number and the new balance as parameters and updates the account with the new balance



Write **pseudocode** to implement the procedure `AddCredit()`. The pseudocode must follow the algorithm represented by the flowchart. Declare any local variables used.

[7]

81. M/J 20/P22/Q1/a

- 1 (a) Selection and repetition are basic constructs of an algorithm.

Name **and** describe **one other** construct.

Name

Description

[3]

82. M/J 20/P22/Q2/b(i)

- (b) Six program modules implement part of an online shopping program. The following table gives the modules and a brief description of each module:

Module	Description
Shop ()	Allows the user to choose a delivery slot, select items to be added to the basket and finally check out
ChooseSlot ()	Allows the user to select a delivery time. Returns a delivery slot number
FillBasket ()	Allows the user to select items and add them to the basket
Checkout ()	Completes the order by allowing the user to pay for the items. Returns a Boolean value to indicate whether or not payment was successful
Search ()	Allows the user to search for a specific item. Returns an item reference
Add ()	Adds an item to the basket. Takes an item reference and a quantity as parameters

- (i) The online shopping program has been split into sub-tasks as part of the design process.

Explain the advantages of decomposing the program into modules. Your explanation should refer to the scenario and modules described in **part (b)**.

[3]

83. M/J 20/P22/Q5/a

- 5 A global 1D array, `Contact`, of type `STRING` is used to store a list of names and email addresses.

There are 1000 elements in the array. Each element stores one data item. The format of each data item is as follows:

`<Name>': '<EmailAddress>`

`Name` and `EmailAddress` are both variable-length strings.

For example:

`"Sharma Himel:hsharma99@stlmail.com"`

A function, `GetName ()`, is part of the program that processes the array. A data item string will be passed to the function as a parameter. The function will return the `Name` part. Validation is **not** necessary.

- (a) Use **structured English** to describe the algorithm for the function `GetName ()`.

[7]

84. M/J 20/P23/Q1/a,b,c

- 1 (a) Algorithms are produced during program development.

State when you would produce an algorithm during program development **and** state its purpose.

When

Purpose

[2]

- (b) Selection is one of the basic constructs used in algorithms.

Explain the term **selection**.

[2]

- (c) Explain the process of **problem decomposition**. State one reason it may be used.

Explanation

Reason

[2]

85. M/J 20/P23/Q2/c

- (c) A student is developing an algorithm to count the number of times a given string, *SearchString*, appears in a 2D array. The array, *Item*, consists of 100 elements organised as 50 rows of 2 columns. *SearchString* could appear in any row or column.

The array is declared in pseudocode as follows:

```
DECLARE Item : ARRAY [1:50, 1:2] OF STRING
```

The structured English description of the algorithm is:

1. SET Count to 0.
2. Examine the first row of the array.
3. IF column 1 element value is equal to *SearchString*, ADD 1 to Count.
4. IF column 2 element value is equal to *SearchString*, ADD 1 to Count.
5. REPEAT from step 3 for next row, UNTIL row is last row.
6. OUTPUT a suitable message and Count.

Write **pseudocode** for the algorithm.

[5]

86. M/J 20/P23/Q3/a

- 3 (a) Study the following pseudocode.

```
Error ← Delta(Plan, Actual)
CASE OF Error
  > 10 : Steer ← Steer - 10
    0 : ZCount ← ZCount + 1
  < -10 : Steer ← Steer + 10
  OTHERWISE OUTPUT "Unexpected Error"
ENDCASE
```

Draw a program flowchart to represent the pseudocode. Variable declarations are **not** required in program flowcharts.

87. O/N 19/P21/Q1/a

- 1** Study the following pseudocode.

```
FUNCTION Search() RETURNS INTEGER
  DECLARE N, C : INTEGER
  DECLARE V, L : REAL
  V ← GetLevel()
  L ← V * 1.34
  C ← 0
  FOR N ← 1 TO 10
    V ← GetLevel()
    IF V > L
      THEN
        C ← C + 1
      ENDIF
  ENDFOR
  OUTPUT "Process complete"
  RETURN C
ENDFUNCTION
```

- (a) (i)** This pseudocode lacks features that would make it easier to read and understand.

State **three** such features.

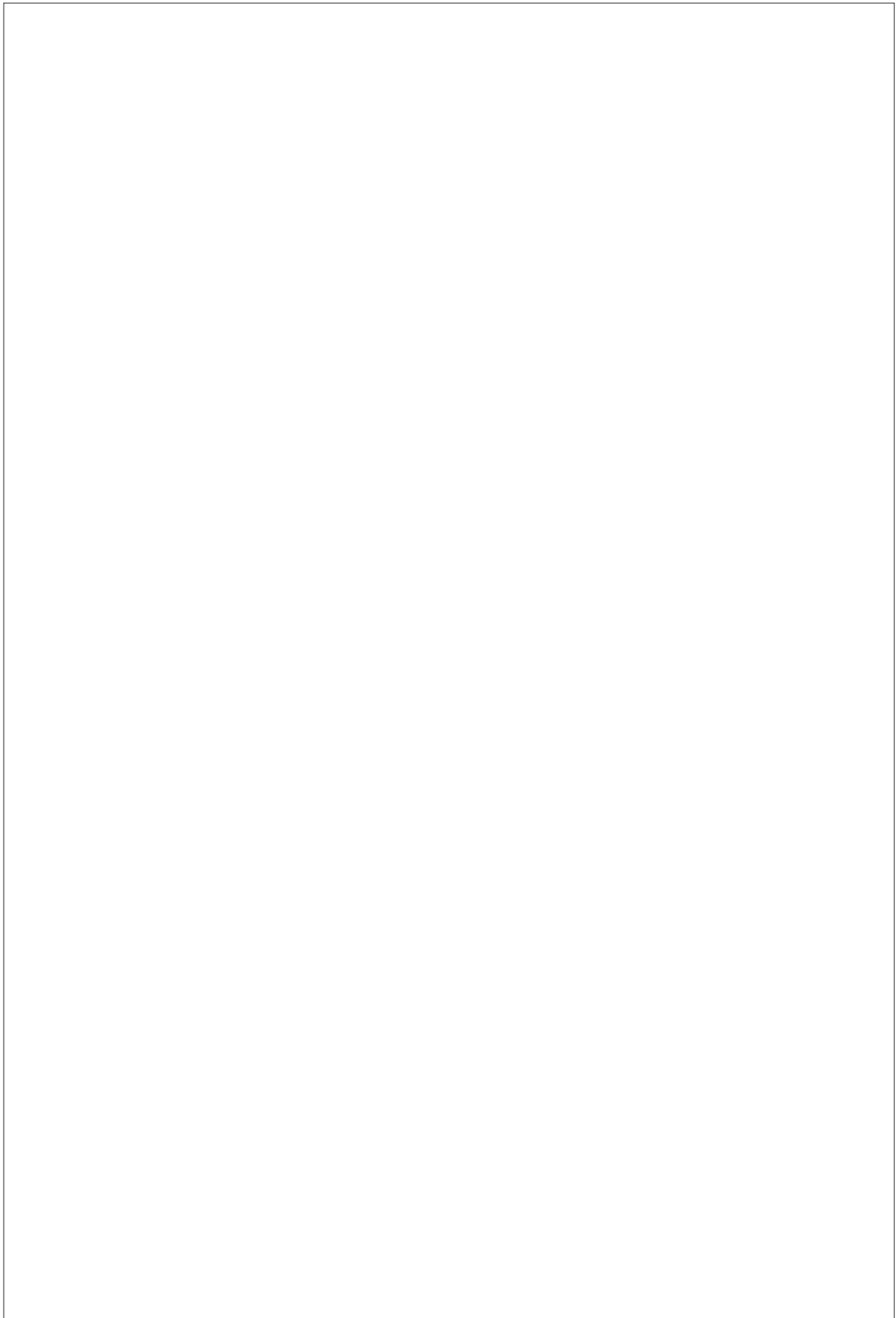
Feature 1

Feature 2

Feature 3

[3]

- (ii)** Draw a program flowchart to represent the algorithm implemented in the pseudocode. Variable declarations are not required in program flowcharts.



88. O/N 19/P21/Q4/a

- 4 A student is developing a program to count how many times each character of the alphabet (A to Z) occurs in a given string. Upper case and lower case characters will be counted as the same. The string may contain non-alphabetic characters, which should be ignored.

The program will:

- check each character in the string to count how many times each alphabetic character occurs
- store the count for each alphabetic character in a 1D array
- output each count together with the corresponding character.

(a) The student has written a structured English description of the algorithm:

1. START at the beginning of the string
2. SELECT a character from the string
3. CONVERT the character to upper case
4. CHECK whether the character is alphabetic and INCREMENT as required.
5. REPEAT from step 2 until last character has been checked
6. OUTPUT a suitable message giving the count of each alphabetic character

Step 4 above is not described in sufficient detail.

The student decides to apply a process to increase the level of detail given in step 4.

State the name of the process **and** use this process to write step 4 in more detail. Use **structured English** for your answer.

Process

Structured English

[4]

89. O/N 19/P22/Q1/a

- 1 Study the following pseudocode.

```
PROCEDURE FillTank()
```

```
    DECLARE Tries : INTEGER  
    DECLARE Full : BOOLEAN
```

```
Tries ← 1

Full ← ReadSensor("F1")

IF NOT Full
  THEN
    WHILE NOT Full AND Tries < 4
      CALL TopUp()
      Full ← ReadSensor("F1")
      Tries ← Tries + 1
    ENDWHILE
    IF Tries > 3
      THEN
        OUTPUT "Too many attempts"
      ELSE
        OUTPUT "Tank now full"
      ENDIF
    ELSE
      OUTPUT "Already full"
    ENDIF
  ENDIF
ENDIF

ENDPROCEDURE
```

- (a) (i)** The pseudocode includes features that make it easier to read and understand.

State **three** such features.

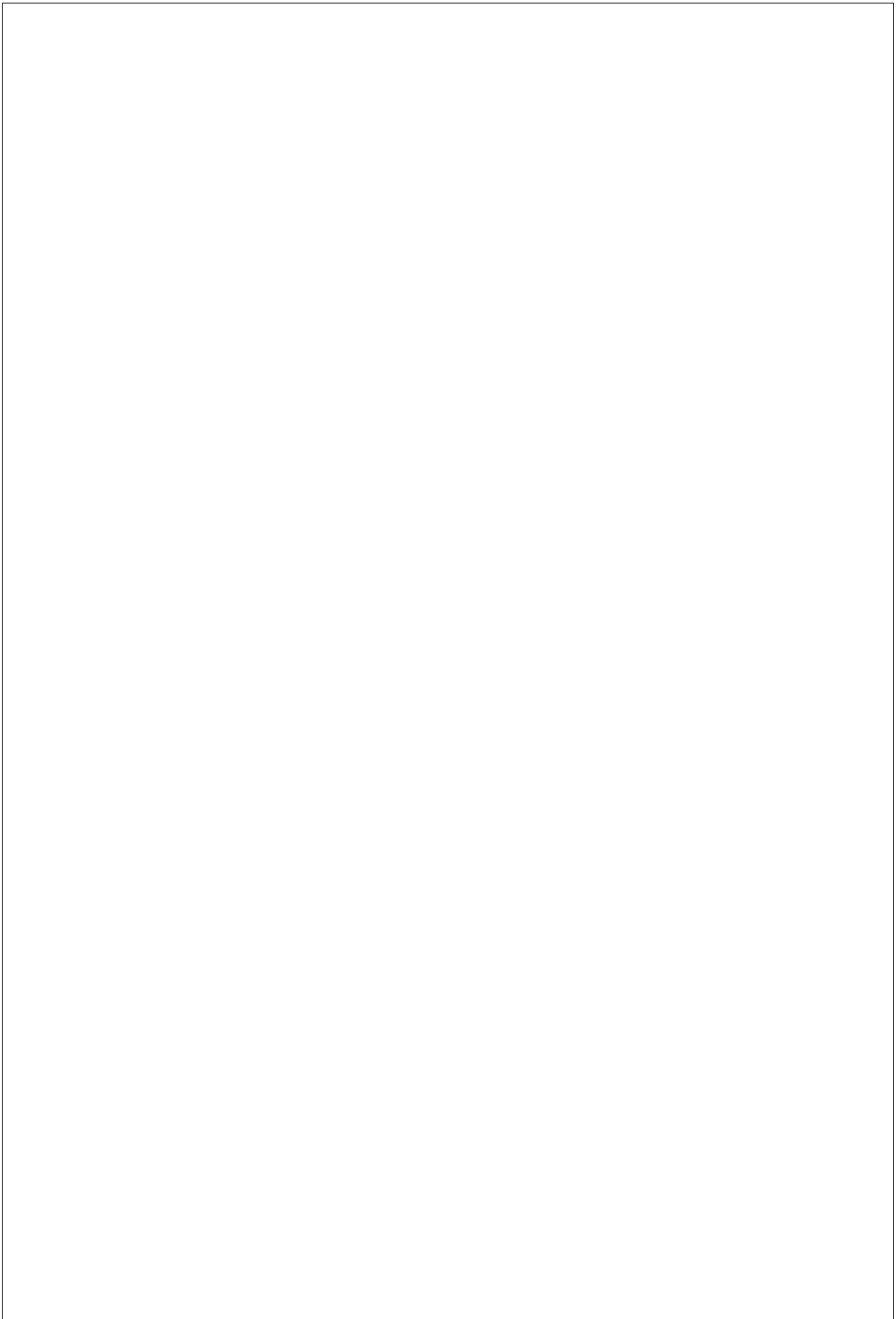
Feature 1

Feature 2

Feature 3

[3]

- (ii)** Draw a program flowchart to represent the algorithm implemented in the pseudocode. Variable declarations are not required in program flowcharts.

A large, empty rectangular box with a thin black border, occupying most of the page below the header. It is intended for the student to write their answer to the question.

[5]

90. O/N 19/P23/Q1/b,c,d

- (b) As part of the development of an algorithm, a programmer may construct an identifier table.

Describe what an identifier table contains.

[2]

- (c) (i) Simple algorithms usually consist of three different stages.

Complete the table below. Write each example statement in **program code**.

The second stage has already been given.

Stage	Example statement
Process	

[5]

- (ii) Write a **single** statement in **program code** that contains **two** of the stages. Do **not** repeat any of the statements from **part (c)(i)**.

[1]

- (d) A software developer is writing a program and includes several features to make it easier to read and understand. One of these features is the use of indentation.

State **three other** features.

Feature 1

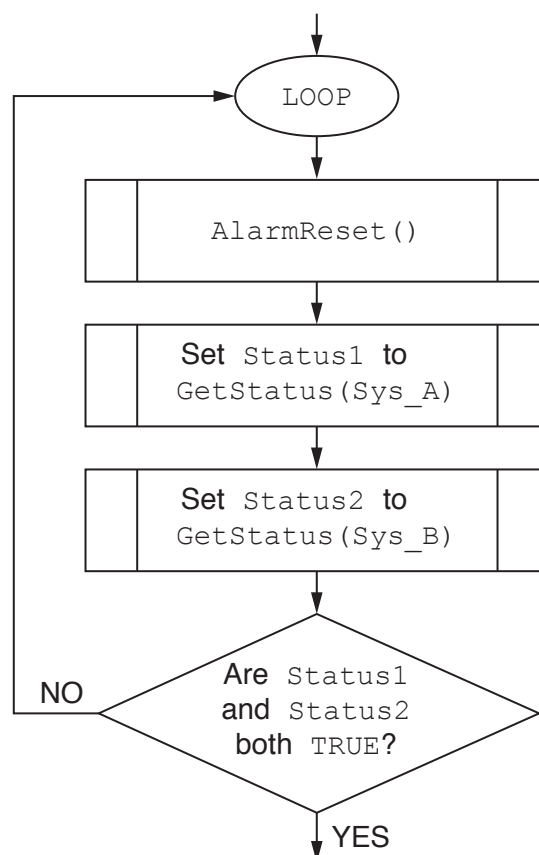
Feature 2

Feature 3

[3]

91. O/N 19/P23/Q2/a(ii)

(ii) Part of a program flowchart is shown.



Implement the flowchart in **pseudocode** using a post-condition loop.

[4]

92. M/J 19/P21/Q1/a

- 1 (a) (i) Algorithms may be expressed using four basic constructs. One construct is sequence.

Complete the following table for two other constructs.

Construct	Pseudocode example
.....	
.....	

[4]

- (ii) Simple algorithms usually consist of input, process and output.

Complete the table by placing ticks (✓) in the relevant boxes.

Pseudocode statement	Input	Process	Output
Temp ← SensorValue * Factor			
WRITEFILE "LogFile.txt", TextLine			
WRITEFILE "LogFile.txt", MyName & MyIDNumber			
READFILE "AddressBook.txt", NextLine			

[4]

93. M/J 19/P21/Q3/a

- 3 (a)** A student is developing an algorithm to search through a 1D array of 100 elements. Each element of the array, `Result`, contains a `REAL` value.

The algorithm will output:

- the average value of all the elements
- the number of elements with a value of zero.

The structured English description of the algorithm is:

1. SET Total value to 0
2. SET Zero count to 0
3. SELECT the first element
4. ADD value of element to Total value
5. IF element value is 0 then INCREMENT Zero count
6. REPEAT from step 4 for next element, until element is last element
7. SET Average to Total / 100
8. OUTPUT a suitable message and Average
9. OUTPUT a suitable message and Zero count

Write **pseudocode** for this algorithm.

[7]

94. M/J 19/P22/Q1/a

1 (a) Algorithms are used in computer programming.

(i) Explain the term **algorithm**.

[2]

(ii) Algorithms usually consist of three different types of activity.

Complete the table below.

The third activity type is given.

Activity type	Pseudocode example
.....	
.....	
Output	

[5]

95. M/J 19/P22/Q5/a

5 A student is learning about text files. She wants to write a program to count the number of lines in a file.

(a) Use **structured English** to describe an algorithm she could use.

[3]

96. M/J 19/P23/Q1/a

- 1 The following pseudocode searches for the longest run of identical characters in the array Message.

```
DECLARE Message : ARRAY[1:100] OF CHAR

PROCEDURE Search()

    DECLARE Index : INTEGER
    DECLARE ThisChar : CHAR
    DECLARE ThisRun : INTEGER
    DECLARE LongRun : INTEGER

    ThisChar ← Message[1]
    ThisRun ← 1
    LongRun ← 1

    FOR Index ← 2 TO 100

        IF Message[Index] = ThisChar
            THEN
                ThisRun ← ThisRun + 1
            ELSE
                ThisChar ← Message[Index]
                IF ThisRun > LongRun
                    THEN
                        LongRun ← ThisRun
                    ENDIF
                ThisRun ← 1
            ENDIF
        ENDIF

    ENDFOR

    OUTPUT "The longest run was " , LongRun

ENDPROCEDURE
```

- (a) Draw a program flowchart to represent the procedure Search().

Variable and array declarations are not required in program flowcharts.

97. M/J 19/P23/Q2/a,b

2 (a) A student is learning about arrays. She wants to write a program to:

- search through a 1D array of 100 elements
- count the number of elements that contain the string “Empty”
- output the number of elements containing “Empty” together with a suitable message.

Use **structured English** to describe the algorithm she could use.

[5]

(b) She uses the process of stepwise refinement to develop her algorithm.

Explain this process.

[3]

98. O/N 18/P22/Q1/a

1 (a) The following table contains statements written in pseudocode.

Show the type of programming construct each statement represents.

Put a tick (✓) in the appropriate column for each statement.

Statement	Selection	Repetition (Iteration)	Assignment
Index $\leftarrow$ Index + 5			
FOR Count $\leftarrow$ 1 TO 100			
TempValue[Index] $\leftarrow$ ReadValue(SensorID)			
IF Index < 30			
UNTIL DayNumber > 7			
OTHERWISE OUTPUT "ERROR"			

[6]

99. O/N 18/P22/Q2/a(i)

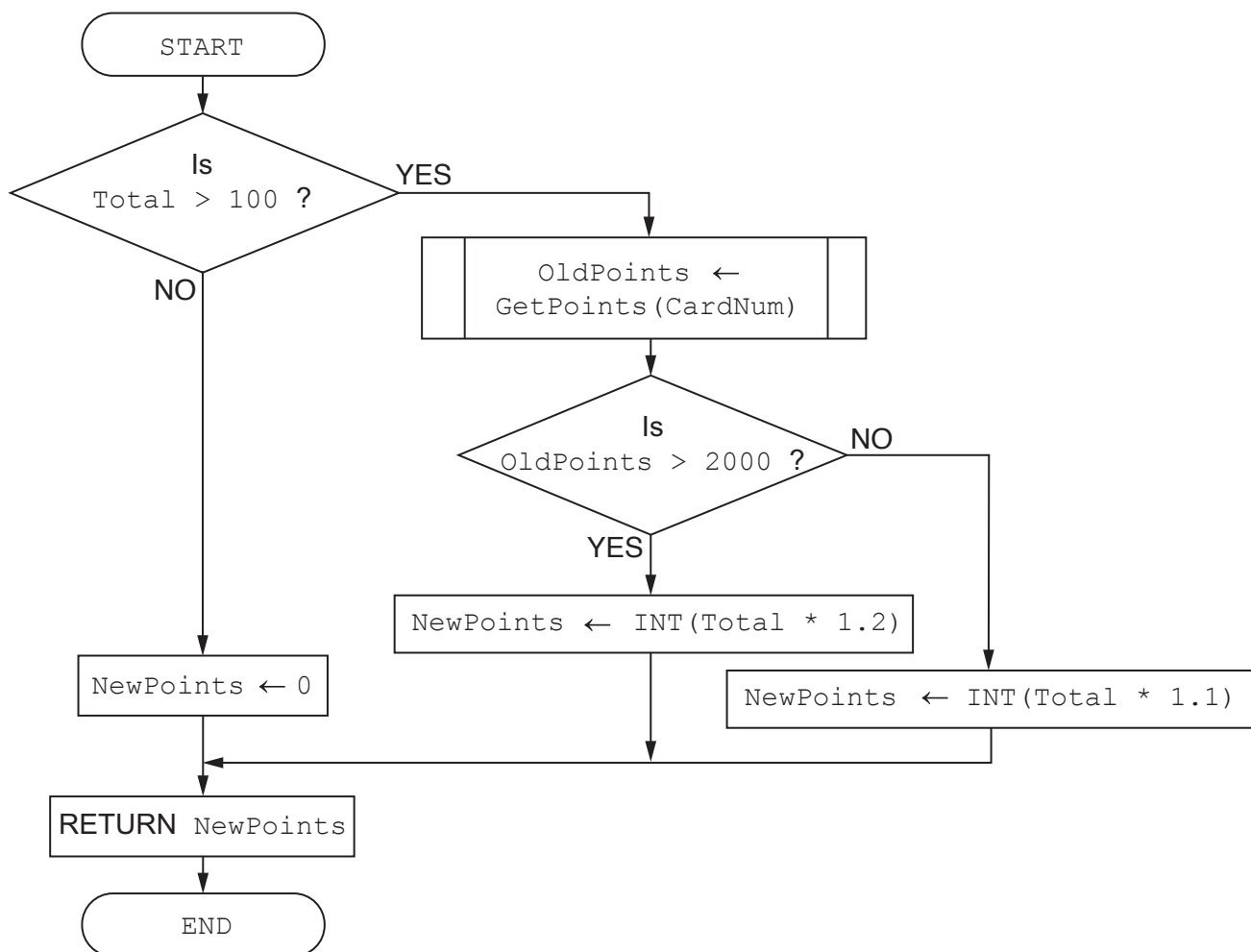
2 Shop customers have a discount card with a unique card number. Customers collect points each time they buy items. The number of points they collect depends on:

- the total amount they spend
- the number of points already collected.

The function `CalcPoints()` takes the card number and the total amount spent as parameters. It returns the number of new points collected. A flowchart for the function is shown.

The function uses the following variables and functions.

Identifier	Data type	Description
CardNum	STRING	A numeric string representing the unique card number
OldPoints	INTEGER	The number of points already collected
NewPoints	INTEGER	The number of new points collected
Total	REAL	The amount spent
GetPoints()	FUNCTION	Takes the card number as a parameter and returns the number of points already collected
INT()	FUNCTION	Refer to the Appendix on page 16



(a) (i) Write **pseudocode** for the `CalcPoints()` function.

Your solution should follow the flowchart as closely as possible.

[7]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns string "FGH"

NUM_TO_STRING(x : REAL) RETURNS STRING
returns a string representation of a numeric value.

Example: If x has the value 87.5 then NUM_TO_STRING(x) will return "87.5"

Note: This function will also work if x is of type integer.

INT(x : REAL) RETURNS INTEGER
returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

100. O/N 18/P22/Q3/b,c

- (b) Stepwise refinement is often used in the development of an algorithm.

Describe **stepwise refinement**.

[2]

- (c) A program needs to search through 1000 elements of an unsorted array to find a given value.

The program will output:

- either the position in the array of the value
- or the message “Not Found”

Outline the steps the program needs to follow.

Do **not** write pseudocode or program code.

[4]

101. O/N 18/P23/Q1/a

- 1 (a) The following table contains statements written in pseudocode.

Show the type of programming construct each statement represents.

Put a tick (✓) in the appropriate column for each statement.

Statement	Assignment	Selection	Repetition (Iteration)
CASE OF TempSensor1			
ELSE			
REPEAT			
ENDFOR			
DayNumber ← DayNumber + 1			
Error ← TRUE			

[6]

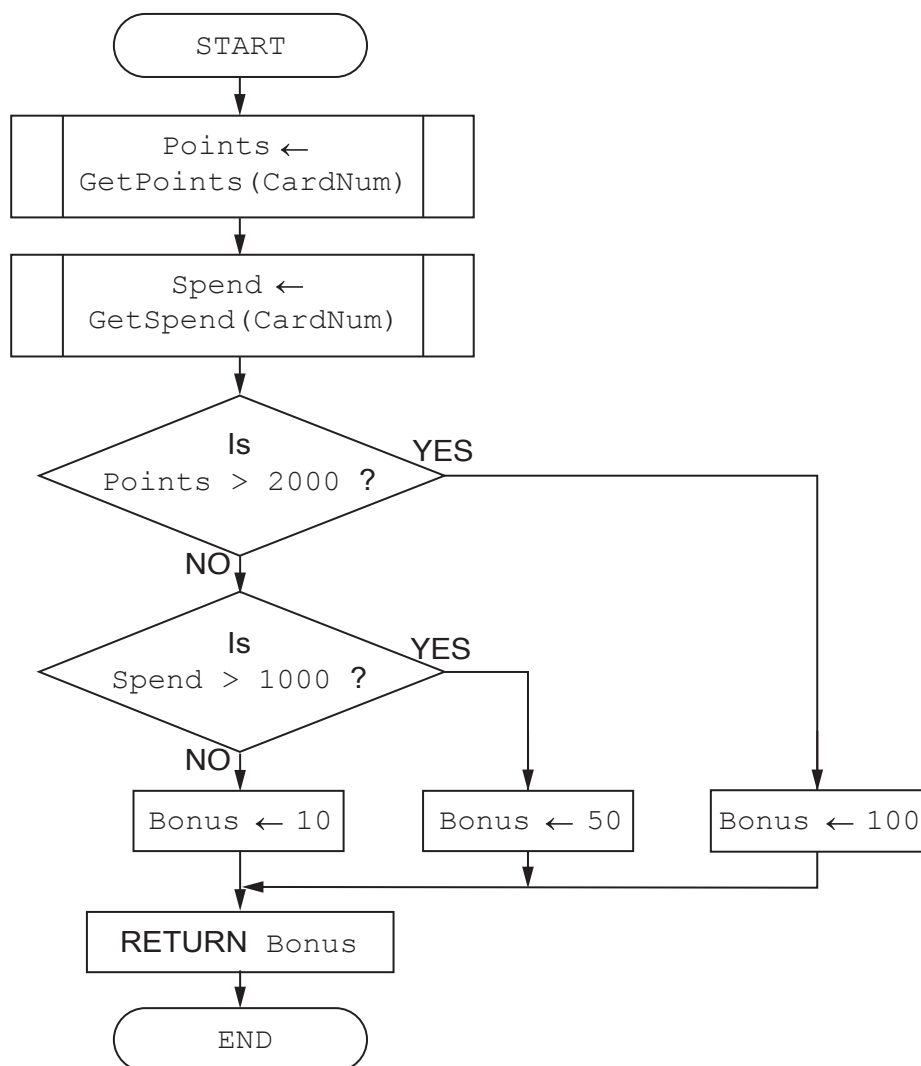
102. O/N 18/P23/Q2/a(i)

- 2 Shop customers have a discount card with a unique card number. Customers collect points when they buy items. At the end of each year, customers are given bonus (extra) points related to the total amount they have spent during the year, and the number of points they have on their card.

The function `CalcBonus()` takes the card number as a parameter. It returns the bonus points for the customer. A flowchart for the function is shown.

The function uses the following variables and functions.

Identifier	Data type	Description
CardNum	STRING	A numeric string representing the unique card number
Points	INTEGER	The number of points collected
Spend	REAL	The total amount that customer has spent during the year
Bonus	INTEGER	The number of bonus points
GetPoints()	FUNCTION	Takes the card number as a parameter and returns the number of points already collected
GetSpend()	FUNCTION	Takes the card number as a parameter and returns the total amount that customer has spent during the year



(a) (i) Write **pseudocode** for the `CalcBonus()` function.

Your solution should follow the flowchart for the function as closely as possible.

103. O/N 18/P23/Q3/b

- 3 An array contains 100 integer values. An algorithm will find the maximum and minimum values stored in the array.

(b) Outline the steps the program will need to follow to implement the algorithm.

Do **not** write pseudocode or program code.

[3]

104. M/J 18/P21/Q1/a

- 1 (a) A program stores data about hospital patients.

Give a suitable **identifier name** for each of the data items.

Description of data item	Suitable identifier name
The temperature of the patient	
The temperature of the room	
The patient identification number	
The name of the nurse taking the measurement	

[4]

105. M/J 18/P21/Q2/a

- 2 The following is a function design in pseudocode.

Line numbers are given for reference only.

```

01  FUNCTION StringClean(InString : STRING) RETURNS STRING
02
03      DECLARE NextChar : CHAR
04      DECLARE OutString : STRING
05      DECLARE Counter : INTEGER
06      DECLARE MyString : STRING
07
08      OutString ← ""
09
10      FOR Counter ← 1 TO LENGTH(InString)
11
12          NextChar ← MID(InString,Counter,1)
13          NextChar ← LCASE(NextChar)
14
15          IF (NextChar >= 'a') AND (NextChar <= 'z')
16
17              THEN
18
19                  OutString ← OutString & NextChar
20
21          ENDIF
22
23      ENDFOR
24
25      RETURN OutString
26
27  ENDFUNCTION

```

- (a) (i) This pseudocode includes features that make it easier to read and understand.

State **four** such features.

Feature 1

Feature 2

Feature 3

Feature 4

[4]

- (ii) State **one** feature that could be added to make the pseudocode easier to understand.

[1]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

Each function returns an error if the function call is not properly formed.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns string "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER
 returns the integer value representing the length of string ThisString

Example: LENGTH("Happy Days") returns 10

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
 returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns string "ABC"

LCASE(ThisChar : CHAR) RETURNS CHAR
 returns the character value representing the lower case equivalent of ThisChar

If ThisChar is not an upper-case alphabetic character then it is returned unchanged.

Example: LCASE('W') returns 'w'

INT(x : REAL) RETURNS INTEGER
 returns the integer part of x

Example: INT(27.5415) returns 27

ASC(ThisChar : CHAR) RETURNS INTEGER
 returns the ASCII value of character ThisChar

Example: ASC('A') returns 65

MOD(ThisNum : INTEGER, ThisDiv : INTEGER) RETURNS INTEGER
 returns the integer value representing the remainder when ThisNum is divided by ThisDiv

Example: MOD(10, 3) returns 1

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

106. M/J 18/P21/Q3

- 3 In a chemical factory, a procedure, `CheckSensor()` is required to allow an operator to monitor the temperature in different locations.

In the factory:

- the temperature is measured by 10 sensors, each at a different location
- each sensor has a unique ID (1 to 10).

The procedure `CheckSensor()` will compare the measured temperature against each of two constant values, `LowTemp` and `HighTemp`. It will perform the following actions depending on the result of the comparison.

Measured temperature	Action
below <code>LowTemp</code>	Output "Cold"
from <code>LowTemp</code> to <code>HighTemp</code>	Output "Normal"
above <code>HighTemp</code>	Call procedure <code>Alarm()</code>

A library function, `GetTemp()`, returns the temperature value from a given sensor.

The structured English representing the algorithm for the procedure `CheckSensor()` is as follows:

1. Prompt for the input of a sensor ID.
2. Input a sensor ID.
3. If the sensor ID is invalid, repeat from step 1.
4. Call the `GetTemp()` function with the sensor ID as the parameter, to obtain the relevant temperature.
5. Compare the temperature against the two constant values and take the appropriate action.

Draw a program flowchart on the next page to represent the algorithm for procedure `CheckSensor()`.

Variable declarations are not required in program flowcharts.

107. M/J 18/P22/Q1/a

- 1 (a) A farm has a number of greenhouses used to grow vegetables. Each greenhouse has a different identification number. A program is needed to store temperature information for each greenhouse throughout the day.

Give a suitable **identifier name** for each of the data items.

Description of data item	Suitable identifier name
The temperature inside the greenhouse	
The temperature outside the greenhouse	
The greenhouse identification number	
The time of the temperature measurements	

[4]

108. M/J 18/P22/Q2/a

2 The following is a function design in pseudocode.

Line numbers are given for reference only.

```
01 FUNCTION CountDigits(InString : STRING) RETURNS CHAR
02
03 DECLARE nc : CHAR
04 DECLARE c : INTEGER
05 DECLARE n : INTEGER
06
07
08 c ← 0
09 n ← LENGTH(InString) // get number of characters for loop
10
11 WHILE n > 0 // repeat until no more characters left
12
13 nc ← LEFT(InString,1)
14 n ← n - 1
15 InString ← RIGHT(InString,n) // remove first character
16
17 IF (nc < '0') OR (nc > '9')
18 THEN
19 // do nothing
20 ELSE
21 c ← c + 1
22 ENDIF
23
24 ENDWHILE
25
26 RETURN c
27
28 ENDFUNCTION
```

(a) (i) This pseudocode includes features that make it easier to read and understand.

State **two** such features.

Feature 1

Feature 2

[2]

(ii) State **two** additional features that should be used to make this pseudocode easier to read and understand.

Feature 1

Feature 2

[2]

109. M/J 18/P22/Q3

- 3** A chocolate factory produces bars of chocolate. A computer program controls the process.

The weight of each bar is stored in an array, `BarWeight`. The array contains 100 elements, representing the weights of 100 bars that make up one shipping box.

A procedure, `CheckWeight()`, is required to:

1. examine each array element and count how many times the weight has exceeded `MaxWeight`
2. compare the count obtained with a limit value, `Threshold`. Call procedure `ServiceCheck()` if the count exceeds the `Threshold`
3. output a message if the count does not exceed the `Threshold`. For example:

`"ShippingBox OK - maximum weight exceeded 3 times."`

Draw a program flowchart on the next page to represent the algorithm for the `CheckWeight()` procedure.

Assume that:

- the array contains 100 valid weight values and the first element is `BarWeight[1]`
- `MaxWeight`, `Threshold` and `BarWeight` are global variables.

Variable declarations are not required in program flowcharts.

[10]

110. M/J 18/P23/Q1/a

- 1 (a)** A program controls the heating system of an energy-efficient house.

Give a suitable **identifier name** for each of the data items.

Description of data item	Suitable identifier name
The temperature inside the house	
The temperature outside the house	
The wind speed	
Whether it was raining or not	

[4]

111. M/J 18/P23/Q2/a

- 2** The following is a function design in pseudocode.

Line numbers are given for reference only.

```

01  FUNCTION GradeCalc(Mark : ARRAY[] OF INTEGER) RETURNS INTEGER
02
03      DECLARE DGradeCount : INTEGER
04      DECLARE n : INTEGER
05      DECLARE Grade : STRING
06      DECLARE ThisMark : INTEGER
07
08      DGradeCount ← 0      // initialise variables
09      n ← 1
10      ThisMark ← 1
11      REPEAT
12          ThisMark ← Mark[n]      // get next mark from Mark array
13
14          IF ThisMark > 74
15              THEN
16                  Grade ← "Distinction"
17                  DGradeCount ← DGradeCount + 1 // one more Distinction
18              ELSE
19                  IF ThisMark > 59
20                      THEN
21                          Grade ← "Merit"
22                      ELSE
23                          IF ThisMark > 39
24                              THEN
25                                  Grade ← "Pass"
26                              ELSE
27                                  Grade ← "Fail"
28                              ENDIF
29                          ENDIF
30                  ENDIF
31
32          OUTPUT "Grade for Student " & n & " is " & Grade
33
34          n ← n + 1
35      UNTIL n = 101 // no more elements in the array
36
37      RETURN Grade // return the number of Distinction grades
38
39  ENDFUNCTION
40
41  DECLARE Mark : ARRAY[1:100] OF INTEGER
42  DECLARE DistinctionGrades : INTEGER
43
44  CALL GetMarks(Mark[]) // fill the array with student marks
45  DistinctionGrades ← GradeCalc(Mark[])

```

(a) This pseudocode includes features that are examples of good practice.

Explain why it is useful to include the two following features.

Comments

Indentation

112. M/J 18/P23/Q4

- 4 A program controls a chemical process in a factory.

The temperature is monitored as part of the control process. The temperature is measured at fixed time intervals and the value is stored in an array, `PTemp`. The array contains 100 elements, representing 100 temperature values. The first element is `PTemp[1]`.

The program will check whether the temperature is outside the acceptable range more than 20 times. This task is performed by a function, `IsTempOK()`.

The algorithm for the function `IsTempOK()` is expressed in structured English as follows:

- 1 Examine each array element and count the number of times that a temperature is less than `MinTemp` or more than `MaxTemp`.
- 2 If the count in step 1 exceeds 20, return `FALSE`, otherwise return `TRUE`.

Draw a program flowchart, on the next page, to represent the algorithm for the function `IsTempOK()`.

Assume:

- the array contains 100 valid temperature values
- `PTemp`, `MinTemp` and `MaxTemp` are global variables.

Note that variable declarations are not required in program flowcharts.

[9]

113. O/N 17/P21/Q2/c

- 2 The following pseudocode represents a simple algorithm.

```

DECLARE NumberFound, Remainder, Number : INTEGER
DECLARE StartNumber, EndNumber, Divisor : INTEGER

INPUT StartNumber
INPUT EndNumber
INPUT Divisor
NumberFound ← 0

FOR Number ← StartNumber TO EndNumber
    Remainder ← MODULUS(Number, Divisor)
    IF Remainder = 0
        THEN
            OUTPUT Number
            NumberFound ← NumberFound + 1
        ENDIF
    ENDFOR
OUTPUT "Count: " & NumberFound

```

For the built-in functions list, refer to the **Appendix** on page 14.

- (c) Draw a program flowchart to represent this algorithm.

Variable declarations are **not** required in program flowcharts.

[10]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

`MODULUS(x : INTEGER, y : INTEGER) RETURNS INTEGER`

returns the remainder when `x` is divided by `y` using integer arithmetic.

Example: `MODULUS(5, 2)` returns 1

`INT(x : REAL) RETURNS INTEGER`

returns the integer part of `x`.

Example: `INT(27.5415)` returns 27

`LENGTH(ThisString : STRING) RETURNS INTEGER`

returns the integer value representing the length of string `ThisString`.

Example: `LENGTH("Happy Days")` returns 10

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`

returns leftmost `x` characters from `ThisString`.

Example: `LEFT("ABCDEFGH", 3)` returns string "ABC"

`RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING`

returns rightmost `x` characters from `ThisString`.

Example: `RIGHT("ABCDEFGH", 3)` returns string "FGH"

Operators (pseudocode)

Operator	Description
<code>&</code>	Concatenates (joins) two strings Example: <code>"Summer" & " " & "Pudding"</code> produces <code>"Summer Pudding"</code>
<code>AND</code>	Performs a logical AND on two Boolean values Example: <code>TRUE AND FALSE</code> produces <code>FALSE</code>
<code>OR</code>	Performs a logical OR on two Boolean values Example: <code>TRUE OR FALSE</code> produces <code>TRUE</code>

114. O/N 17/P22/Q2/b

- 2** A 1D array, `ClassName`, of type `STRING` contains 100 elements.

The following pseudocode represents a simple algorithm to process the array.

```
DECLARE SearchValue : STRING
DECLARE FoundFlag : BOOLEAN
DECLARE Index : INTEGER

INPUT SearchValue
FoundFlag ← FALSE
Index ← 1

WHILE Index < 101 AND FoundFlag = False
    IF ClassName[Index] = SearchValue
        THEN
            OUTPUT Index
            FoundFlag ← TRUE
        ENDIF
    Index ← Index + 1
ENDWHILE

IF FoundFlag = FALSE
    THEN
        OUTPUT "Not found"
    ENDIF
```

- (b)** Draw a program flowchart to represent this algorithm.

Note that variable declarations are not required in program flowcharts.

[9]

115. O/N 17/P23/Q2/c

- 2** The following pseudocode represents a simple algorithm.

```
DECLARE NumberFound, Remainder, Number : INTEGER
DECLARE StartNumber, EndNumber, Divisor : INTEGER

INPUT StartNumber
INPUT EndNumber
INPUT Divisor
NumberFound ← 0

FOR Number ← StartNumber TO EndNumber
    Remainder ← MODULUS(Number, Divisor)
    IF Remainder = 0
        THEN
            OUTPUT Number
            NumberFound ← NumberFound + 1
        ENDIF
    ENDFOR
OUTPUT "Count: " & NumberFound
```

For the built-in functions list, refer to the **Appendix** on page 14.

(c) Draw a program flowchart to represent this algorithm.

Variable declarations are **not** required in program flowcharts.

[10]

Appendix of this paper:

Appendix

Built-in functions (pseudocode)

In each function, if the function call is not properly formed, the function returns an error.

```
MODULUS(x : INTEGER, y : INTEGER) RETURNS INTEGER
```

returns the remainder when x is divided by y using integer arithmetic.

Example: MODULUS(5, 2) returns 1

```
INT(x : REAL) RETURNS INTEGER
```

returns the integer part of x .

Example: INT(27.5415) returns 27

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```

returns the integer value representing the length of string `ThisString`.

Example: LENGTH("Happy Days") returns 10

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns leftmost x characters from `ThisString`.

Example: LEFT("ABCDEFGH", 3) returns string "ABC"

```
RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING
```

returns rightmost x characters from `ThisString`.

Example: RIGHT("ABCDEFGH", 3) returns string "FGH"

Operators (pseudocode)

Operator	Description
&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" produces "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE produces FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE produces TRUE

116. M/J 17/P21/Q1/a

- 1 (a) Simple algorithms usually consist of three different stages.

Complete the following table.

Add a description of the stage and an example pseudocode statement.

The first stage has been given.

Stage	Description and example
Input	Description: Pseudocode example:
.....	Description: Pseudocode example:
.....	Description: Pseudocode example:

[7]

117. M/J 17/P21/Q2/a,b,c

- 2 One of the security features of a multi-user computer system is a user login process. The user must complete this successfully before they can access the resources of the system.

As part of the login process the user enters their user ID followed by a password. The system then compares the password entered with the password held in a file.

(a) The steps involved in the login process are described as follows:

- User enters their ID and password.
- Validation checks:
 - Compare user ID with data from the file.
 - Indicate whether or not the user ID was found.
 - If user ID found, check whether passwords match.

The description above is not detailed enough to allow a program to be written. The validation checks must be expressed as a more detailed algorithm.

Give the name of the process of increasing the level of detail of the algorithm.

[1]

(b) An identifier table is created as the algorithm is developed. A section of the table is shown. Complete the table.

Identifier	Data Type	Description
UserIDInput		Stores the user ID entered
PasswordInput		
UserIDFound		
PasswordValid		

[5]

(c) The validation checks described in **part (a)** are to be used as the basis for program code.

Use **structured English** to write a more detailed algorithm.

You should:

- use the identifiers given in the previous table
- assume that the password file, `password.txt`, is organised as a simple text file. The user ID and password are stored together, one entry per line.

[8]

118. M/J 17/P22/Q1/a

- 1 (a) Simple algorithms usually consist of input, process and output.

The statements in the following table are in a generic programming language.

Complete the table by placing ticks in the relevant boxes.

Item	Statement	Input	Process	Output
1	<code>String1 = "Hello World"</code>			
2	<code>DISPLAY RIGHT(String1, 5)</code>			
3	<code>READFILE (MyFile, String2)</code>			
4	<code>WRITEFILE (MyFile, "Data is " & String2)</code>			

[6]

119. M/J 17/P22/Q2/a,b,c

- 2 A multi-user computer system maintains a text file containing the ID and preferred name for each user.
User IDs are unique. Preferred names may be repeated.

- (a) Stepwise refinement is to be applied to the following three steps.

After a user logs in, a welcome message is produced as follows:

1. Search for the user ID in the file.
2. Read the preferred name from the file.
3. Output the welcome message.

Describe the goal of **stepwise refinement**.

[2]

- (b) An initial identifier table is created as part of the stepwise refinement. A section of the table is shown. Complete this table.

Identifier	Data type	Description
SearchUserID		Stores the user ID entered
FileUserID		
FilePreferredName		
IDFoundFlag		

[5]

- (c) The file may not contain an entry for every user. If the user ID is not found, then the name “Guest” should be used as the preferred name.

Use **structured English** to write a more detailed algorithm for the welcome message process described in **part (a)**.

Use the identifiers given in the previous table.

Assume that the file, `UserNames.txt`, is organised as a simple text file. The user ID and preferred name are held together in unencrypted form, one entry per line.

[8]

120. M/J 17/P23/Q1/a

- 1 (a) Simple algorithms usually consist of three different stages.

Complete the following table.

Add a description of the stage and an example pseudocode statement.

The first stage has been given.

Stage	Description and example
Input	Description: Pseudocode example:
.....	Description: Pseudocode example:
.....	Description: Pseudocode example:

[7]

121. M/J 17/P23/Q2/a,b,c

- 2 One of the security features of a multi-user computer system is a user login process. The user must complete this successfully before they can access the resources of the system.

As part of the login process the user enters their user ID followed by a password. The system then compares the password entered with the password held in a file.

- (a) The steps involved in the login process are described as follows:

- User enters their ID and password.
- Validation checks:
 - Compare user ID with data from the file.
 - Indicate whether or not the user ID was found.
 - If user ID found, check whether passwords match.

The description above is not detailed enough to allow a program to be written. The validation checks must be expressed as a more detailed algorithm.

Give the name of the process of increasing the level of detail of the algorithm.

[1]

- (b) An identifier table is created as the algorithm is developed. A section of the table is shown. Complete the table.

Identifier	Data Type	Description
UserIDInput		Stores the user ID entered
PasswordInput		
UserIDFound		
PasswordValid		

[5]

- (c) The validation checks described in **part (a)** are to be used as the basis for program code.

Use **structured English** to write a more detailed algorithm.

You should:

- use the identifiers given in the previous table
- assume that the password file, `password.txt`, is organised as a simple text file. The user ID and password are stored together, one entry per line.

[8]

122. O/N 16/P21/Q1/a,b,c

also Unit 12 (5 marks), Unit 11 (3 marks); this unit 6 marks

- 1 A programmer wants to write a program to calculate the baggage charge for a passenger's airline flight.

Two types of ticket are available for a flight:

- economy class (coded E)
- standard class (coded S)

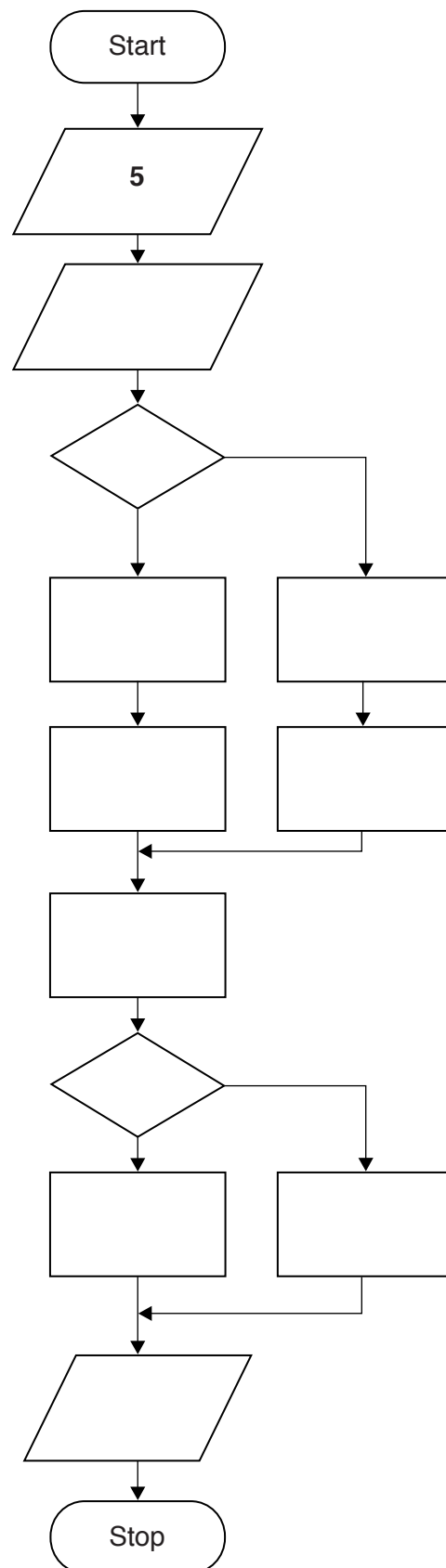
Each ticket type has a baggage weight allowance as shown below. The airline makes a charge if the weight exceeds the allowance.

Ticket type	Baggage allowance (kg)	Charge rate per additional kg (\$)
'E'	16	3.50
'S'	20	5.75

- (a) A program flowchart will document the program. The flowchart will contain the following statements:

Statement number	Statement
1	Charge $\leftarrow$ 0
2	INPUT BaggageWeight
3	Charge $\leftarrow$ ExcessWeight * ChargeRate
4	Is ExcessWeight > 0 ?
5	INPUT TicketType
6	ExcessWeight $\leftarrow$ BaggageWeight - BaggageAllowance
7	BaggageAllowance $\leftarrow$ 16
8	ChargeRate $\leftarrow$ 3.5
9	OUTPUT Charge
10	ChargeRate $\leftarrow$ 5.75
11	BaggageAllowance $\leftarrow$ 20
12	Is TicketType = 'E' ?

Complete the flowchart by putting the appropriate **statement number** in each flowchart symbol. Statement 5 has been done for you.



(b) The programmer needs data to test the flowchart.

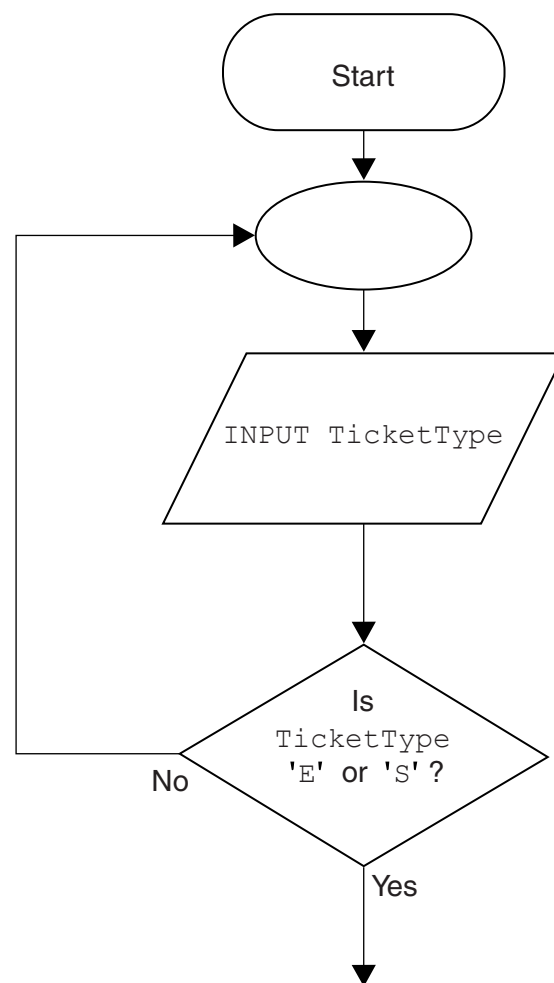
[6]

Complete the table of test data below to show **five** tests.

TicketType	BaggageWeight	Explanation	Expected output
E	15		
			
			
			
			

[5]

- (c) The program design is to be amended. The value input by the user for the ticket type is to be validated. Part of the amended flowchart is shown below.



Write **pseudocode** to use a pre-condition loop for this validation.

[3]

123. O/N 16/P21/Q2/b

2 A sensing device sends bit values to a computer along data channels.

- Channel 1 transmits a sequence of binary values from a sensor
- Channel 2 transmits at regular intervals to indicate whether the sensor is switched on or off:
 - 0 indicates switched off
 - 1 indicates switched on

A program tests the bits received from the sensing device.

A program reads the signal from Channel 2 after every six values from Channel 1.

A built-in function `READ(<ChannelNumber>)` reads a value from the specified channel.

Pseudocode for the program is as follows:

```

01      BitCount ← 0
02      Status2 ← READ(2)
03      WHILE Status2 = 1
04
05          FOR ReadingCount ← 1 TO 6
06              ThisBit ← READ(1)
07              IF ThisBit = 1
08                  THEN
09                      BitCount ← BitCount + 1
10              ENDIF
11              IF BitCount = 5
12                  THEN
13                      OUTPUT "Error - Investigate"
14                      BitCount ← 0
15              ENDIF
16          ENDFOR
17
18      Status2 ← READ(2)
19  ENDWHILE
  
```

(b) Identify the following constructs in the given program, using line numbers.

For multi-line constructs give the first line number only.

Construct	Line number
Assignment	
Selection	
Iteration	

124. O/N 16/P22/Q1/a,b,c

also Unit 12 (5 marks), Unit 11 (3 marks); this unit 9 marks

- 1** A number of players take part in a competition. The competition consists of a number of games. Each game is between two players. The outcome of a game is that each player is awarded a grade (A, B, C or D). Each grade has an associated number of points as shown in the table below.

Grade	Points
A	0
B	1
C	3
D	5

The points total for all players is recorded. After each game is completed, the total number of points for each player is updated.

For example:

- before the game between Ryan and Karina, Ryan's total is 5 points and Karina's total is 3 points
- the result of the game between Ryan and Karina is: Ryan achieved grade B, Karina achieved grade D
- the players' points totals are updated to: Ryan has 6 and Karina has 8

When a player's points total reaches 12 or higher, that player is removed from the competition.

A programmer will write a program to update the player total after a game.

The program will output:

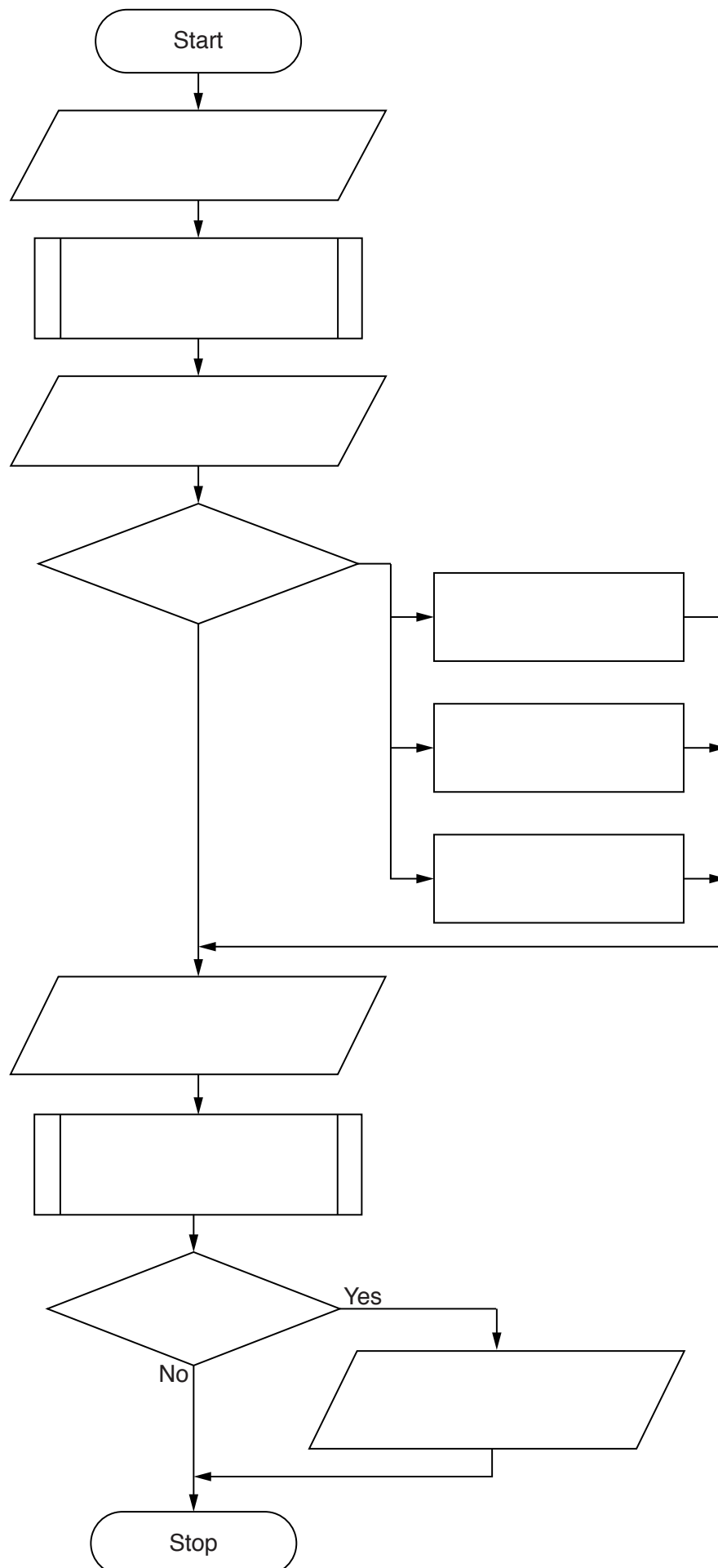
- the player's updated points total
- the message 'ELIMINATED' if the player is removed from the competition.

The programmer designs the identifier table below:

Identifier	Data type	Description
PlayerName	STRING	Name of the player
PlayerGameGrade	CHAR	Game grade for the player
PointsTotal	INTEGER	Current player points
SavePlayerTotal	procedure	Procedure has parameters <code>PlayerName</code> and <code>PointsTotal</code> and saves the updated player total
ReadPlayerTotal	function	Function has a parameter <code>PlayerName</code> and returns the current total for that player

(a) Complete the following program flowchart by:

- filling in the boxes, using pseudocode where appropriate
- labelling the lines of the flowchart, where necessary.



(b) Test data is to be produced to test the flowchart.

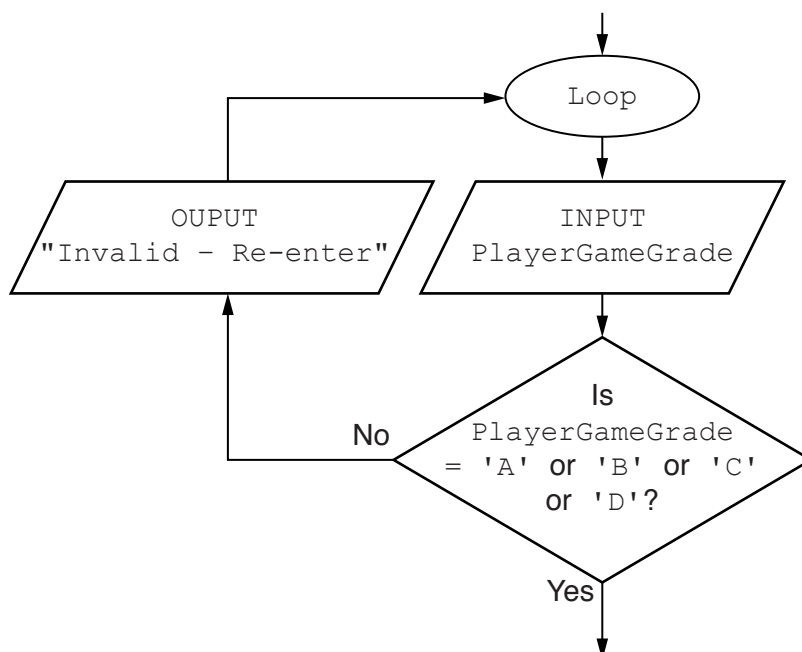
Complete the table of test data below to show **five** tests that should be used to test different paths through the flowchart.

Test data		Expected results	
PointsTotal	PlayerGameGrade	Updated PointsTotal	Output

[5]

- (c) The programmer amends the design to validate the value of player game grade that the user inputs.

The amended part of the flowchart is shown below.



Write the equivalent **pseudocode** using a pre-condition loop, for this part of the amended flowchart.

[3]

125. O/N 16/P22/Q5/b(i)

- 5** A team keeps a record of the scores made by each of their eight players in a number of games.

The data in the two tables below shows:

- the scores of the eight players after twenty games
- the eight player names.

	1	2	3		8
1	12	17	67		31
2	35	82	44		29
3	61	39	80		17
4	81	103	21		11
5	56	0	98		4
...					
19	45	6	81		77
20	12	11	3		6

1	Vorma
2	Ravi
3	Chada
4	Nigam
5	Bahri
6	Smith
7	Goyal
8	Lata

The team wants a computer program to input and record the player data.

- (a) A programmer designs the following pseudocode for the input of a player's score from one game.

```

01 INPUT GameNumber
02 INPUT PlayerNumber
03 INPUT PlayerGameScore
04 PlayerScore[GameNumber, PlayerNumber] ← PlayerGameScore

```

Describe the data structure the programmer has used for the storage of all player scores.

[2]

- (b) The player names are permanently stored in a text file `NAMES.TXT`, with one name on each line. The player names will be read by the program and stored in a 1D array.

The design given in **part (a)** will be expanded so that the user is prompted for the player name instead of the player number. Step 02 now becomes:

```

02.1 Read the player names from file NAMES.TXT into the array PlayerName
02.2 INPUT ThisPlayerName
02.3 Search the PlayerName array for ThisPlayerName to find the PlayerNumber

```

- (i) State the computing term for the expansion of one or more steps in the original design.

[1]

126. O/N 16/P23/Q1/a,b,c

also Unit 12 (5 marks), Unit 11 (3 marks); this unit 6 marks

- 1 A programmer wants to write a program to calculate the baggage charge for a passenger's airline flight.

Two types of ticket are available for a flight:

- economy class (coded E)
- standard class (coded S)

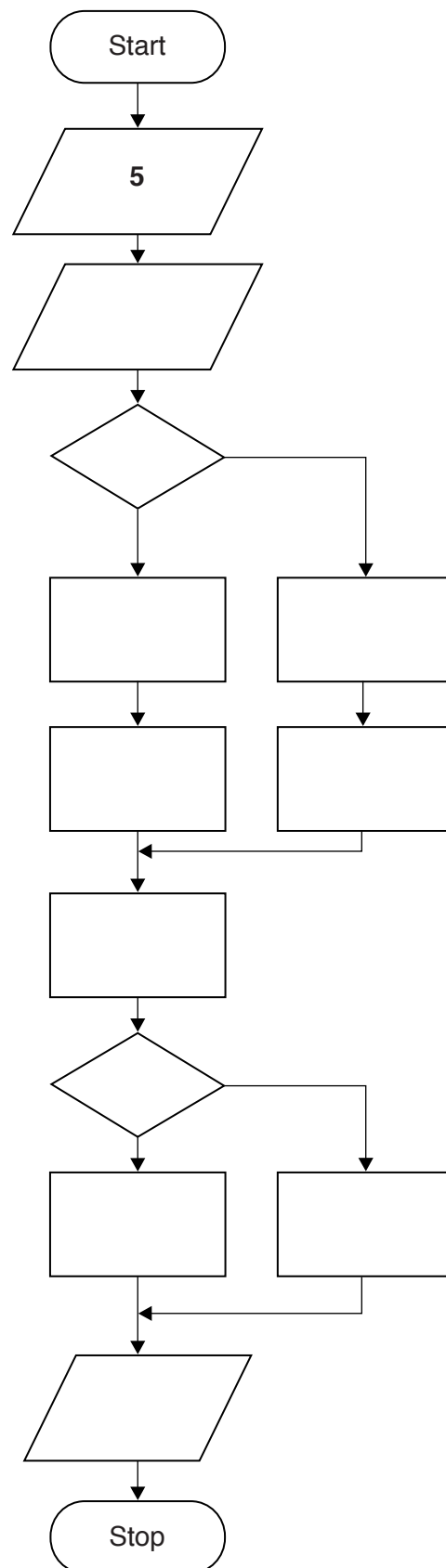
Each ticket type has a baggage weight allowance as shown below. The airline makes a charge if the weight exceeds the allowance.

Ticket type	Baggage allowance (kg)	Charge rate per additional kg (\$)
'E'	16	3.50
'S'	20	5.75

- (a) A program flowchart will document the program. The flowchart will contain the following statements:

Statement number	Statement
1	Charge $\leftarrow$ 0
2	INPUT BaggageWeight
3	Charge $\leftarrow$ ExcessWeight * ChargeRate
4	Is ExcessWeight > 0 ?
5	INPUT TicketType
6	ExcessWeight $\leftarrow$ BaggageWeight - BaggageAllowance
7	BaggageAllowance $\leftarrow$ 16
8	ChargeRate $\leftarrow$ 3.5
9	OUTPUT Charge
10	ChargeRate $\leftarrow$ 5.75
11	BaggageAllowance $\leftarrow$ 20
12	Is TicketType = 'E' ?

Complete the flowchart by putting the appropriate **statement number** in each flowchart symbol. Statement 5 has been done for you.



(b) The programmer needs data to test the flowchart.

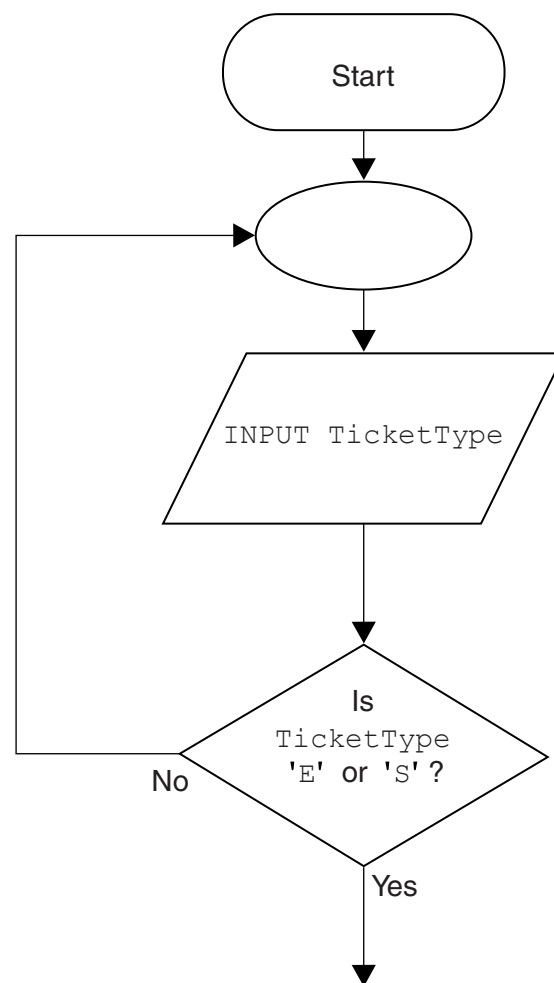
[6]

Complete the table of test data below to show **five** tests.

TicketType	BaggageWeight	Explanation	Expected output
E	15		
			
			
			
			

[5]

- (c) The program design is to be amended. The value input by the user for the ticket type is to be validated. Part of the amended flowchart is shown below.



Write **pseudocode** to use a pre-condition loop for this validation.

[3]

127. O/N 16/P23/Q2/b

2 A sensing device sends bit values to a computer along data channels.

- Channel 1 transmits a sequence of binary values from a sensor
- Channel 2 transmits at regular intervals to indicate whether the sensor is switched on or off:
 - 0 indicates switched off
 - 1 indicates switched on

A program tests the bits received from the sensing device.

A program reads the signal from Channel 2 after every six values from Channel 1.

A built-in function `READ(<ChannelNumber>)` reads a value from the specified channel.

Pseudocode for the program is as follows:

```

01      BitCount ← 0
02      Status2 ← READ(2)
03      WHILE Status2 = 1
04
05          FOR ReadingCount ← 1 TO 6
06              ThisBit ← READ(1)
07              IF ThisBit = 1
08                  THEN
09                      BitCount ← BitCount + 1
10              ENDIF
11              IF BitCount = 5
12                  THEN
13                      OUTPUT "Error - Investigate"
14                      BitCount ← 0
15              ENDIF
16          ENDFOR
17
18      Status2 ← READ(2)
19  ENDWHILE
  
```

(b) Identify the following constructs in the given program, using line numbers.

For multi-line constructs give the first line number only.

Construct	Line number
Assignment	
Selection	
Iteration	

128. M/J 16/P21/Q2/a,b

- 2** A team is designing a software system to monitor temperature in a process. To do this, the system needs to sample the temperature repeatedly. If the temperature exceeds a given threshold value, an alarm will sound.

The system is to be software-based. It will include a subroutine, `SampleTemp`, which samples the temperature and sets the alarm state to either ON or OFF.

The initial design stage will produce a prototype of `SampleTemp` with a user interface.
The structured English for this is:

1. IF the temperature does not exceed threshold value, SET alarm state to OFF
2. INPUT threshold value (to two decimal places)
3. INPUT sensor value (a whole number in the range 0 to 100)
4. MULTIPLY sensor value by conversion factor 1.135 to give temperature
5. IF temperature exceeds threshold value SET alarm state to ON
6. IF temperature exceeds threshold value OUTPUT message "Temperature Alarm"
7. IF temperature does not exceed threshold value OUTPUT message "Temperature OK"

- (a)** The procedure needs four variables. Complete the identifier table below for these variables.

Identifier	Data type	Description
AlarmState		
SensorValue		
ThresholdValue		
Temperature		

[4]

- (b)** Write the **pseudocode** equivalent of the structured English. Use the identifiers from the table in **part (a)**.

[6]

129. M/J 16/P21/Q4/a

- 4 (a)** Structured programming involves the breaking down of a problem into modules.

Give **two** reasons why this is done.

1

2

[2]

130. M/J 16/P22/Q2/a,b

- 2** A team is designing a software system to monitor temperature in a process. To do this, the system needs to sample the temperature repeatedly. If the temperature exceeds a given threshold value, an alarm will sound.

The system is to be software-based. It will include a subroutine, `SampleTemp`, which samples the temperature and sets the alarm state to either ON or OFF.

The initial design stage will produce a prototype of `SampleTemp` with a user interface.
The structured English for this is:

1. IF the temperature does not exceed threshold value, SET alarm state to OFF
2. INPUT threshold value (to two decimal places)
3. INPUT sensor value (a whole number in the range 0 to 100)
4. MULTIPLY sensor value by conversion factor 1.135 to give temperature
5. IF temperature exceeds threshold value SET alarm state to ON
6. IF temperature exceeds threshold value OUTPUT message "Temperature Alarm"
7. IF temperature does not exceed threshold value OUTPUT message "Temperature OK"

- (a)** The procedure needs four variables. Complete the identifier table below for these variables.

Identifier	Data type	Description
AlarmState		
SensorValue		
ThresholdValue		
Temperature		

[4]

- (b)** Write the **pseudocode** equivalent of the structured English. Use the identifiers from the table in **part (a)**.

[6]

131. M/J 16/P22/Q4/a

- 4 (a)** Structured programming involves the breaking down of a problem into modules.

Give **two** reasons why this is done.

1

2

[2]

132. M/J 16/P23/Q2/a,b

- 2** The engine management system of a car includes an energy-saving facility. When certain conditions are met, this facility will automatically stop the engine.

The system is to be software-based. It will include a subroutine, `EnergySaver`, which repeatedly takes data from sensors in the car. The subroutine decides whether or not to set the `EngineStop` value.

The table of identifiers used by this subroutine is shown below.

- (a)** Complete the identifier table below by stating the data types.

Identifier	Data type	Description
Accelerator		Accelerator pedal position Values: 0 to 100 in steps of 1 Meaning: 0: none (not pressed) 100: maximum (fully pressed)
EngineTemp		Engine temperature in °C (-50 to +150 stored to 1 decimal place)
NormalTemp		Normal engine temperature in °C Whole number; typical value 90
Speed		Road speed of car (in km/hr) Values: 0 to 200 in steps of 1
EngineStop		Value used to signal engine must be stopped Possible values: TRUE: stop engine FALSE: run engine

[5]

The condition for stopping the engine is that all three of the following are true:

- Accelerator is not pressed
- Engine temperature is normal or above
- Car speed is zero

The initial design stage will produce a prototype of `EnergySaver`, with a user interface.

The structured English for this is:

1. INPUT value for accelerator pedal position
2. INPUT value for engine temperature
3. INPUT value for normal engine temperature
4. INPUT value for car speed
5. EVALUATE engine stopping condition
6. IF stopping condition satisfied SET engine stop value to TRUE
7. IF stopping condition not satisfied SET engine stop value to FALSE
8. OUTPUT message indicating engine stop value

- (b) Write the **pseudocode** equivalent of the structured English. Use the identifiers from the table in **part (a)**.

For the built-in functions list, refer to the **Appendix** on the last page.

[6]

Appendix of this paper:

Appendix

Built-in functions (Pseudocode)

In each function below, if the function call is not properly formed, the function returns an error.

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns the string of length y starting at position x from ThisString

Example: **MID**("ABCDEFGH", 2, 3) will return string "BCD"

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns the leftmost x characters from ThisString

Example: **LEFT**("ABCDEFGH", 3) will return string "ABC"

RIGHT(ThisString: STRING, x : INTEGER) RETURNS STRING

returns the rightmost x characters from ThisString

Example: **RIGHT**("ABCDEFGH", 3) will return string "FGH"

CHR(x : INTEGER) RETURNS CHAR

returns the character whose ASCII value is x

Example: **CHR**(87) will return 'W'

ASC(x : CHAR) RETURNS INTEGER

returns the ASCII value of character x

Example: **ASC**('W') will return 87

LCASE(x : CHAR) RETURNS CHAR

returns the lower case equivalent character of x

Example: **LCASE**('W') will return 'w'

UCASE(x : CHAR) RETURNS CHAR

returns the upper case equivalent character of x

Example: **UCASE**('h') will return 'H'

```
INT(x : REAL) RETURNS INTEGER
```

returns the integer part of x

Example: **INT(27.5415)** will return **27**

133. O/N 15/P21/Q3/b

3 A program is to simulate the operation of a particular type of logic gate.

- The gate has two inputs (TRUE or FALSE) which are entered by the user.
- The program will display the output (TRUE or FALSE) from the gate.

The program uses the following identifiers in the pseudocode below:

Identifier	Data type	Description
InA	BOOLEAN	Input signal
InB	BOOLEAN	Input signal
OutZ	BOOLEAN	Output signal

```

01  INPUT InA
02  INPUT InB
03  IF (InA = FALSE AND InB = FALSE) OR (InA = FALSE AND InB = TRUE)
                                OR (InA = TRUE AND InB = FALSE)
04      THEN
05          OutZ ← TRUE
06      ELSE
07          OutZ ← FALSE
08  ENDIF
09  OUTPUT OutZ

```

(b) The selection statement (lines 03 – 08) could have been written with more simplified logic.

Rewrite this section of the algorithm in **pseudocode**.

[3]

134. O/N 15/P21/Q4

4 A program is to be written to calculate the discount given on purchases.

A purchase may qualify for a discount depending on the amount spent. The purchase price (Purchase), the discount rate (DiscountRate) and amount paid (Paid) is calculated as shown in the following pseudocode algorithm.

```
INPUT Purchase
```

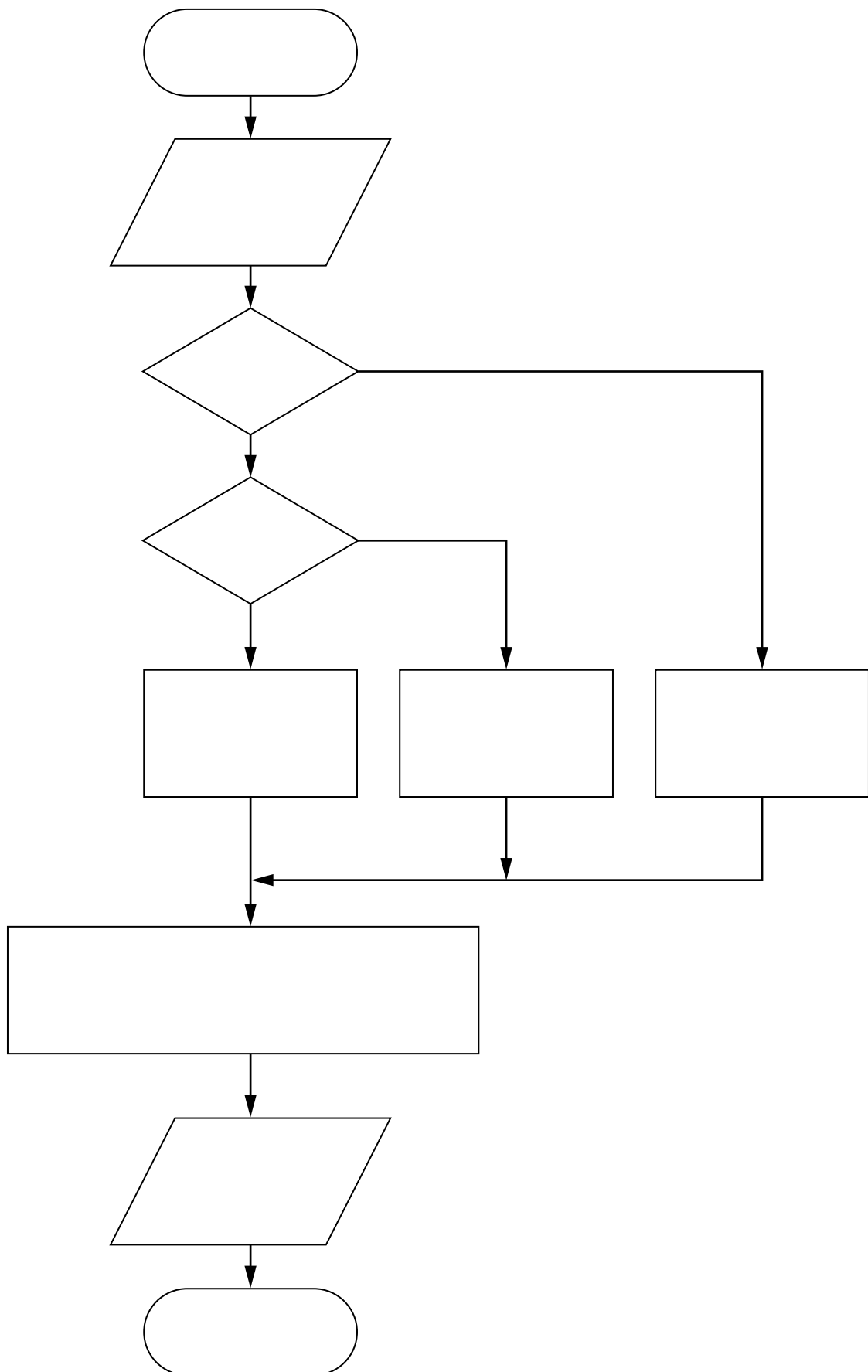
```
IF Purchase > 1000
  THEN
    DiscountRate ← 0.10
  ELSE
    IF Purchase > 500
      THEN
        DiscountRate ← 0.05
      ELSE
        DiscountRate ← 0
    ENDIF
  ENDIF
ENDIF

Paid ← Purchase * (1 - DiscountRate)
OUTPUT Paid
```

The algorithm is also to be documented with a program flowchart.

Complete the flowchart by:

- filling in the flowchart boxes
- labelling, where appropriate, lines of the flowchart



135. O/N 15/P21/Q5/a

5 A driver buys a new car.

The value of the car reduces each year by a percentage of its current value.

The percentage reduction is:

- in the first year, 40%
- in each following year, 20%

The driver writes a program to predict the value of the car in future years.

The program requirements are:

- enter the cost of the new car (to nearest \$)
- calculate and output the value of the car at the end of each year
- the program will end when either the car is nine years old, or when the value is less than \$1000

(a) Study the incomplete pseudocode which follows in **part (b)** and fill in the identifier table.

Identifier	Data type	Description

[3]

136. O/N 15/P22/Q2/b

2 A program is to simulate the operation of a particular type of logic gate.

- The gate has two inputs (0 or 1) which are entered by the user.
- The program will display the output (0 or 1) from the gate.

The program uses the following identifiers in the pseudocode below:

Identifier	Data type	Description
P	INTEGER	Input signal
Q	INTEGER	Input signal
X	INTEGER	Output signal

```

01 INPUT P
02 INPUT Q
03 IF (P = 1 AND Q = 0) OR (P = 0 AND Q = 1) OR (P = 0 AND Q = 0)
04     THEN
05         X ← 0
06     ELSE
07         X ← 1
08 ENDIF
09 OUTPUT X

```

(b) The selection statement (lines 03 – 08) could have been written with more simplified logic.

Rewrite this section of the algorithm in **pseudocode**.

[3]

137. O/N 15/P22/Q3

3 Regular customers at a supermarket use a rewards card at the point-of-sale.

Points are calculated from every transaction and added to the points total stored on the card.

One reward point is given for every \$1 spent.

When the points total exceeds 500, the customer can either:

- pay the full amount due and increase their points total
- get \$1 deducted from the amount due in exchange for 500 reward points

The new points total and amount to be paid is printed on the receipt.

A program is to be written with the following specification:

- read the points total from the card
- process the amount spent
- output the amount to be paid and the new points total

A user-defined function `CalculatePoints` has already been coded to calculate the new points earned from the amount spent.

Study the following pseudocode:

```
INPUT AmountDue
NewPoints ← CalculatePoints(AmountDue)
PointsTotal ← PointsTotal + NewPoints

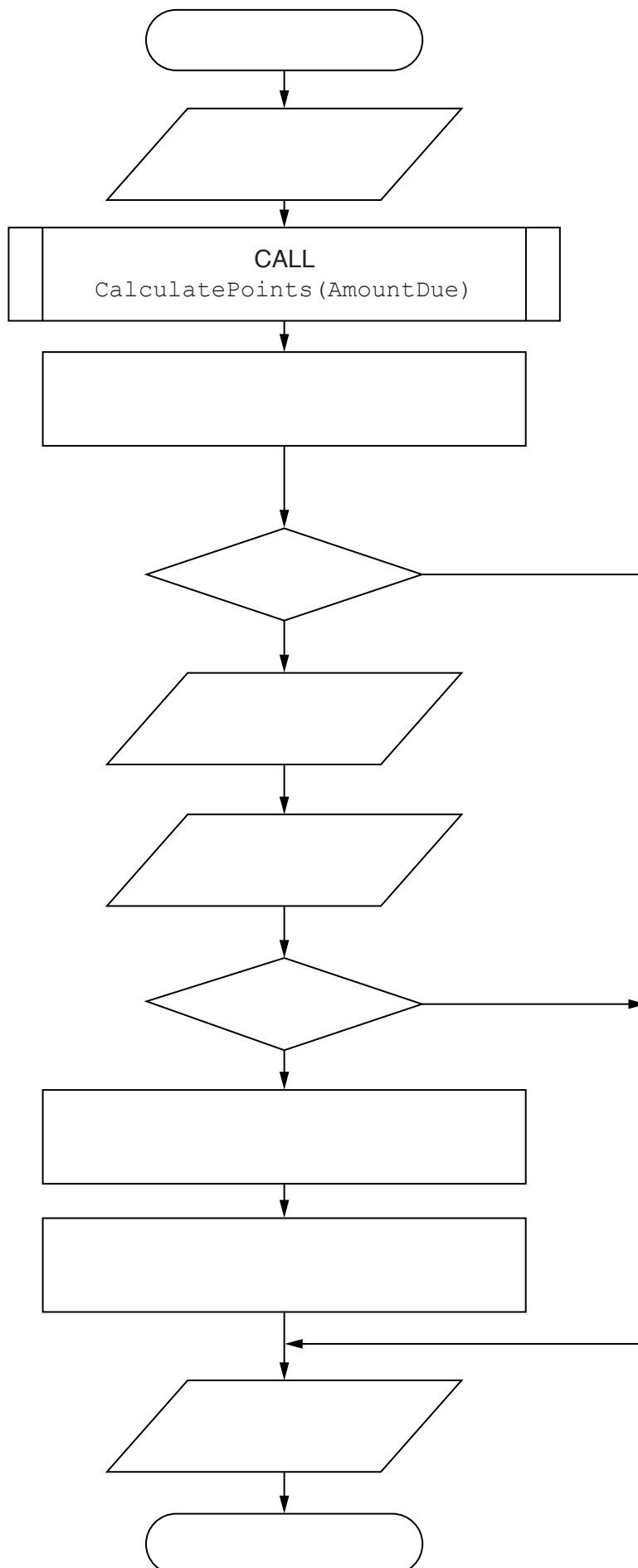
IF PointsTotal > 500
    THEN
        OUTPUT "Exchange points?"
        INPUT Response
        IF Response = "YES"
            THEN
                PointsTotal ← PointsTotal - 500
                AmountDue ← AmountDue - 1
            ENDIF
        ENDIF
    ENDIF

OUTPUT AmountDue, PointsTotal
```

The algorithm is also to be documented with a program flowchart.

Complete the flowchart by:

- filling in the flowchart boxes
- labelling, where appropriate, lines of the flowchart



138. O/N 15/P22/Q5/b

5 A program is to process a set of integers.

The integers are stored in an array, Num. The first N elements are to be processed.

The pseudocode for this program is shown below:

```

FOR i ← 1 TO (N - 1)
  j ← 1
  REPEAT
    IF Num[j] > Num[j + 1]
      THEN
        Temp ← Num[j]
        Num[j] ← Num[j + 1]
        Num[j + 1] ← Temp
      ENDIF
    j ← j + 1
  UNTIL j = (N - i + 1)
ENDFOR

```

(b) Complete the identifier table documenting the use of each of the variables.

Identifier	Data type	Description
Num	ARRAY[1:100] OF INTEGER	The array of numbers.
N		
i		
j		
Temp		

[5]

139. O/N 15/P22/Q6/b(i)

6 Some pseudocode statements follow which use the following built-in functions:

ONECHAR(ThisString : STRING, Position : INTEGER) RETURNS CHAR
 returns the single character at position Position (counting from the start of the string with value 1) from the string ThisString.
 For example: ONECHAR("Barcelona", 3) returns 'r'.

CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER
 returns the number of characters in the string ThisString.
 For example: CHARACTERCOUNT("South Africa") returns 12.

(b) A program is to be written as follows:

- the user enters a string
- the program will form a new string with all <Space> characters removed
- the new string is output

```
NewString ← ""
INPUT InputString

j ← CHARACTERCOUNT(InputString)
FOR i ← 1 TO j
    NextChar ← ONECHAR(InputString, i)
    IF NextChar <> " "
        THEN
            // the & character joins together two strings
            NewString ← NewString & NextChar
        ENDIF
    ENDFOR

OUTPUT NewString
```

(i) Complete the identifier table below.

Identifier	Data type	Description
InputString	STRING	The string value input by the user

[4]

140. O/N 15/P22/Q7/b(ii)

7 ASCII character codes are used to represent a single character.

Part of the code table is shown below.

ASCII code table (part)

Character	Decimal	Character	Decimal	Character	Decimal
<Space>	32	I	73	R	82
A	65	J	74	S	83
B	66	K	75	T	84
C	67	L	76	U	85
D	68	M	77	V	86
E	69	N	78	W	87
F	70	O	79	X	88
G	71	P	80	Y	89
H	72	Q	81	Z	90

Some pseudocode statements follow which use these built-in functions:

`CHARACTERCOUNT(ThisString : STRING) RETURNS INTEGER`
 returns the number of characters in the string `ThisString`.
 For example: `CHARACTERCOUNT("South Africa")` returns 12.

`CHR(ThisInteger : INTEGER) RETURNS CHAR`
 returns the character with ASCII value `ThisInteger`.
 For example: `CHR(66)` returns 'B'.

`ASC(ThisCharacter : CHAR) RETURNS INTEGER`
 returns the ASCII value for character `ThisCharacter`.
 For example: `ASC('B')` returns 66.

(b) A program is to be written to input a message string and then encrypt the message.

Study the following pseudocode:

```

OUTPUT "Enter message"
INPUT MyMessage
EncryptString ← ""

FOR i ← 1 TO CHARACTERCOUNT(MyMessage)
    NextNum ← ASC(ONECHAR(MyMessage, i) + 3)
    EncryptString ← EncryptString & CHR(NextNum)
ENDFOR

OUTPUT EncryptString
  
```

(ii) Describe the encryption algorithm used to encrypt the message string entered by the user.

141. O/N 15/P23/Q3/b

3 A program is to simulate the operation of a particular type of logic gate.

- The gate has two inputs (TRUE or FALSE) which are entered by the user.
- The program will display the output (TRUE or FALSE) from the gate.

The program uses the following identifiers in the pseudocode below:

Identifier	Data type	Description
InA	BOOLEAN	Input signal
InB	BOOLEAN	Input signal
OutZ	BOOLEAN	Output signal

```

01  INPUT InA
02  INPUT InB
03  IF (InA = FALSE AND InB = FALSE) OR (InA = FALSE AND InB = TRUE)
                                OR (InA = TRUE AND InB = FALSE)
04      THEN
05          OutZ ← TRUE
06      ELSE
07          OutZ ← FALSE
08  ENDIF
09  OUTPUT OutZ

```

(b) The selection statement (lines 03 – 08) could have been written with more simplified logic.

Rewrite this section of the algorithm in **pseudocode**.

[3]

142. O/N 15/P23/Q4

4 A program is to be written to calculate the discount given on purchases.

A purchase may qualify for a discount depending on the amount spent. The purchase price (Purchase), the discount rate (DiscountRate) and amount paid (Paid) is calculated as shown in the following pseudocode algorithm.

```

INPUT Purchase
IF Purchase > 1000
    THEN
        DiscountRate ← 0.10
    ELSE
        IF Purchase > 500
            THEN
                DiscountRate ← 0.05
            ELSE
                DiscountRate ← 0
        ENDIF
    ENDIF
ENDIF

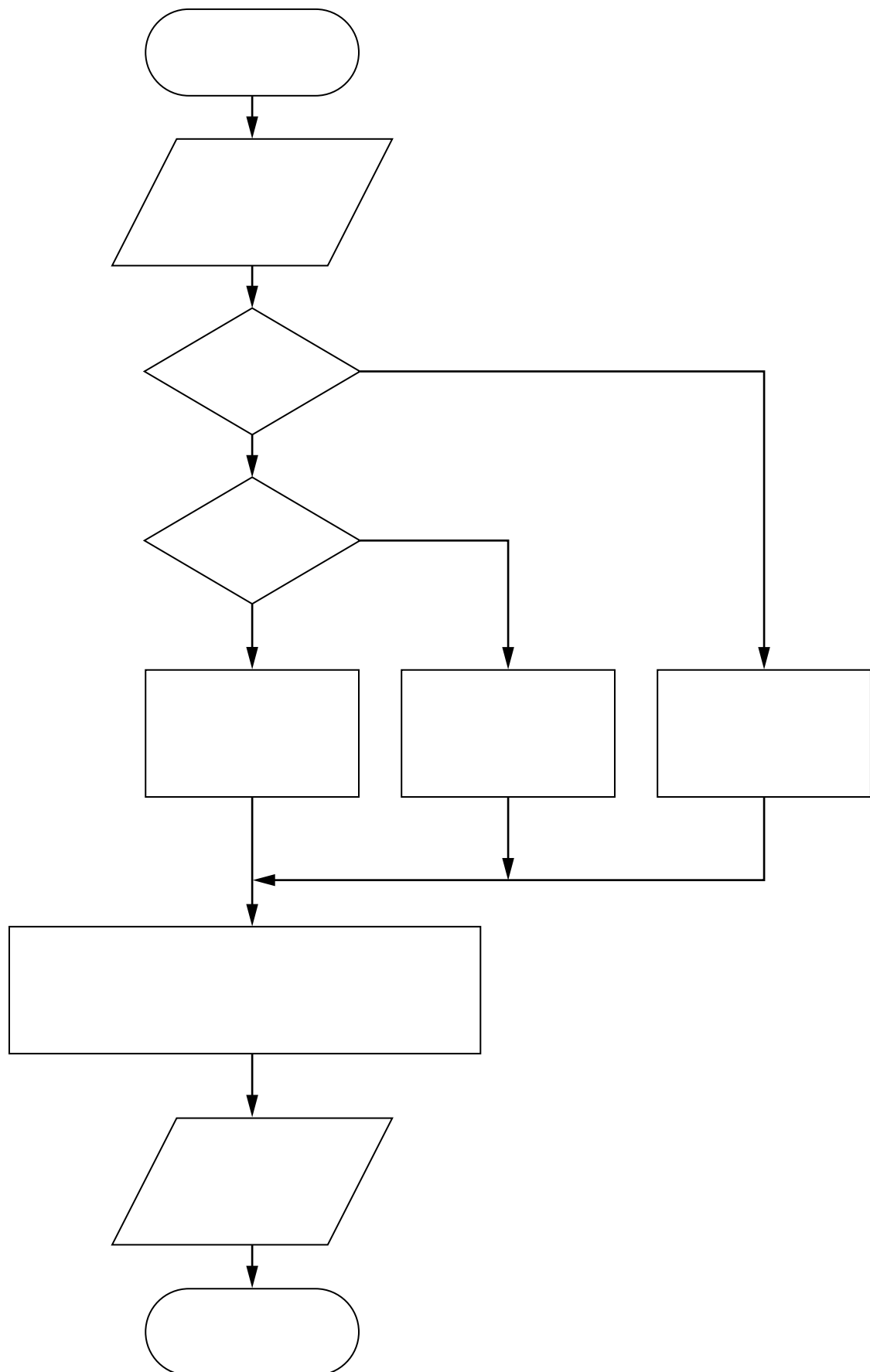
Paid ← Purchase * (1 - DiscountRate)
OUTPUT Paid

```

The algorithm is also to be documented with a program flowchart.

Complete the flowchart by:

- filling in the flowchart boxes
- labelling, where appropriate, lines of the flowchart



143. O/N 15/P23/Q5/a

5 A driver buys a new car.

The value of the car reduces each year by a percentage of its current value.

The percentage reduction is:

- in the first year, 40%
- in each following year, 20%

The driver writes a program to predict the value of the car in future years.

The program requirements are:

- enter the cost of the new car (to nearest \$)
- calculate and output the value of the car at the end of each year
- the program will end when either the car is nine years old, or when the value is less than \$1000

(a) Study the incomplete pseudocode which follows in **part (b)** and fill in the identifier table.

Identifier	Data type	Description

[3]

144. M/J 15/P21/Q1/a,b(i)

- 1** A marathon runner records their time for a race in hours, minutes and seconds.

An algorithm is shown below in structured English.

INPUT race time as hours, minutes and seconds

CALCULATE race time in seconds

STORE race time in seconds

OUTPUT race time in seconds

- (a)** The identifier table needs to show the variables required to write a program for this algorithm.

Complete the table.

Identifier	Data type	Description
RaceHours	INTEGER	The hours part of the race time.

[3]

- (b)** Before the program is written, the design is amended.

The new design includes input of the runner's current personal best marathon time (in seconds).

The output will now also show one of the following messages:

- "Personal best time is unchanged"
- "New personal best time"
- "Equals personal best time"

- (i)** Show the additional variable needed for the new design.

Identifier	Data type	Description

[1]

145. M/J 15/P22/Q1/a,b(i)

- 1** A marathon runner records their time for a race in hours, minutes and seconds.

An algorithm is shown below in structured English.

INPUT race time as hours, minutes and seconds

CALCULATE race time in seconds

STORE race time in seconds

OUTPUT race time in seconds

- (a)** The identifier table needs to show the variables required to write a program for this algorithm.

Complete the table.

Identifier	Data type	Description
RaceHours	INTEGER	The hours part of the race time.

[3]

- (b)** Before the program is written, the design is amended.

The new design includes input of the runner's current personal best marathon time (in seconds).

The output will now also show one of the following messages:

- "Personal best time is unchanged"
- "New personal best time"
- "Equals personal best time"

- (i)** Show the additional variable needed for the new design.

Identifier	Data type	Description

[1]

Answers Section

1. M/J 26/P21/Q2/a(ii,iii)

2(a)(ii)	MainsOn (= TRUE) AND WaterOn (= TRUE) Or reverse order ...	1
2(a)(iii)	IF MainsOn AND (Tray1 OR Tray2) THEN WashStart ← TRUE ENDIF OR: WashStart ← MainsOn AND (Tray1 OR Tray2)	3

2. M/J 26/P21/Q4

4	<i>Example solution:</i> Initialise Count to zero REPEAT INPUT integer number IF integer number > 0 then increment the count UNTIL integer number equals 0 OUTPUT Count <i>Mark as follows:</i> MP1 Initialise Count to zero and later evidence of use of counter MP2 Conditional loop and terminated when 0 is input MP3 Test for number > 0 and increment count inside the loop MP4 Output the count (either 'sum' or 'count') Max 4	4
---	--	---

3. M/J 26/P22/Q2/a,b,c

2(a)	One mark for either explanation: - To filter out <u>information / data</u> that is not necessary for the task - To keep only <u>information / data</u> that is necessary for the task Max 1 mark	1	
2(b)	One mark for each TWO data items Example answers 1 Tool ID // Tool name // Type of tool 2 Customer ID // Customer name // Customer address // Account reference 3 Hire ID 4 (Hire) Date 5 Hire time 6 Length of hire: Full or Half day 7 Cost of hire 8 Payment details 9 Tool status Max 2 marks	2	Reject Quantity Reject Return date if Hire date already given MP 2 Reject just 'Customer'
2(c)	One mark for each process. Examples include: 1 Input customer details / customer ID / payment details / Tool ID 2 Search for required tool // Checking if tool returned // Checking if tool available // Searching for dates when tool is available 3 Record tool booking // Amend Tool booking // Create tool booking // Record when tool returned // Confirm booking 4 Calculating time when tool has to be returned 5 Change / check tool status e.g. hired / available / damaged // Prevent same tool being hired twice 6 Process payment / calculate hire cost Max 2 marks	2	

4. M/J 26/P22/Q3

3	Example solution Mark points: MP 1 (Step 4) Open <u>WordList.txt</u> / File in read mode (and subsequently close) MP 2 (Step 5) Read the next line from the file in a loop MP 3 (Step 6) Increment line counter in a loop MP 4 (Step 7) Check if the line read equals the word (input) in a loop MP 5 (Step 8) ...If it does output line counter MP 6 (Step 9) If word not found in file then output "Not Found" after all iterations MP 7 (Step 10) Otherwise repeat from step 5 Max 6 marks from 7 possible marks	6 MP 1 cannot be in a loop structure Do not accept once if MP 2, 3, 4 is clearly not in a loop Award only one mark for MP 5 and MP 6 if both outputs are possible MP 7 Can be any loop but it must be clear where it starts and ends and must include at least one of the end conditions.
---	--	--

5. M/J 26/P23/Q2/b,c

2(b)	<pre>IF MainsOn (=TRUE) THEN DisplayOn ← TRUE ENDIF</pre> Accept: <pre>DisplayOn ← MainsOn</pre> Accept '=' in place of the assignment arrow	1	
2(c)	MP1 The robot is only ready to move if... the mains power is on and MP2 The strength of the (suction) value selected is non-zero	2	Reject – Zero for a 'repeat' of the pseudocode statement.

6. M/J 26/P23/Q4

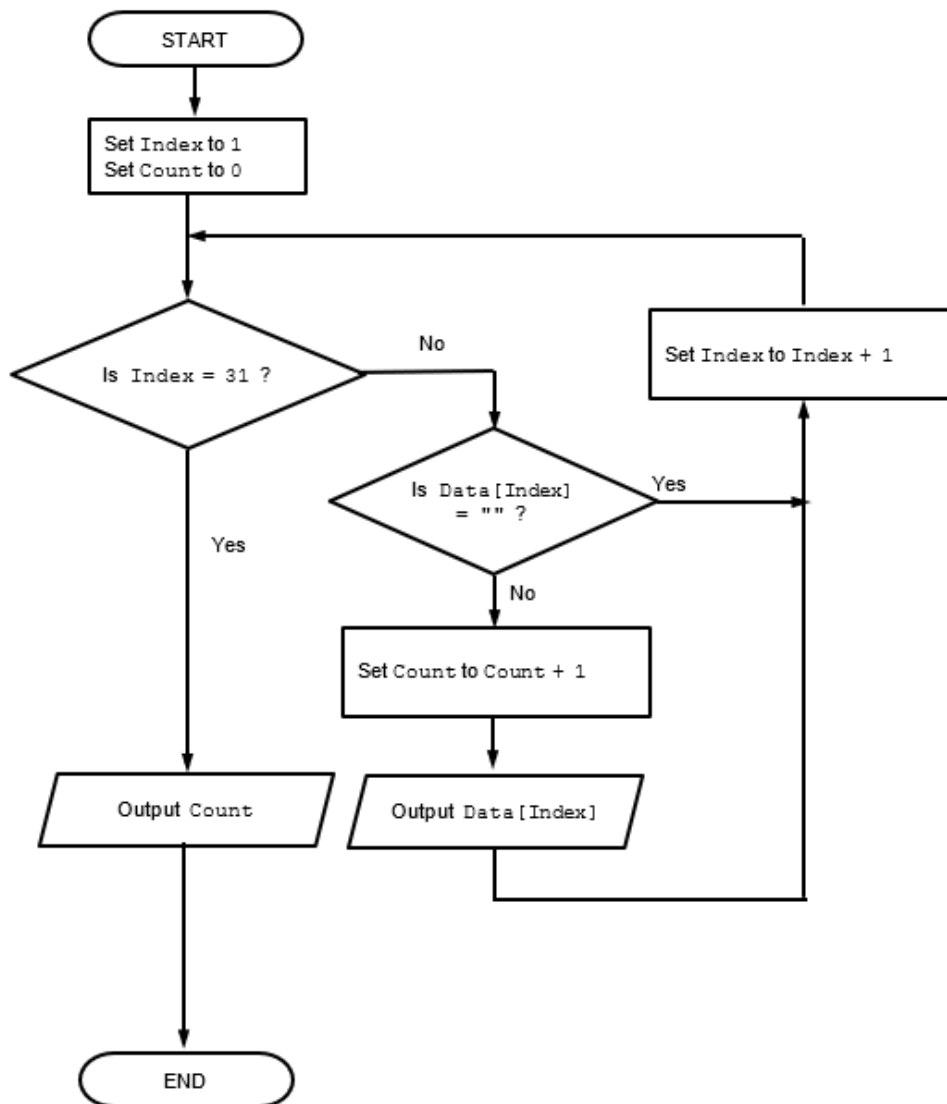
4	<i>Example solution:</i> • Initialise count to zero • REPEAT • Input next number • If number > 120 then increment the count • Until number = -1 • Output the count <i>Mark as follows:</i> MP1 Initialise count and later evidence of use of counter (not 'sum') MP2 Conditional loop and terminated when -1 is input MP3 Test for number > 120 and increment the count (not 'sum') inside the loop MP4 Output the count (Accept either 'sum' or 'count')	4	A 'sum' solution will have a Max 2.
---	---	---	--

7. O/N 25/P21/Q2

2

Example solution

5



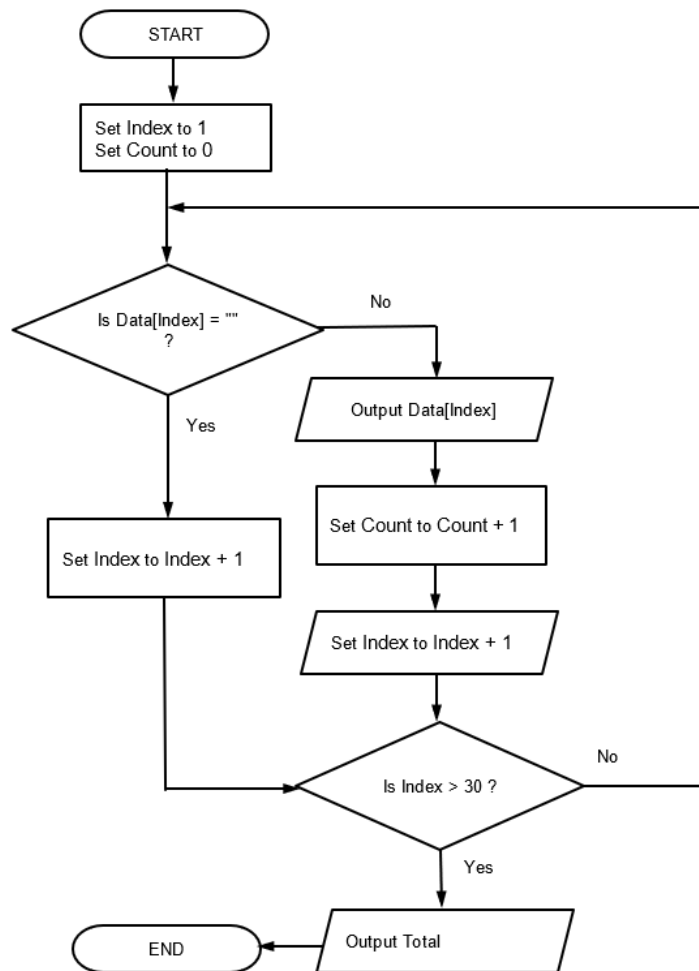
Mark as follows:

- 1 Initialisation of variables used as **array index** and **count**
- 2 Loop through exactly 30 elements in the array
- 3 Test for empty string in current array element // Test for non-empty string
- 4 Increment count if appropriate **in a loop**
- 5 Both required outputs under correct conditions with correct outputs

2

Alternative solution

Checking for empty string first



8. O/N 25/P22/Q1/a

1(a)

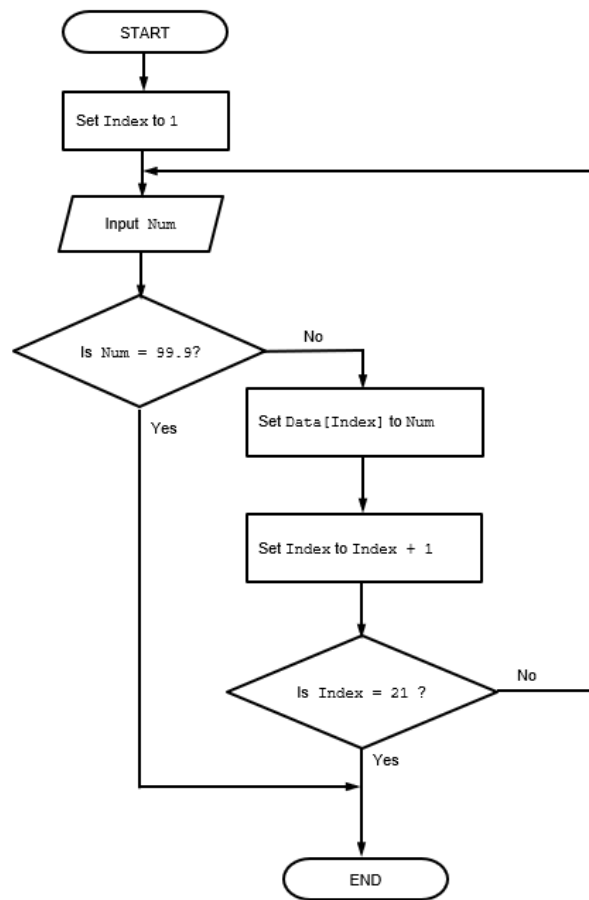
Pseudocode example	Selection	Iteration	Subroutine
IF Status = FALSE THEN FOR Count ← 1 TO 20 CALL Reset (Count) NEXT Count ENDIF	✓	✓	✓
OTHERWISE : Status ← TRUE	✓		
WHILE AllDone() = TRUE		✓	✓
30 : NextChar ← 'X'	✓		

One mark per row

4

9. O/N 25/P22/Q2

2



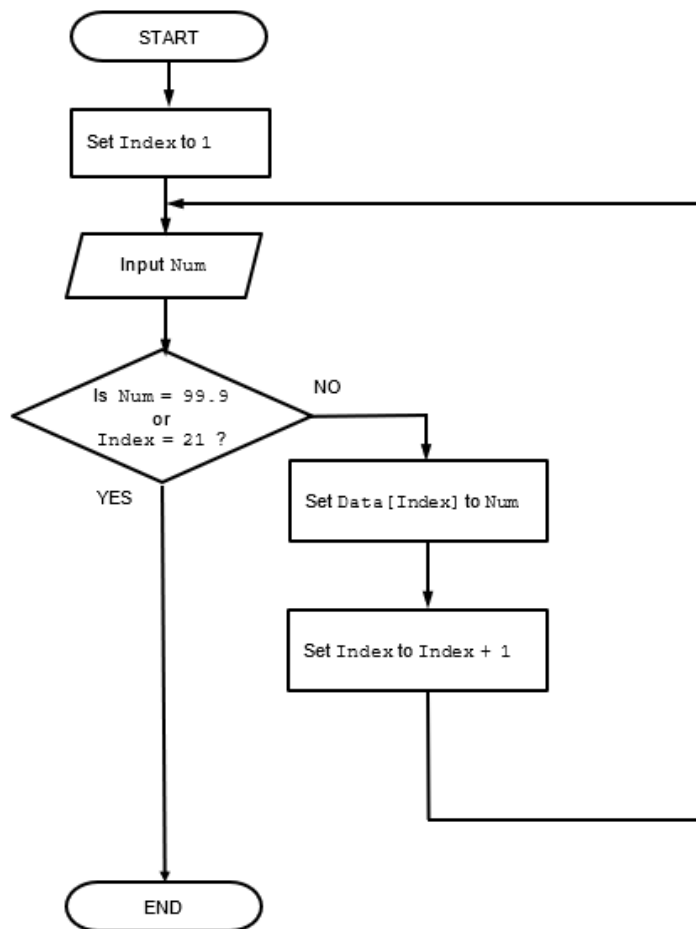
5

Mark as follows:

MP1 Initialisation of the array index**MP2** Input number **in a loop****MP3** Test for termination value (99.9) **and** correct ending**MP4** Test for array full / maximum 20 iterations **and** correct ending**MP5** Assign number to Data array element **and** increment index

2

Alternative solution:



10. O/N 25/P22/Q3/a

3(a)

Mark as follows:

6

MP1	1		open the file OldFile.txt in read (mode)
MP2	2		open the file NewFile.txt in write (mode)
	3		read a line from OldFile.txt and store in LineX
MP3	4		read a line from OldFile.txt and store in LineY
	5		read a line from OldFile.txt and store in LineZ
MP4	6		set NewString to LineX & '\ ' & LineY & '\ ' & LineZ
MP5	7		write NewString to NewFile.txt
MP6	8		repeat from step 3 until end of OldFile.txt
	9		close both files

11. O/N 25/P22/Q4/c

4(c)	MP1 Check that the parameter value is in the range 0 to 35 MP2 ... and if not, return FALSE MP3 Use the parameter value as the index to array <code>CheckTotal</code> MP4 Return the value from given index position	4
------	---	----------

12. O/N 25/P23/Q2/a,b

2(a)	One mark per point: 1 Open the file in append mode (and subsequently close) before loop 2 Initialise a variable to 1 / 0 before loop 3 Use the variable as an index to the array to access an element in a loop 4 Test if element value is blank / empty in a loop 5 If not blank then write the value to the file in a loop 6 Increment the variable in a loop 7 Repeat from MP3 until variable value is 66 / 65 / all elements (in the array) have been checked Max 6	6
2(b)	One mark per point: Construct: Selection Use: To test whether the (array) element is blank Alternative: Construct: Sequence Use: To specify the order in which the steps of the algorithm have to be followed so that the task is completed correctly	2

13. M/J 25/P21/Q1/a,b,c

1(a)

Example value	Explanation	Variable name	Data type
"Fruit"	a category of stock that is sold in the shop	Category	STRING
20/02/2025	when an item was sold	DateSold	DATE
12.67	the cost of an item	(Item) Cost/Price	REAL
TRUE	to indicate if an item is in stock	(Item) InStock	BOOLEAN

Mark as follows:
1 mark per row for each variable name **and** data type

4

1(b)	<table><tr><th>Pseudocode extract</th><th>Assignment</th><th>Selection</th><th>Iteration</th></tr><tr><td>Result ← CalculateTotal()</td><td>✓</td><td></td><td></td></tr><tr><td>WHILE IsClosed</td><td></td><td></td><td>✓</td></tr><tr><td>REPEAT INPUT Value UNTIL Sales[4] > Value</td><td>✓</td><td></td><td>✓</td></tr><tr><td>IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF</td><td>✓</td><td>✓</td><td></td></tr><tr><td>CASE OF Option</td><td></td><td>✓</td><td></td></tr></table>	Pseudocode extract	Assignment	Selection	Iteration	Result ← CalculateTotal()	✓			WHILE IsClosed			✓	REPEAT INPUT Value UNTIL Sales[4] > Value	✓		✓	IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF	✓	✓		CASE OF Option		✓		5
	Pseudocode extract	Assignment	Selection	Iteration																						
	Result ← CalculateTotal()	✓																								
	WHILE IsClosed			✓																						
	REPEAT INPUT Value UNTIL Sales[4] > Value	✓		✓																						
	IF Sales[Current] <= 150 THEN Discount ← TRUE ENDIF	✓	✓																							
	CASE OF Option		✓																							
Mark as follows: One mark for each correct row																										

1(c)	Decomposition involves: MP1 Breaking down a problem into sub problems MP2 In order to explain / understand the process/task of how a (stock control) can be managed in more detail MP3 Leading to the concept of the program designed as a series of modules / procedures / functions // or by example of different stock control modules MP4 Makes the program easier to test / debug Max 3 Max 2 if no mention of stock control	3
------	---	---

14. M/J 25/P21/Q3

3	Example solution: <pre> DECLARE GeneratedValue : INTEGER DECLARE Guess : INTEGER GeneratedValue ← INT(RAND(100)) + 1 REPEAT OUTPUT "Enter a guess between 1 and 100: " INPUT Guess IF Guess > GeneratedValue THEN OUTPUT "Guess was too high" ENDIF IF Guess < GeneratedValue THEN OUTPUT "Guess was too low" ENDIF UNTIL Guess = GeneratedValue OUTPUT "Number guessed correctly" MP1 Declare all variables used MP2 Uses RAND() function MP3 Uses INT() function MP4 Correct syntax for random number between 1 and 100 MP5 Conditional loop until correct number guessed MP6 Prompt and input a guess in a loop MP7 Check if guess is too high in a loop MP8 Check if guess is too low in a loop MP9 Output appropriate message (x2) when the number is not guessed correctly (in the loop) and output correct guess message Max 8 </pre>	8
---	--	---

15. M/J 25/P22/Q2/a

2(a)	Mark as follows: 1. To increase the level of detail of the algorithm // To break the problem / task into smaller steps ... 2. ... until steps can be directly translated into lines of code // from which it can be programmed	2
------	---	---

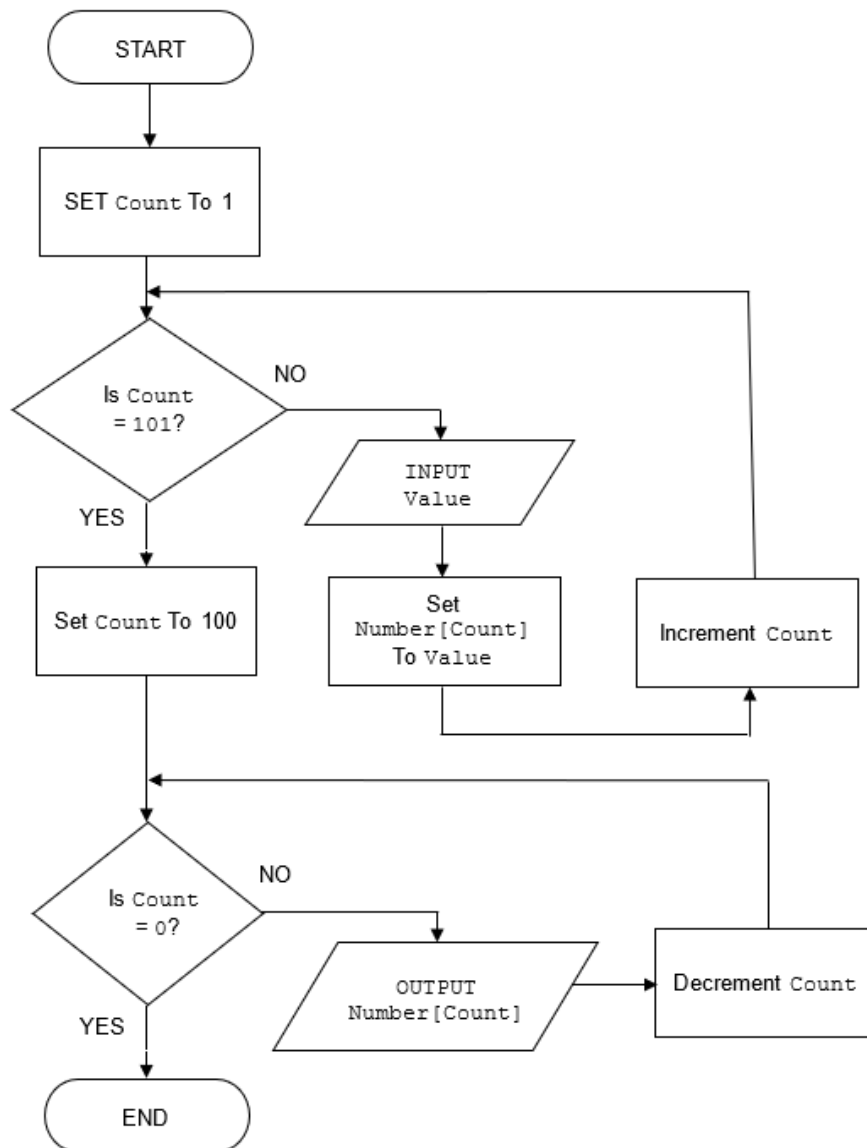
16. M/J 25/P22/Q3

3	Example solution <pre> DECLARE FirstNumber : INTEGER DECLARE SecondNumber : INTEGER DECLARE ThirdNumber : INTEGER FirstNumber ← INT(RAND(21)) - 10 OUTPUT FirstNumber REPEAT SecondNumber ← INT(RAND(21)) - 10 UNTIL FirstNumber <> SecondNumber OUTPUT SecondNumber IF FirstNumber < 0 AND SecondNumber < 0 THEN ThirdNumber ← INT(RAND(6)) + 30 OUTPUT ThirdNumber ENDIF </pre> Mark points: 1. Declare all variables used 2. Use <code>RAND()</code> function with any integer parameter 3. Use <code>INT()</code> function with any numeric parameter 4. Conditional loop until two different random numbers are generated 5. Use <code>INT()</code> function using random number generated between –10 and 10 inclusive in a loop 6. Output two different random integers / numbers 7. Check if both random integers / numbers are negative 8. then output a (third) random integer / number between 30 and 35 inclusive	8
---	--	---

17. M/J 25/P22/Q4

4

6



One mark for each functional group as listed:

1. Initialise Index used for `Number` to either 1 or 0 before first loop
2. Loop 100 times
3. Input value
4. Increment array index **and** store value in `Number` array at element referenced by array `index` **in a loop**
5. Output loop starts at last element in `Number` array
6. Output `Number` array element referenced by `Index` **in a loop**
7. Output all elements of `Number` array **once** in reverse order

Max 6

18. M/J 25/P23/Q3

3	Example solution: <pre> FUNCTION RollDice (NoOfTimes : INTEGER) RETURNS REAL DECLARE RollValue : INTEGER DECLARE Average : REAL DECLARE Count : INTEGER DECLARE Sum : INTEGER Sum ← 0 FOR Count ← 1 TO NoOfTimes RollValue ← INT(RAND(6)) + 1 OUTPUT RollValue Sum ← Sum + RollValue NEXT Count Average ← Sum / NoOfTimes RETURN Average ENDFUNCTION </pre> MP1 Declare all variables used and initialise Sum to 0 MP2 Loop for 'number of times' parameter MP3 Use RAND() function with Integer parameter in a loop MP4 Use INT() function in a loop MP5 Generate a random integer between 1 and 6 in a loop MP6 Output each value generated in a loop MP7 Calculate Sum and if Average used check declared as REAL and return Average and check function header returns REAL	7
---	---	---

19. M/J 25/P23/Q4/a

4(a)	MP1 Open the file (Sales.txt) in read mode and subsequently close MP2 Read a line (from the text file) MP3 Convert the line read (string) to a number MP4 If the number is greater than 500 and then output week number MP5 Increment week number (must have been Initialised ...) MP6 Repeat from step 2 for 52 times // Repeat from step 2 until end of file Max 5	5
------	---	---

20. O/N 24/P21/Q3/a,b

3(a)	One mark per emboldened part: 1 Open the file (Result.txt) in <u>read</u> mode 2 Loop until EOF(Result.txt) // EOF // end of file 3 Read a / the / next line (from the file) and store in ThisLine 4 Assign LineY to LineX 5 Assign LineZ to LineY 6 Assign ThisLine to LineZ 7 After the loop, close the file (Result.txt) 8 Output LineX, LineY, LineZ	4
3(b)	Example answer: So that the last three lines of the file are output in the correct order A mark for mentioning one of: • (Ensuring) the lines are output in correct order • Ordering the last <u>three</u> lines • Ordering the lines so they are in the same order as (they occur) <u>in the file</u> Max 1 marks	1

21. O/N 24/P22/Q7/a,b

7(a)	• Customer ID – to reference the other customer details • Email address – to send the email • Name – to personalise the email • Date of last visit – to select which customers should receive an email • Unique voucher code (or method of code generation) – to include in the email Mark as follows: One mark per item and justification Max 3 marks	3
7(b)	Abstraction	1

22. O/N 24/P23/Q1/a

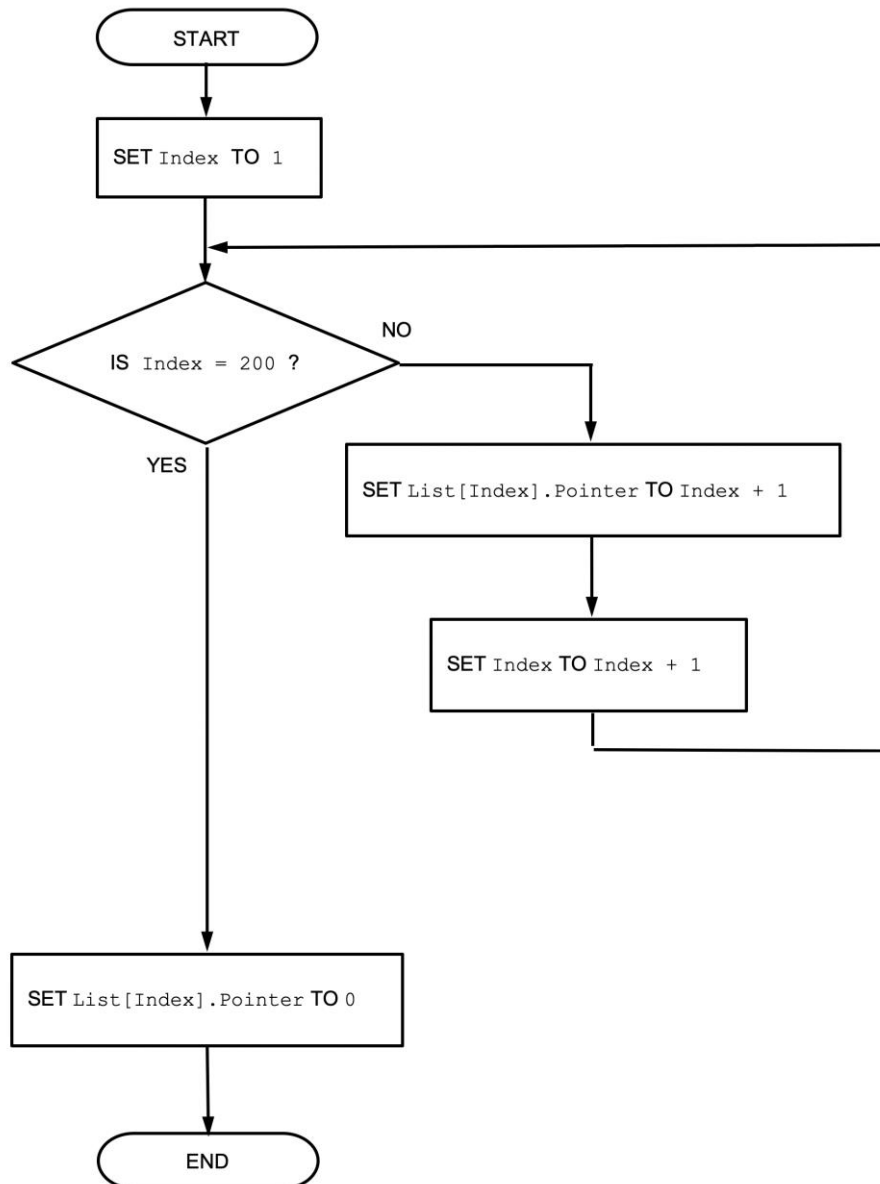
1(a)	Pseudocode example	Selection	Iteration	Subroutine	4
	<pre>FOR Index ← 1 TO 3 IF Safe[Index] = TRUE THEN Flap[Index] ← 0 ENDIF NEXT Index</pre>	✓	✓		
	CASE OF Compound(3)	✓		✓	
	REPEAT UNTIL AllDone() = TRUE		✓	✓	
	WHILE Result[3] <> FALSE		✓		
One mark per row					

23. O/N 24/P23/Q2/a,b

2(a)	<pre>DECLARE Count, Total, NextNumber : INTEGER Total ← 0 FOR Count ← 1 TO 100 OUTPUT "Input an integer value" INPUT NextNumber IF NextNumber > 0 THEN Total ← Total + NextNumber ENDIF NEXT Count OUTPUT Total</pre> Mark as follows: MP1 Declarations of all variables used MP2 Loop for 100 iterations MP3 Prompt and input a value in a loop and MP4 Test for value > 0 // >=1 in a loop MP5 Sum the Total in a loop MP6 Output of Total after the loop Max 5 marks	5
2(b)	MP1 Construct: Iteration / Repetition MP2 Use: To loop through all <u>100</u> inputs // To loop <u>100</u> times ALTERNATIVE: MP1 Construct: Selection MP2 Use: To test whether the value input is positive	2

24. O/N 24/P23/Q3/b,c

3(b)

4

Mark as follows:

- MP1** Use of variable as array index **and** initialisation to 1 / to first element of array
- MP2** Loop for 199 / 200 iterations
- MP3** Assign Index+1 to each pointer field in a loop
- MP4** Assign 0 to 200th pointer field

3(c)	One mark per step: MP1 Assign <code>HeadPointer</code> to <code>ThisPointer</code> / Current Pointer // Identify first node using <u>headpointer</u> MP2 Loop the following until <code>ThisPointer</code> value is 0 (zero) / null <u>pointer</u> reached MP3 ... Output data (field) from array element at index <code>ThisPointer</code> // Output data (field) of the current node / array element MP4 ... Assign pointer field from current array element / index / to <code>ThisPointer</code> / Current Pointer // Assign pointer field from current node to <code>ThisPointer</code> / Current Pointer	4
------	---	----------

25. O/N 24/P23/Q7/b

7(b)	One mark for name and use Mark as follows: Examples include: Sub-module: <code>ScanCard()</code> / <code>GetID()</code> Use: Read the barcode from the loyalty card / Get the customer ID from the barcode Sub-module: <code>GetPoints()</code> Use: Get the number of points the customer has Sub-module: <code>ExchangePoints()</code> / <code>UpdateCard()</code> Use: Reduce the number of loyalty points Sub-module: <code>GetDiscount()</code> / <code>CalculateDiscount()</code> Use: Calculate the discount Sub-module: <code>CalculatePoints()</code> Use: Calculate points (following a purchase) Sub-module: <code>UpdatePoints()</code> Use: Update total points a customer has (following a purchase) Sub-module: <code>ShowDiscount()</code> Use: Display the discount Max 4 marks	4
------	--	----------

26. M/J 24/P21/Q1/c

1(c)(i)	The names do not reflect / indicate the purpose of the variable // the names are not meaningful	1
1(c)(ii)	They make the program more difficult to understand / debug / maintain	1

1(c)(iii)	One mark from: • Indentation / use of white space • Capitalisation of keywords • Use of comments • Use of modular programming • Use of local variables Note: max 1 mark	1
-----------	--	----------

1(a)	Assignment statement	Data type	4
	A ← LEFT(MyName, 1)	CHAR / STRING	
	B ← Total * 2	INTEGER / REAL	
	C ← INT(ItemCost) / 3	REAL	
	D ← "Odd OR Even"	STRING	
	One mark per row		

27. M/J 24/P21/Q2/a

2(a)	<pre> graph TD Start([START]) --> Init[Set Tries to 0 Set Count to 0] Init --> Input[/INPUT A, B/] Input --> IncTries[Set Tries to Tries + 1] IncTries --> IsAB{Is A > B ?} IsAB -- Yes --> IncCount[Set Count to Count + 1] IsAB -- No --> IsCount10{Is Count = 10 ?} IncCount --> IsCount10 IsCount10 -- Yes --> Output[/OUTPUT Tries/] Output --> End([END]) IsCount10 -- No --> Input </pre> One mark per outlined region: 1 Initialise both counts 2 Increment <code>Tries</code> every time a pair is input 3 Compare <code>A > B</code> and increment <code>Count</code> if TRUE 4 Test for <code>Count = 10</code> (10th time <code>A > B</code>) – MUST include Yes / No labels 5 If so output <code>Tries</code>, otherwise loop	5
------	---	---

28. M/J 24/P21/Q7/a

7(a)(i)	To make the solution easier to design / implement / solve	1
---------	---	---

7(a)(ii)	One mark per item and justification Item: mobile phone number Justification: to send the text message Item: name Justification: to personalise the text message Item: exercise interest Justification: to determine whether this member would be interested	3
7(a)(iii)	Examples include: • Add a member to a list of those interested in the new class • Remove the member from future SMS messages • Read/process Message • Identify who from One mark for each. Max 2 marks	2

29. M/J 24/P22/Q1/a,c

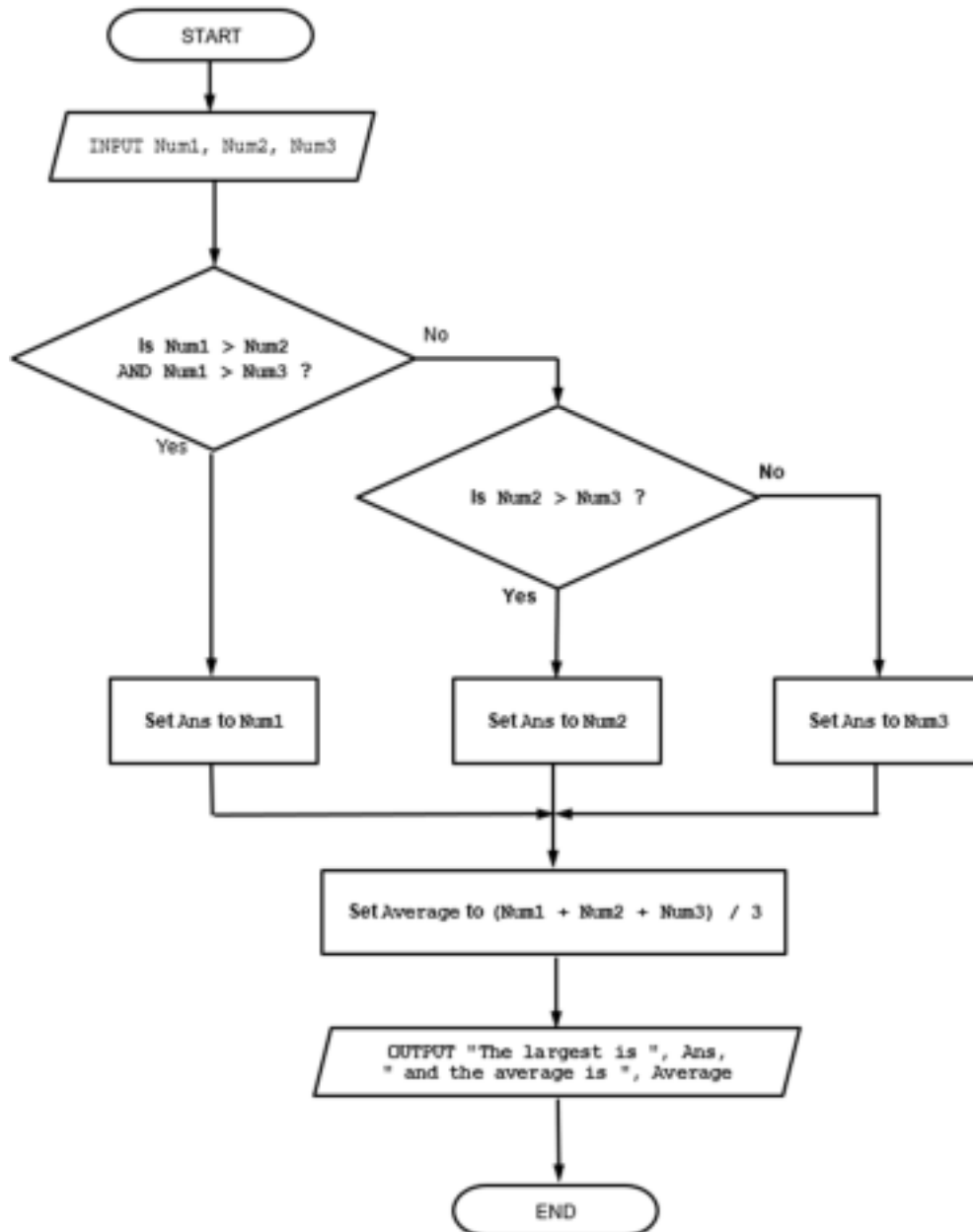
1(a)	<table><tr><th>Pseudocode example</th><th>Selection</th><th>Iteration</th><th>Input/Output</th></tr><tr><td>FOR Index ← 1 TO 10 Data[Index] ← 0 NEXT Index</td><td></td><td>✓</td><td></td></tr><tr><td>WRITEFILE ThisFile, "****"</td><td></td><td></td><td>✓</td></tr><tr><td>UNTIL Level > 25</td><td></td><td>✓</td><td></td></tr><tr><td>IF Mark > 74 THEN READFILE OldFile, Data ENDIF</td><td>✓</td><td></td><td>✓</td></tr></table> One mark per row.	Pseudocode example	Selection	Iteration	Input/Output	FOR Index ← 1 TO 10 Data[Index] ← 0 NEXT Index		✓		WRITEFILE ThisFile, "****"			✓	UNTIL Level > 25		✓		IF Mark > 74 THEN READFILE OldFile, Data ENDIF	✓		✓	4
Pseudocode example	Selection	Iteration	Input/Output																			
FOR Index ← 1 TO 10 Data[Index] ← 0 NEXT Index		✓																				
WRITEFILE ThisFile, "****"			✓																			
UNTIL Level > 25		✓																				
IF Mark > 74 THEN READFILE OldFile, Data ENDIF	✓		✓																			
1(c)	1 mark for stating a suitable way of documenting: • <u>Identifier table</u> 1 mark for giving one piece of information that should be recorded. examples include: Explanation of what (each) variable is used for The purpose of (each) variable An example of data values stored // Initialisation value	2																				

1(b)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																			
------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

30. M/J 24/P22/Q2/a,b

2(a)

5



Mark points

- 1 Condition for selecting one of the input numbers as largest value
- 2 ... and assign to Ans
- 3 Condition for selecting largest number for all three number input and assigning to Ans
- 4 Average calculated using Num1, Num2 and Num3 and stored in a variable.
Reject use of DIV
- 5 Output of Ans and average in output symbol / parallelogram

2(b)	Example solutions: <pre> Flag ← GetStat() WHILE Flag <> TRUE FOR Port ← 1 TO 3 CALL Reset(Port) NEXT Port Flag ← GetStat() ENDWHILE </pre> Alternative: <pre> REPEAT Flag ← GetStat() IF Flag <> TRUE THEN FOR Port ← 1 TO 3 CALL Reset(Port) NEXT Port ENDIF UNTIL Flag = TRUE </pre> One mark per point: 1 (Outer) conditional loop testing <code>Flag</code> 2 Correct assignment of <code>Flag</code> from <code>GetStat()</code> in a loop 3 (Inner) loop checking / counting port // Check if <code>Port</code> is different to 4 4 ... loop for 3 iterations 5 a call to <code>Reset()</code> in a loop	5
------	--	----------

31. M/J 24/P22/Q7/a

7(a)	Examples include: Module: <code>IdentifyMember()</code> Use: Identifies a club member who has expressed an interest in a given class Module: <code>GetMemberPhoneNumber()</code> Use: Gets the mobile phone number of a member Module: <code>CreateMessage()</code> Use: Generates a text message to a member Module: <code>SendMessage()</code> Use: Sends a text message to a member in the waiting list One mark for name and use Note: max 3 marks	3
------	--	----------

32. M/J 24/P22/Q8/a

8(a)	One mark for reason, one for benefit Reason: (Program is) easier to design / implement / test / debug / modify Benefit: Easier to check that each stage works as expected	2
------	---	----------

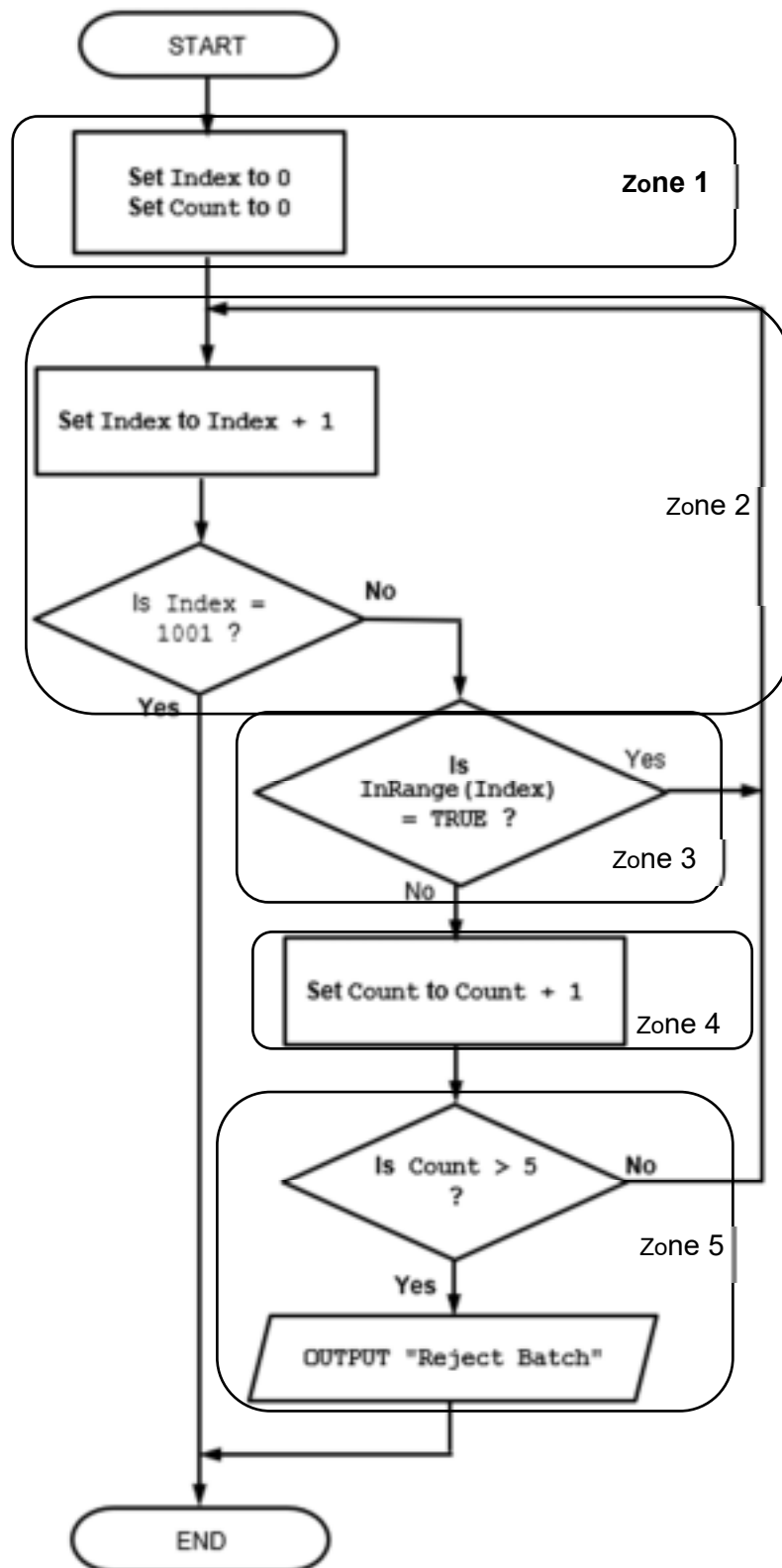
33. M/J 24/P23/Q2/a,b

2(a)	One mark per point: 1. Open the file (in read mode and subsequently close) 2. Initialise an index variable (to 1 // 0) 3. Repeat (the next three steps) until the end of file is reached 4. Read a line from the file (into a string variable) 5. Store the line / variable in the array at the index 6. Increment the index in a loop Note: max 5 marks	5
2(b)	One mark per point: Construct: a <u>conditional</u> loop Use: To keep repeating until the end of the file is reached ALTERNATIVE: Construct: a <u>selection</u> statement Use: To test / check the value returned by the EOF () function	2

34. M/J 24/P23/Q3/b

3(b)

5



One mark per zone

35. O/N 23/P21/Q2/a,b

2(a)	Max 5 marks MP1 Set total to <u>zero</u> MP2 Input a number MP3 Check if number greater than 29 and less than 71 MP4 ... if check is true - add number to total MP5 Repeat from step 2 99 times // for a total of 100 iterations MP6 Output the total	5
2(b)	MP1 An iterative construct // a (count-controlled) loop MP2 A selection construct // an IF statement	2

36. O/N 23/P22/Q1/a

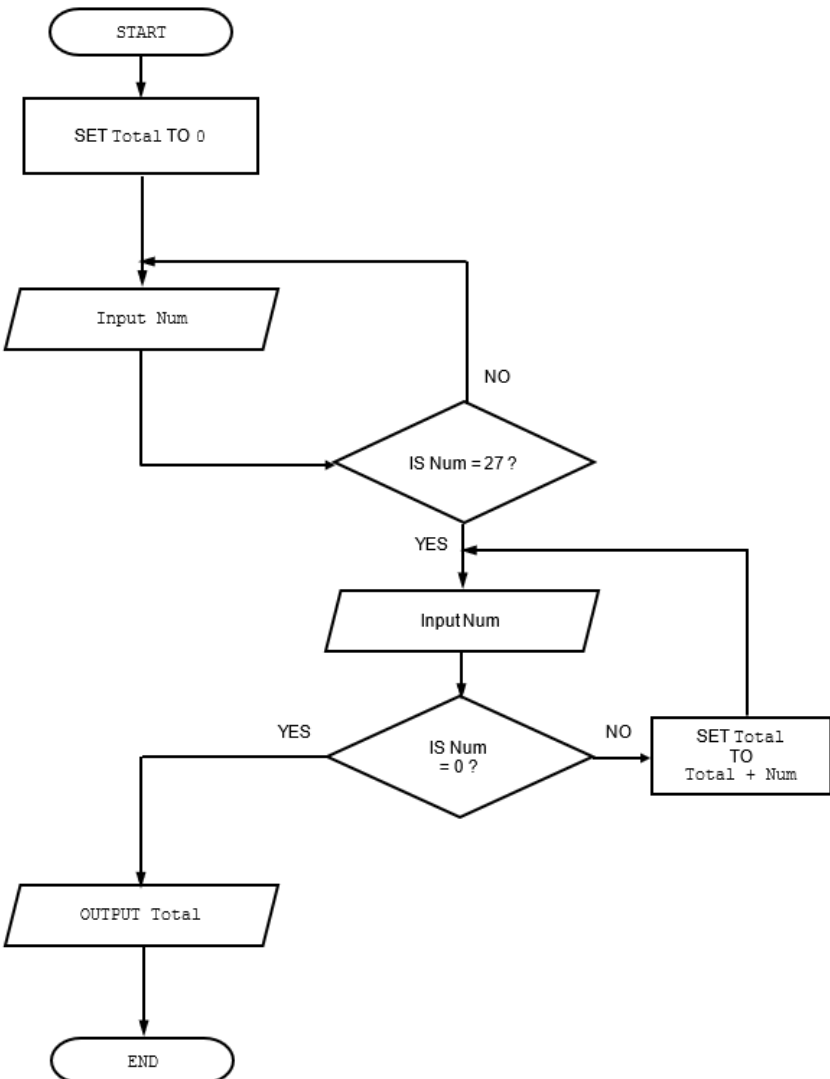
1(a)

4

One mark for each row with appropriate variable name and data type

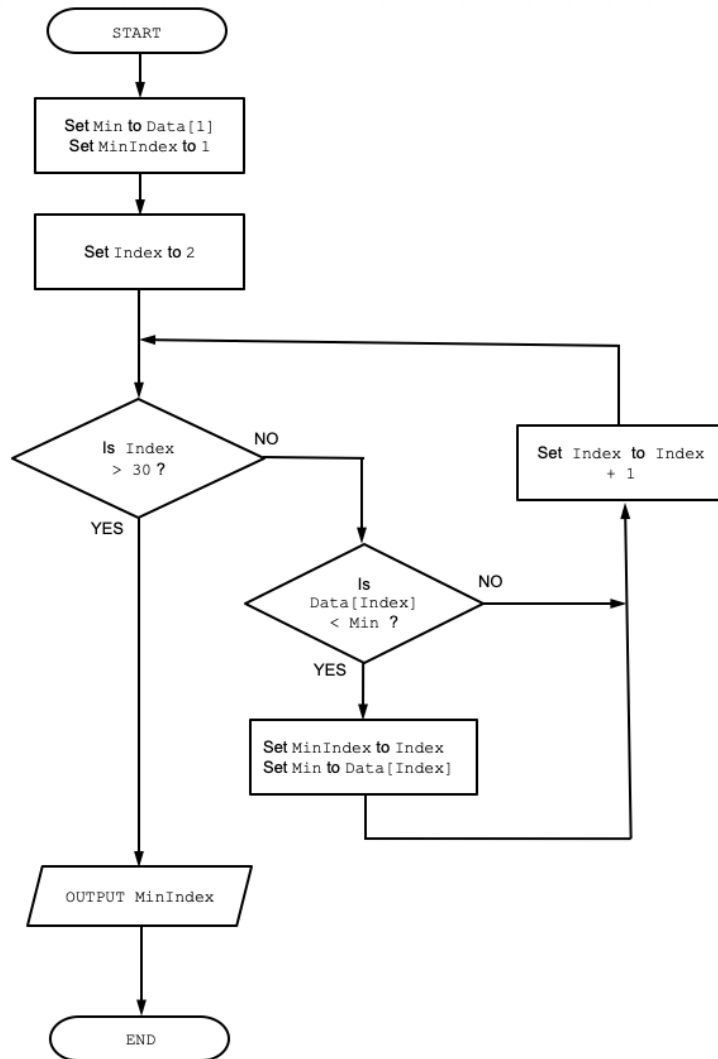
Example value	Explanation	Variable name	Data type
"Mr Khan"	The name of the customer	CustomerName	STRING
3	The number of items in the order	NumItems	INTEGER
TRUE	A value to indicate whether this is a new customer	NewCustomer	BOOLEAN
15.75	The deposit paid by the customer	Deposit	REAL

37. O/N 23/P22/Q2/a

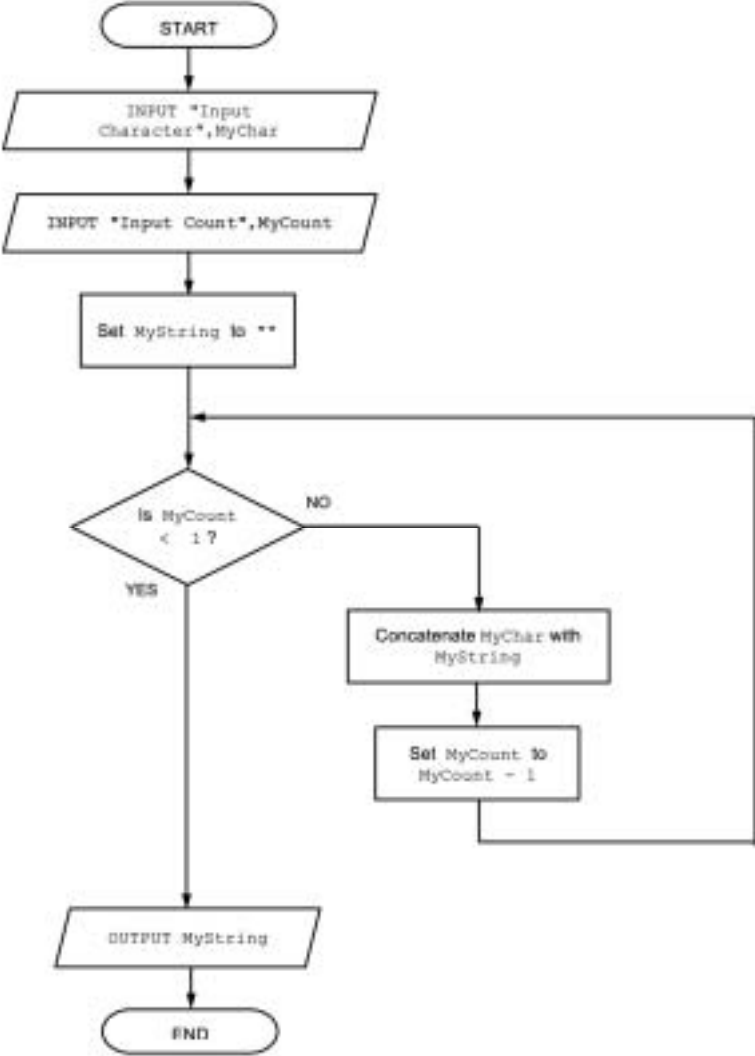
2(a)	Example Solution  <pre> graph TD START([START]) --> SET0[SET Total TO 0] SET0 --> Input1[/Input Num/] Input1 --> Is27{IS Num = 27 ?} Is27 -- NO --> Input1 Is27 -- YES --> Input2[/Input Num/] Input2 --> Is0{IS Num = 0 ?} Is0 -- YES --> Output[/OUTPUT Total/] Output --> END([END]) Is0 -- NO --> Sum[SET Total TO Total + Num] Sum --> Input1 </pre> One mark per point: 1 Initialise <code>Total</code> to zero 2 Check for only first input of 27 in a loop then attempt to sum values 3 Loop until 0 input 4 Sum values input (after input of first 27) in a loop 5 Output <code>Total</code> after a reasonable attempt	5
------	--	----------

38. O/N 23/P23/Q2/a

2(a)

5**MP1** Initialise Min to first value in Data **and** MinIndex to 1**MP2** Loop through 29 more values (or 30 values in total)**MP3** Compare element from Data[] with Min**MP4** Set new Min **AND** save MinIndex when element value < Min **in a loop****MP5** Output MinIndex

39. M/J 23/P21/Q2/a

2(a)	 <pre> graph TD Start([START]) --> InputChar[/INPUT "Input Character", MyChar/] InputChar --> InputCount[/INPUT "Input Count", MyCount/] InputCount --> SetString[Set MyString to ""] SetString --> Decision{Is MyCount < 1?} Decision -- YES --> Output[/OUTPUT MyString/] Output --> End([END]) Decision -- NO --> Concat[Concatenate MyChar with MyString] Concat --> DecCount[Set MyCount to MyCount - 1] DecCount --> Decision </pre> One mark per point: - Both prompts - Both inputs using correct identifiers as given in question, <code>MyChar</code> and <code>MyCount</code> - Initialise <code>MyString</code> to empty string and subsequent output after loop - Loop <code>MyCount</code> iterations including decrement <code>MyCount</code> - Use of Concatenate or equivalent pseudocode statement inside loop Max 4 Marks	4
------	---	---

40. M/J 23/P21/Q3/b

3(b)	One mark for reference to each: 1 Reference to the use of constants or variables for the two threshold values of 10 and 100 // Input amount spent (by customer and store in a numeric variable) 2 Work out one band that amount maps to 3 Work out all bands that amount maps to 4 Calculate rounded value of amount / whole number part of amount 5 Calculate the points by multiplying the (rounded) amount by the appropriate value for appropriate band /all bands 6 Output the number of points Note: Max 5 from available points	5
------	--	----------

41. M/J 23/P21/Q7/a

7(a)	Examples include: Module: <code>GetOverdueLoan()</code> Use: Identifies an overdue book Module: <code>IdentifyStudent()</code> Use: Identifies a student (with an overdue book)¹ Module: <code>GetStudentEmail()</code> Use: Gets the email address of a student with an overdue book Module: <code>CreateEmail()</code> Use: Generates an email to a student with an overdue book Module: <code>SendEmail()</code> Use: Sends an email to a student with an overdue book One mark for name and use Note: Max 3 marks	3
------	--	----------

42. M/J 23/P22/Q1/b

1(b)	One mark per bullet point: • Indentation • White space • Comments • Sensible / meaningful variable names // use of Camel Case • Capitalised keywords Note: Max 3 marks	3
------	---	----------

43. M/J 23/P22/Q2/c

2(c)	One mark per array definition bullet: • A (1D) array containing 7 elements • of type STRING One mark per Step: Step1: Assign value "Sunday" to first element, "Monday" to second element etc. Step2: Use the function <code>DAYINDEX()</code> to return / find the day number from <code>MyDoB</code> Step3: Use the returned value as the array index / to access the element that contains the name / string Step4: Output the element / name / string Note: Max 2 for Array definition, Max 4 for steps	6
------	---	----------

44. M/J 23/P22/Q3/b

3(b)	One mark per bullet: 1 Open file in <code>READ</code> mode 2 Loop to <code>EOF()</code> // read / process all the lines in file 3 Loop will end when return value from <code>AddToQueue()</code> is <code>FALSE</code> / queue is full 4 Read a line from the file in a loop 5 Pass string to <code>AddToQueue()</code> // <code>AddToQueue()</code> is executed with line as parameter	5
------	--	----------

45. M/J 23/P22/Q7/a

7(a)(i)	Correct answers include: • Information: customer name Justification: to personalise / address the email • Information: email address Justification: so that the email can be delivered correctly • Information: product category preference Justification: to check that the customer would be interested in the product • Information: contact preference Justification: to check that the customer will accept contact via email • Information: order history Justification: to send details of product similar to that already bought // to identify frequent shoppers • Information: new product information Justification: to include information about the new product in the email One mark for each piece of information and matching justification Note: Max 3 marks	3
7(a)(ii)	One mark for each piece of information and matching justification: • postal address Justification: sending an email, not a letter • payment details Justification: Nothing being bought / sold at this stage • order history Justification: Customer preference used to decide if new product is relevant Note: Max 2 marks	2

46. M/J 23/P23/Q1/d

1(d)	One mark for each of rows 2 - 5				4																								
<table><tr><th>Pseudocode statement</th><th>Input</th><th>Process</th><th>Output</th></tr><tr><td>INPUT MyChoice</td><td>✓</td><td></td><td></td></tr><tr><td>OUTPUT FirstName & LastName</td><td></td><td>✓</td><td>✓</td></tr><tr><td>WRITEFILE OutputFile, TextLine</td><td></td><td></td><td>✓</td></tr><tr><td>READFILE MyFile, TextLine</td><td>✓</td><td></td><td></td></tr><tr><td>Result ← SQRT (NextNum)</td><td></td><td>✓</td><td></td></tr></table>						Pseudocode statement	Input	Process	Output	INPUT MyChoice	✓			OUTPUT FirstName & LastName		✓	✓	WRITEFILE OutputFile, TextLine			✓	READFILE MyFile, TextLine	✓			Result ← SQRT (NextNum)		✓	
Pseudocode statement	Input	Process	Output																										
INPUT MyChoice	✓																												
OUTPUT FirstName & LastName		✓	✓																										
WRITEFILE OutputFile, TextLine			✓																										
READFILE MyFile, TextLine	✓																												
Result ← SQRT (NextNum)		✓																											

47. M/J 23/P23/Q3/b(ii)

3(b)(ii)	Max 5 marks MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 POP value MP4 Write value to the text file MP5 Repeat from Step 2 // loop referencing the stack items Alternative solution: Not using POP primitive MP1 Open the text file in WRITE mode MP2 Check there is a value on the stack MP3 Read value from ToS location MP4 Write the value to the text file – Must some attempt at ‘the value’ NOT ‘<u>all</u> the values’ MP5 Decrement ToS MP6 Repeat from step 2 // loop referencing the stack items	5
----------	--	----------

48. M/J 23/P23/Q7/a

7(a)(i)	To filter out information (that is not necessary to solve the problem) // to include only essential information	1
7(a)(ii)	Required: Student : Student name / email (address) Loan: Return/Issue date Book: Book title Not Required: Student: Home address / DoB / tutor / subject choices Book: Library location / category / author / book title	2

7(a)(iii)	Max 2 marks Examples could include: • Clear the loan // indicate that the book has been returned // update loan history • Take the student off 'block' // allow the student to borrow further books • Send acknowledgement to the student when book is returned	2
-----------	---	----------

49. O/N 22/P21/Q2/a,b

2(a)	One mark for Explanation: • Abstraction is used to filter out information / data that is not necessary for the task Or the opposite: • To keep only information / data that is necessary for the task One mark for each TWO data items (not dependent on 'Explanation'): Items include: • Car details: ID, Car Registration, car type etc • Customer details: ID, name, address, licence details etc • Start date (of hire) • Return date / Number of days (of hire) • Cost of hire	3
2(b)	One mark for each (Max 2) Examples include: 1 Input customer details 2 Input car details 3 Input payment details 4 Create hire / start hire 5 Return car / end hire 6 Change / check car status (hired / available / written-off) 7 Cancel hire 8 Process payment / calculate hire cost	2

50. O/N 22/P21/Q3

3	One mark per point (Max 5): 1 Declare a variable / an integer <code>Max</code> 2 Assign value of first / any element to <code>Max</code> 3 Set up a loop to repeat 200 times / from start to end of array 4 Use the loop counter as the array index 5 If value of current element is greater than <code>Max</code>... 6 ...then assign value to <code>Max</code> 7 After the loop, Output <code>Max</code>	5
---	--	----------

51. O/N 22/P22/Q2/a

2(a)(i)	One mark per step (or equivalent): 1 Open file in <u>APPEND</u> mode (and subsequent Close) 2 Prompt and Input a student name and mark 3 If mark greater than or equal to 20 jump to step 5 4 Write only the name to the file 5 Repeat from Step 2 for 35 times / the number of students	5
2(a)(ii)	Data in a file is saved after the computer is switched off / stored permanently // no need to re-enter the data when the program is re-run	1
2(a)(iii)	Example answer: So that existing file data is not overwritten.	1

52. O/N 22/P22/Q4/a

4(a)	One mark per point: 1 Input UserID and use of GetAverage() and assignment 2 Initialisation of Total to zero and Index to 4 3 Conditional loop with Index from 4 to 6 4 Assignment of Last from element SameMonth[Index] in a loop 5 Structure: IF...THEN...ELSE...ENDIF in a loop 6 Correct assignments and final call to Update() after the loop <pre> INPUT UserID Average ← GetAverage(UserID) Total ← 0 Index ← 4 WHILE Index < 7 // REPEAT Last ← SameMonth[Index] IF Average > Last THEN Total ← Total + Average ELSE Total ← Total + Last ENDIF Index ← Index + 1 ENDWHILE // UNTIL Index = 7 CALL Update(UserID, Total) </pre> <u>Alternative solution using FOR loop:</u> One mark per point FOR loop solution: 1 Input UserID and use of GetAverage() and assignment 2 Initialisation of Total to zero 3 loop Index from 4 to 6 4 Assignment of Last from array SameMonth in a loop 5 Comparison IF...THEN...ELSE...ENDIF in a loop 6 Appropriate assignments in a loop AND final call to Update() after the loop <pre> INPUT UserID Average ← GetAverage(UserID) Total ← 0 FOR Index ← 4 TO 6 Last ← SameMonth[Index] IF Average > Last THEN Total ← Total + Average ELSE Total ← Total + Last ENDIF NEXT Index CALL Update(UserID, Total) </pre>	6
------	--	----------

53. O/N 22/P22/Q5

5	One mark per IF...THEN...ENDIF clause: 1 IF A AND B AND C THEN CALL Sub1() ENDIF 2 IF (A AND B) AND NOT C THEN CALL Sub2() ENDIF 3 IF (NOT A) AND (NOT C) THEN CALL Sub3() ENDIF 4 IF (NOT A) AND C THEN CALL Sub4() ENDIF	4
---	---	---

54. O/N 22/P23/Q2

2	One mark for name and two marks for use (Max 3 in total): Examples include: Module: SelectCharity() Use: Allows the user to choose a particular charity Module: SpecifyAmountAndType() Use: Allows the user to specify a single or regular payment Module: MakePayment() Use: Make payment to the charity Module: ValidatePayment() Use: Validate payment details (by accessing bank computer) Module: AddBankAccountDetails() / AddPaymentDetails() Use: Allows the user to add bank account information that donation to be taken from Module: AddDonorDetails() Use: Allows user to add details such as name and contact details	3
---	---	---

55. O/N 22/P23/Q3

3	One mark per point, for example: 1 Declare two (REAL) variables for the two sum values AND initialise both to zero 2 Prompt AND Input a number 3 If number greater than zero add to positive sum and If number less than zero add to negative sum 4 Repeat from step 2 if number not zero 5 After loop the Output <code>SumPos</code> and <code>SumNeg</code>	5
---	--	---

56. O/N 22/P23/Q5/c

5(c)	One mark for reason One mark for each example (Max 2) Reason: • Makes the code easier to understand // Describes the purpose of the identifier // Makes the code easier to debug/test/maintain Further examples include: • White space • Indentation • Keywords in capitals • Comments • Local variables // parameters	3
------	--	---

57. O/N 22/P23/Q7/b(ii)

7(b)(ii)	One mark per point (Max 3): 1. Loop through 500 elements (while error number not found) 2. Compare <code>ErrCode</code> for current element with the error number 3. If same, set element value to 999 (and terminate loop) 4. ... and call <code>SortArrays()</code> (to move 999 to the end) – once only	3
----------	---	---

58. O/N 21/P21/Q1/c

1(c)(i)	<table><tr><th>Information</th><th>Essential</th><th>Not essential</th></tr><tr><td>Departure time</td><td>✓</td><td></td></tr><tr><td>Flight Number</td><td></td><td>✓</td></tr><tr><td>Departure airport</td><td>✓</td><td></td></tr><tr><td>Aircraft type</td><td></td><td>✓</td></tr><tr><td>Ticket price</td><td>✓</td><td></td></tr><tr><td>Number of seats in aircraft</td><td></td><td>✓</td></tr></table>	Information	Essential	Not essential	Departure time	✓		Flight Number		✓	Departure airport	✓		Aircraft type		✓	Ticket price	✓		Number of seats in aircraft		✓	3
	Information	Essential	Not essential																				
	Departure time	✓																					
	Flight Number		✓																				
	Departure airport	✓																					
	Aircraft type		✓																				
	Ticket price	✓																					
	Number of seats in aircraft		✓																				
One mark for two rows correct Two mark for four rows correct Three mark for all rows correct																							

1(c)(ii)	One mark for technique and one for benefit, Max 1 mark for 'Benefit' Technique: Abstraction Benefit: • The solution is simplified so easier / quicker to design / implement • The system is tailored to the need of the user	2
----------	---	---

1(c)(iii)	Answers include: • Destination / arrival airport • Arrival time / flight duration • Date of flight • Seat number • Seat availability Max 2 marks	2
-----------	--	---

59. O/N 21/P21/Q2/a,b

2(a)	One mark for reference to:	6
	- The use a variable as an index to the array - A loop to iterate through the array - An Inner loop (with a reducing range) - Test if current element is greater than next element - if so then swap elements - Description of swap - Attempt at efficient algorithm	
	Max 6 marks	

2(b)	<pre> Count ← 1 Flag ← FALSE WHILE Flag = FALSE AND Count <= 5 CALL ReBoot() Count ← Count + 1 Flag ← Check() ENDWHILE IF Flag = FALSE THEN CALL Alert(27) ENDIF </pre> One mark per point: 1 Initialisation of Count AND Flag 2 WHILE ... ENDWHILE // REPEAT ... UNTIL loop 3 ... including both conditions 4 Call ReBoot() AND increment Count inside the loop 5 Assign return value from Check() to Flag inside the loop 6 Final test of Flag AND call to Alert(27) not in a loop	6
------	--	----------

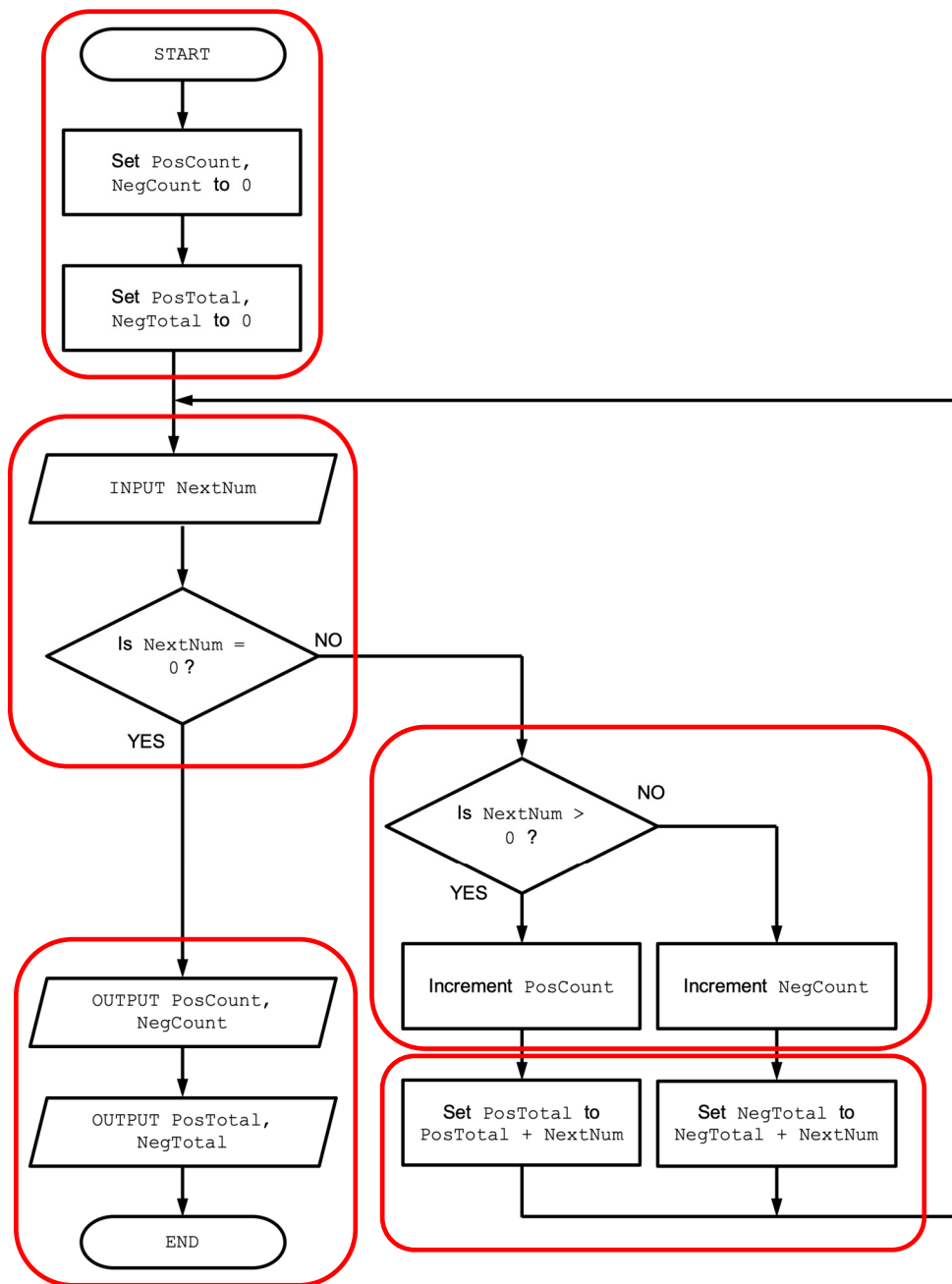
60. O/N 21/P22/Q1/a

1(a)	The process involves: 1 Breaking down a problem / task into sub problems / steps / smaller parts 2 In order to explain / understand // easier to solve the problem 3 Leading to the concept of program modules // assigning problem parts to teams Max 2	2
------	---	----------

61. O/N 21/P22/Q2/a

2(a)

5



One mark for each outlined group.

Note:

- Sum and increment steps (bottom-right rectangles) may be in reverse order in which case sum group will have two output lines
- Max 4 for non-working solution

62. O/N 21/P22/Q4/b

4(b)	One mark per point: • Meaningful variable names • Indentation / white space / blank lines • Capitalisation of keywords	3
------	---	----------

63. O/N 21/P23/Q1/c

1(c)(i)	<table border="1"> <thead> <tr> <th>Information</th><th>Essential</th><th>Not essential</th></tr> </thead> <tbody> <tr> <td>Departure time</td><td>✓</td><td></td></tr> <tr> <td>Flight Number</td><td></td><td>✓</td></tr> <tr> <td>Departure airport</td><td>✓</td><td></td></tr> <tr> <td>Aircraft type</td><td></td><td>✓</td></tr> <tr> <td>Ticket price</td><td>✓</td><td></td></tr> <tr> <td>Number of seats in aircraft</td><td></td><td>✓</td></tr> </tbody> </table> One mark for two rows correct Two mark for four rows correct Three mark for all rows correct	Information	Essential	Not essential	Departure time	✓		Flight Number		✓	Departure airport	✓		Aircraft type		✓	Ticket price	✓		Number of seats in aircraft		✓	3
Information	Essential	Not essential																					
Departure time	✓																						
Flight Number		✓																					
Departure airport	✓																						
Aircraft type		✓																					
Ticket price	✓																						
Number of seats in aircraft		✓																					
1(c)(ii)	One mark for technique and one for benefit, Max 1 mark for 'Benefit' Technique: Abstraction Benefit: • The solution is simplified so easier / quicker to design / implement • The system is tailored to the need of the user	2																					
1(c)(iii)	Answers include: • Destination / arrival airport • Arrival time / flight duration • Date of flight • Seat number • Seat availability Max 2 marks	2																					

64. O/N 21/P23/Q2/a,b

2(a)	One mark for reference to: 1 The use a variable as an index to the array 2 A loop to iterate through the array 3 An Inner loop (with a reducing range) 4 Test if current element is greater than next element 5 if so then swap elements 6 Description of swap 7 Attempt at efficient algorithm Max 6 marks	6
2(b)	<pre>Count ← 1 Flag ← FALSE WHILE Flag = FALSE AND Count <= 5 CALL ReBoot() Count ← Count + 1 Flag ← Check() ENDWHILE IF Flag = FALSE THEN CALL Alert(27) ENDIF</pre> One mark per point: 1 Initialisation of Count AND Flag 2 WHILE ... ENDWHILE // REPEAT ... UNTIL loop 3 ... including both conditions 4 Call ReBoot() AND increment Count inside the loop 5 Assign return value from Check() to Flag inside the loop 6 Final test of Flag AND call to Alert(27) not in a loop	6

65. M/J 21/P21/Q1/a

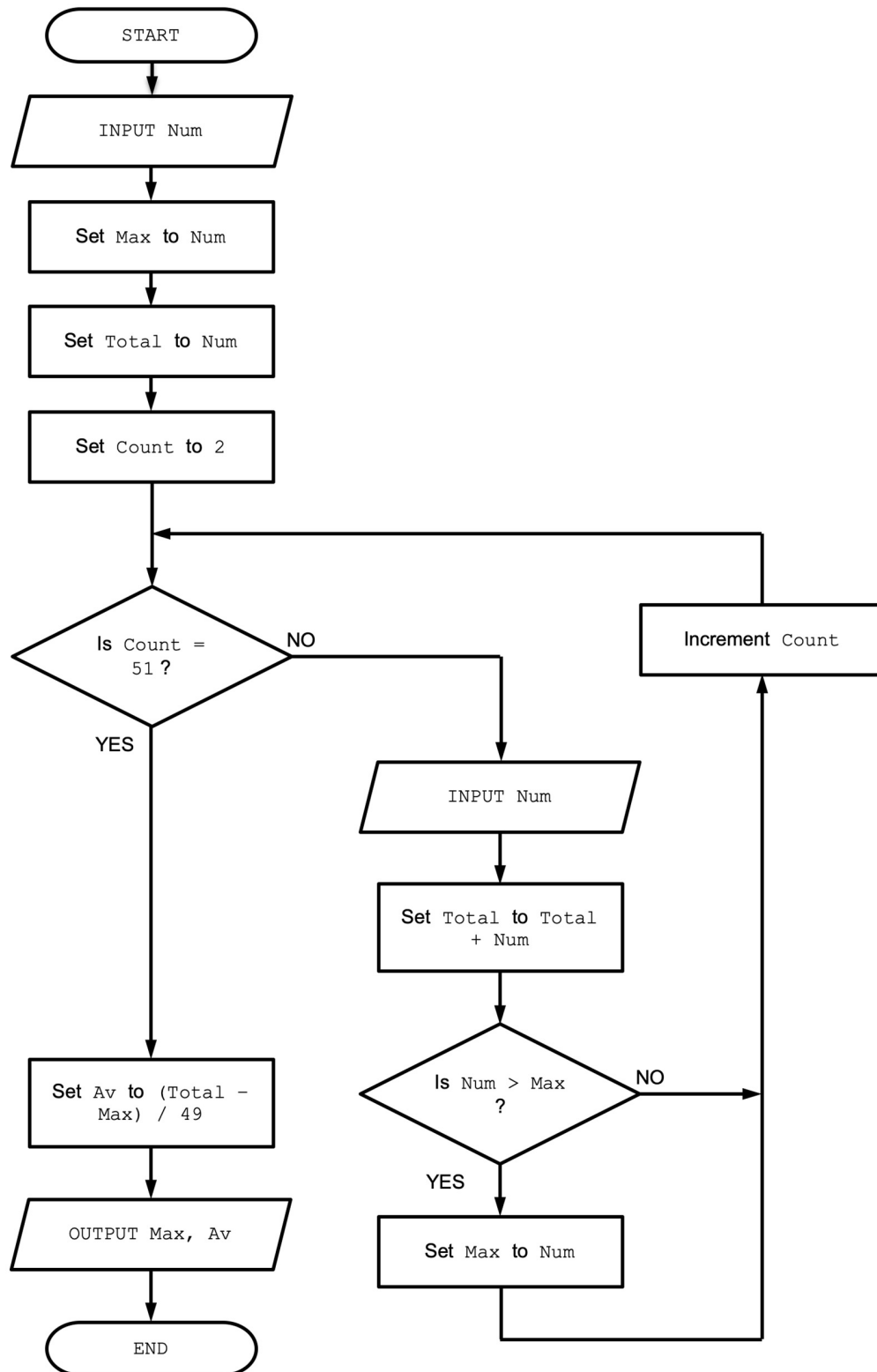
1(a)	Example value	Explanation	Variable name	Data type	4
	"Wong"	The preferred name of the member joining the football club	MemberName	STRING	
	FALSE	A value to indicate whether an existing member of the club lives at the same address	FamilyMember	BOOLEAN	
	19/02/1983	When the member joined the football club	StartDate	DATE	
	1345	The number of points a member has earned. Members of the club earn points for different activities.	Points	INTEGER	
One mark for each appropriate variable name plus data type					

66. M/J 21/P21/Q2/c

6

2(c)

Example solution:



One mark for each functional group as listed below:

2(c)	Explanation of mark points: 1 Initialise MAX to first value input 2 Set Total to zero 3 Input 49 more values (or 50 values in total) 4 Sum all values input 5 Set new MAX when Input value > MAX in a loop 6 Sum all but largest (or subtract MAX from total), calculate and output average	
------	---	--

67. M/J 21/P21/Q7/b

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return FALSE / set flag 4 If word not already in array, loop to find unused element (<i>second loop</i>) 5 Store word in unused element and return TRUE, otherwise return FALSE VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return FALSE / set flag 5 If word not already in array and unused element available, store word in unused element and return TRUE otherwise return FALSE Note: Max 4 marks	4
------	--	----------

68. M/J 21/P22/Q1/b

1(b)					3
	Statement	Selection	Assignment	Iteration	
	Index ← Index + 1		✓		
	IF Modified = TRUE THEN	✓			
	ENDWHILE			✓	
One mark per row					

1(a)(i)	Variable	Example data value	Data type	4
	Name	"Catherine"	STRING	
	Index	100	INTEGER	
	Modified	FALSE	BOOLEAN	
	Holiday	25/12/2020	DATE	
One mark per data type				

69. M/J 21/P22/Q2/b

2(b)(i)	Answers include: • User ID / Username • Book ID • Date of loan / return date One mark for 1 correct Two marks for all 3 correct Note: Max 2 marks	2
2(b)(ii)	Many examples but must be data that is NOT required for a loan, but which COULD be required somewhere by the library system. Note: must be data relating to users, books or loans Answers include: • Users name / address / phone number / DOB • Book title / author / publisher / library rack number / ISBN number / price • Date of loan / return date (if not already given in part (i)) • The length of the loan (assumed to be the same for all books)	1
2(b)(iii)	Many examples including: • Create loan / borrow book • Return book • Send letter / email / contact a user ref an overdue book • View the loan history for a given book • View the loan history for a given user One mark for each Note: Max 2 marks	2

70. M/J 21/P22/Q4

4	Marks awarded for a description of each of the following steps of the algorithm: 1 Reference variables for <code>Count</code> of students and <code>Total</code> marks 2 Loop through all students (<code>Count</code>) 3 Input individual mark (in loop) 4 Compare mark with threshold / boundary values to determine grade (in loop) 5 Output the grade for a student (in loop) 6 Maintain a <code>Total</code> (and <code>Count</code> if required) (in loop) 7 Calculate average by dividing <code>Total</code> by <code>Count</code> and Output (after loop) Note: Max 6 marks	6
----------	--	----------

71. M/J 21/P23/Q1/a

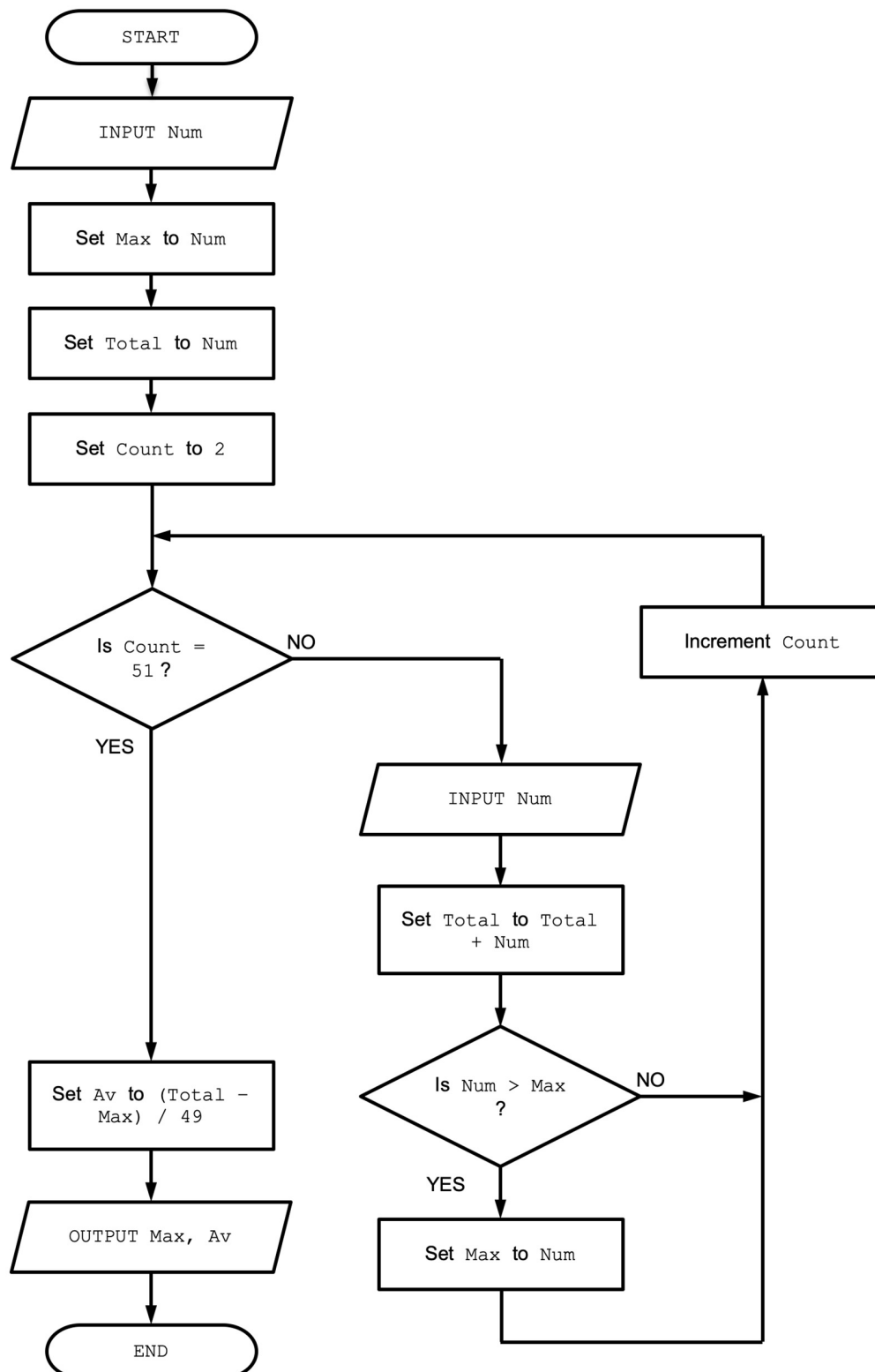
1(a)	<table><tr><th>Example value</th><th>Explanation</th><th>Variable name</th><th>Data type</th></tr><tr><td>"Wong"</td><td>The preferred name of the member joining the football club</td><td>MemberName</td><td>STRING</td></tr><tr><td>FALSE</td><td>A value to indicate whether an existing member of the club lives at the same address</td><td>FamilyMember</td><td>BOOLEAN</td></tr><tr><td>19/02/1983</td><td>When the member joined the football club</td><td>StartDate</td><td>DATE</td></tr><tr><td>1345</td><td>The number of points a member has earned. Members of the club earn points for different activities.</td><td>Points</td><td>INTEGER</td></tr></table>	Example value	Explanation	Variable name	Data type	"Wong"	The preferred name of the member joining the football club	MemberName	STRING	FALSE	A value to indicate whether an existing member of the club lives at the same address	FamilyMember	BOOLEAN	19/02/1983	When the member joined the football club	StartDate	DATE	1345	The number of points a member has earned. Members of the club earn points for different activities.	Points	INTEGER	4
Example value	Explanation	Variable name	Data type																			
"Wong"	The preferred name of the member joining the football club	MemberName	STRING																			
FALSE	A value to indicate whether an existing member of the club lives at the same address	FamilyMember	BOOLEAN																			
19/02/1983	When the member joined the football club	StartDate	DATE																			
1345	The number of points a member has earned. Members of the club earn points for different activities.	Points	INTEGER																			
One mark for each appropriate variable name plus data type																						

72. M/J 21/P23/Q2/c

2(c)

Example solution:

6



One mark for each functional group as listed below:

2(c)	Explanation of mark points: 1 Initialise MAX to first value input 2 Set Total to zero 3 Input 49 more values (or 50 values in total) 4 Sum all values input 5 Set new MAX when Input value > MAX in a loop 6 Sum all but largest (or subtract MAX from total), calculate and output average	
------	---	--

73. M/J 21/P23/Q7/b

7(b)	Marks awarded for any reference to each of the following steps of the algorithm: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 If word is the same as the current array element then return FALSE / set flag 4 If word not already in array, loop to find unused element (<i>second loop</i>) 5 Store word in unused element and return TRUE, otherwise return FALSE VARIATION: 1 Mention of variable for use as array index 2 Use of a loop (to check through the array) 3 Save index of (first) unused element found 4 If word is the same as the current array element then return FALSE / set flag 5 If word not already in array and unused element available, store word in unused element and return TRUE otherwise return FALSE Note: Max 4 marks	4
------	--	----------

74. 9608 M/J 21/P21/Q2/a,b

2

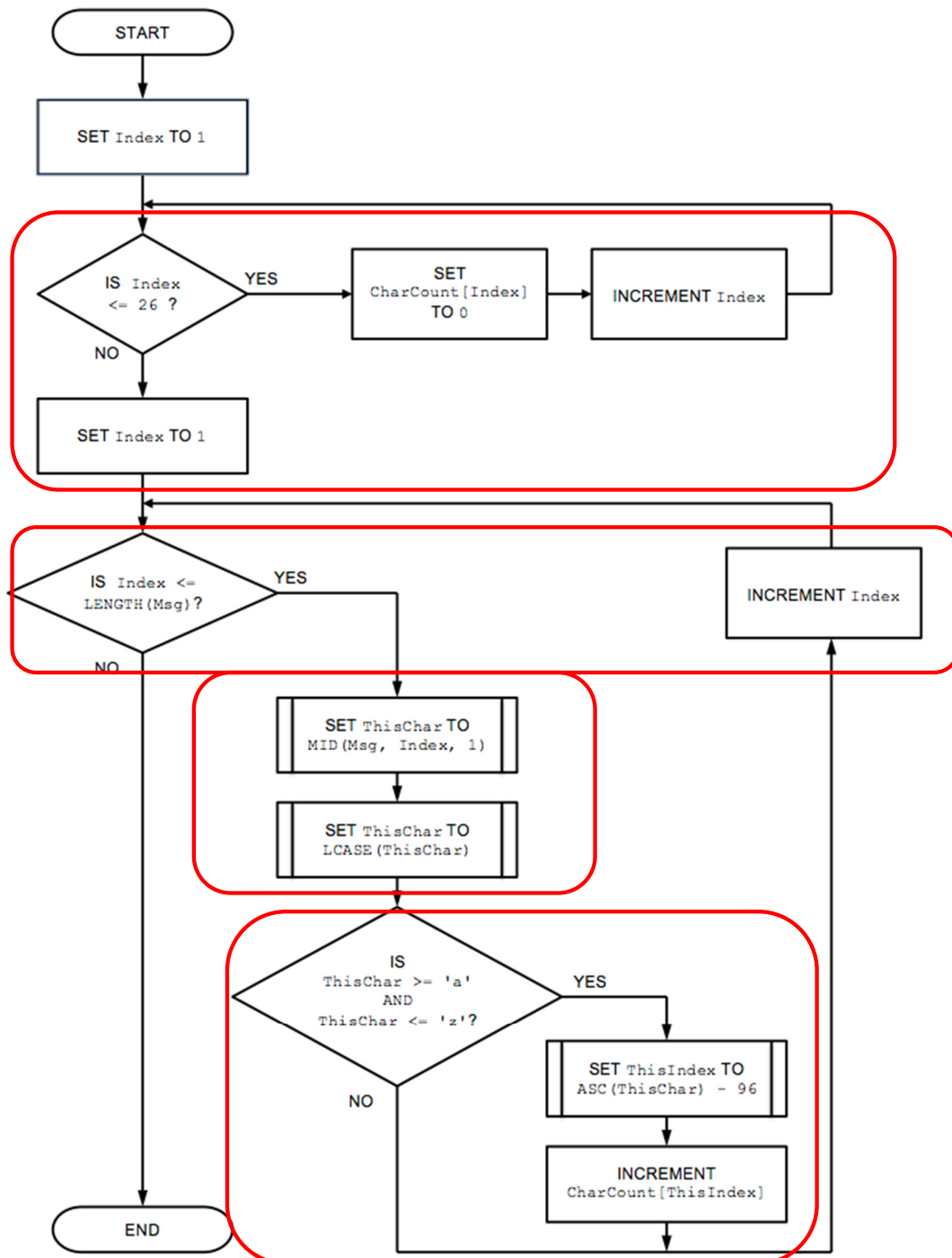
Mark as follows:

First mark for START, Initialisation of Index **and** END

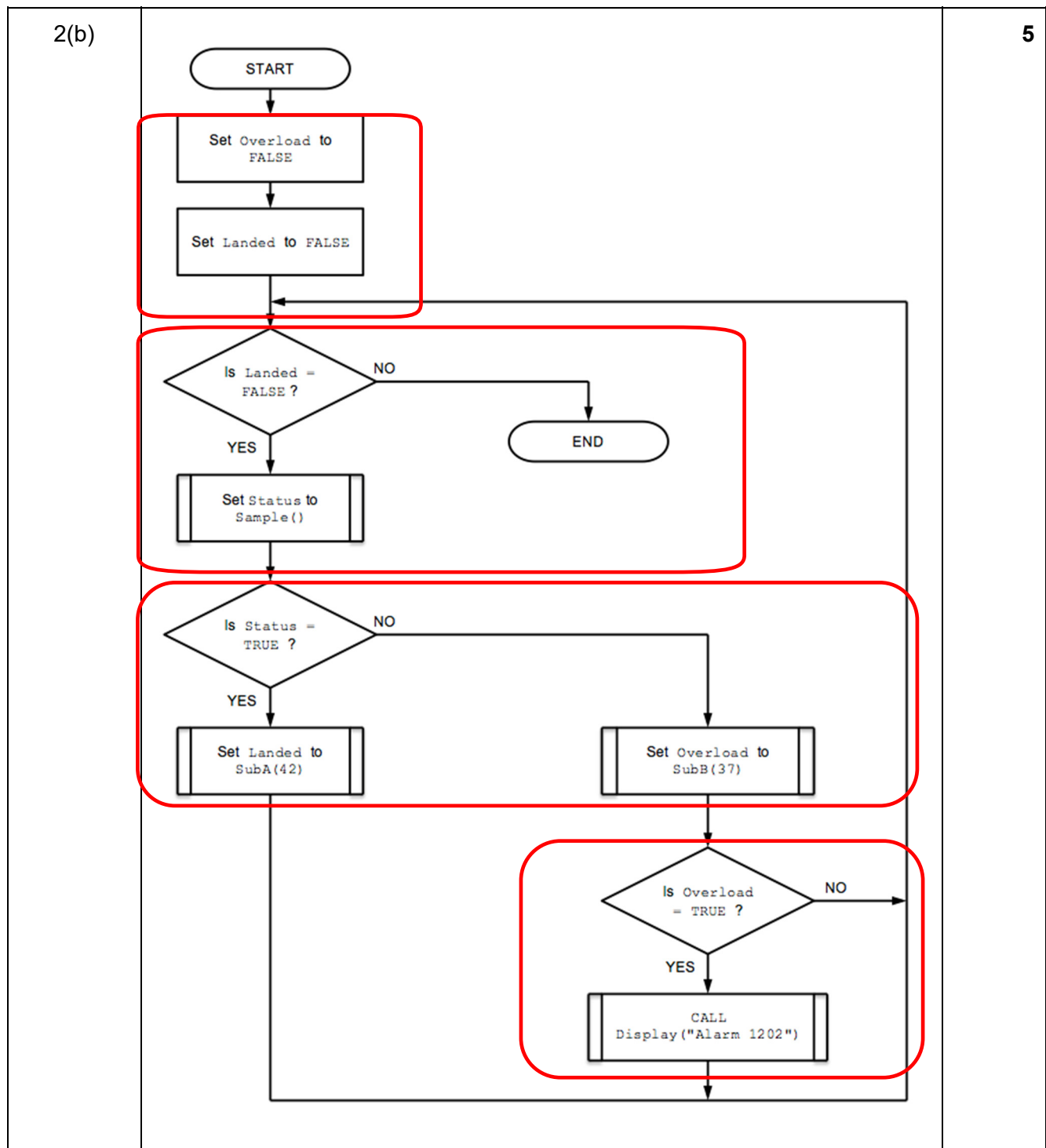
Then one mark per area outlined, in correct place.

At least one decision box label (YES / NO) must be present.

5



2(b)	One mark per point. Algorithm should mention: 1 Initialise variable to hold Max value 2 Loop through 26 elements of array 3 Test if element > Maxand if so set new Max value 4 Method of checking for duplicates 5 Output a message giving alphabetic char with largest count value - needs use of CHR () 6 Output a suitable message if targets count value is shared	6
------	--	---

75. 9608 M/J 21/P22/Q2/b

76. 9608 M/J 21/P23/Q1/a

1(a)	One mark per bullet point. • INPUT • Example input statement in language stated • PROCESS • Example process statement in language stated • Example OUTPUT statement in language stated	5
------	---	----------

77. 9608 M/J 21/P23/Q2/b

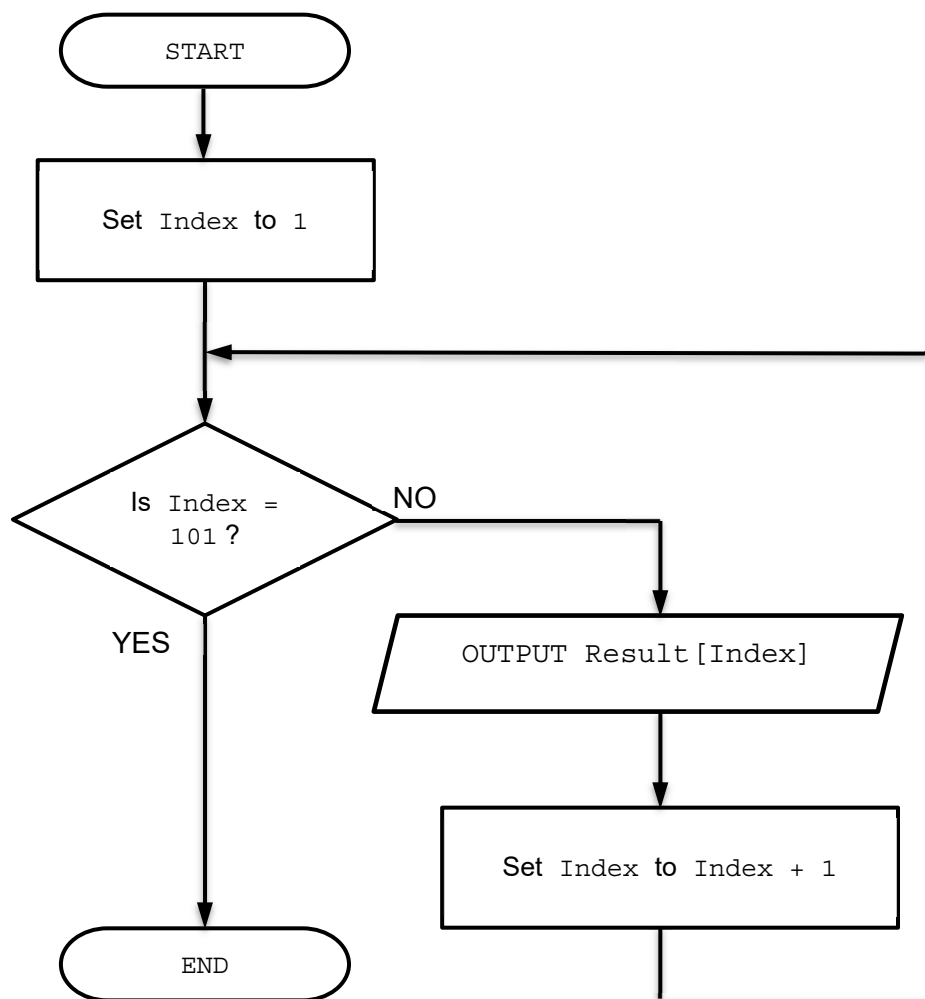
2(b)(i)	Makes it easier to understand the purpose of each identifier / what the identifier is used for / the purpose of the program	1
2(b)(ii)	One mark per point: 1 The use of modular programming (to avoid repeated code) 2 The use of library / tried and tested subroutines 3 Good formatting to make the code easier to read (indentation, white space) 4 Use of <u>local</u> variables 5 Use of constants 6 Use of comments to explain functionality of code Max 3 marks	3

78. 9608 M/J 21/P23/Q3/a,b,c

3(a)	• To break the problem down into sub-tasks • where each sub-task can be implemented by a program module / is easier to solve. One mark for each phrase (or equivalent)	2
------	--	----------

3(b)

4



1 mark for each of the following:

- 1 Initialise Index
- 2 Test index for 100 elements
- 3 End when 100 elements output
- 4 Output, increment and repeat

3(c)	<pre> Online ← FALSE WHILE Online = FALSE IF Active = TRUE THEN Call Sync() ELSE Call Reset() IF Active = FALSE THEN Call Error("No Signal") ELSE Online ← TRUE ENDIF ENDIF Call ReCheck() ENDWHILE </pre> 1 mark for each of the following: 1 Initialise Online 2 WHILE .. ENDWHILE loop, terminated when Online = TRUE 3 IF Active = TRUE THEN .. ELSE .. ENDIF 4 Nested IF Active = FALSE THEN .. ELSE .. ENDIF 5 Call Sync() and Call Reset() and Call Error() and assignment to Online in appropriate place in pseudocode 6 Final call to ReCheck() in appropriate place	6
------	--	---

79. M/J 20/P21/Q1/a,c

1(a)	An algorithm is a solution to a problem expressed as: a <u>sequence</u> of defined <u>steps</u> / <u>stages</u> / <u>instructions</u> / <u>lines of code</u> 1 mark for each underlined term (or equivalent)	2
------	---	---

1(c)	<table><tr><th>Term</th><th>Description</th></tr><tr><td>Selection</td><td>Checking that a program performs as expected</td></tr><tr><td>Black-box testing</td><td>A method for increasing the level of detail of an algorithm</td></tr><tr><td>Stepwise refinement</td><td>To test a condition to determine the path of program execution</td></tr><tr><td>Iteration</td><td>A method of executing certain lines of code more than</td></tr></table>	Term	Description	Selection	Checking that a program performs as expected	Black-box testing	A method for increasing the level of detail of an algorithm	Stepwise refinement	To test a condition to determine the path of program execution	Iteration	A method of executing certain lines of code more than	3
Term	Description											
Selection	Checking that a program performs as expected											
Black-box testing	A method for increasing the level of detail of an algorithm											
Stepwise refinement	To test a condition to determine the path of program execution											
Iteration	A method of executing certain lines of code more than											

One mark for each correct line to max 3

80. M/J 20/P21/Q3/a

3(a)	<pre> PROCEDURE AddCredit (TopUp : REAL, PhoneNum : STRING) DECLARE Multiple : REAL DECLARE Balance : REAL Multiple ← 1 Balance ← GetBalance (PhoneNum) IF Balance > 10 THEN Multiple ← 1.125 ELSE IF Balance > 5 THEN Multiple ← 1.1 ENDIF ENDIF TopUp ← TopUp * Multiple SetBalance (PhoneNum, Balance + TopUp) ENDPROCEDURE </pre> 1 mark for each of the following: 1 PROCEDURE heading and ending including parameters 2 Initialise Multiple 3 Assign value to Balance using GetBalance() 4 Check for Balance > 10 and assignment: Multiple ← 1.25 5 Check for Balance > 5 and assignment: Multiple ← 1.1 6 Assignment: TopUp ← TopUp * Multiple 7 Calling SetBalance() with correct parameters Note: MP6 could be included in MP7 statement	7
------	--	----------

81. M/J 20/P22/Q1/a

1(a)	One mark for name Max two marks for description: one for each underlined word or phrase (or equivalent) Name: Sequence Description: <u>Instructions / lines of code are executed</u> in a <u>fixed order</u> OR Name: Assignment Description: A <u>value</u> is given to a <u>variable</u>	3
------	--	----------

82. M/J 20/P22/Q2/b(i)

2(b)(i)	Advantages include: • Easier to solve / implement / program the solution as online shopping is a complex task • Easier to debug / maintain as each module can be tested separately e.g. test <code>FillBasket()</code> first then test <code>Checkout()</code> • Tasks may be shared among a team of programmer. e.g. <code>Checkout()</code> and <code>Search()</code> modules could be developed in parallel / by teams with different expertise Note: Must include reference to given scenario to achieve all 3 marks - Max 2 if no reference.	3
---------	---	----------

83. M/J 20/P22/Q5/a

5(a)	Mark as follows: 1 SET Name to "" 2 SET Index to 1 3 SELECT the character from input parameter string at Index position 4 IF character is not colon then concatenate character with Name 5 ...INCREMENT Index 6 ...REPEAT from step 3 7 RETURN Name Alternative Solution: Mark as follows: 1 SET Index to 1 2 SELECT the character from input parameter string at Index position 3 IF character is colon then go to 5 4 Else INCREMENT Index and repeat from 2 5 Extract a substring from the left of the parameter string (and assign this to variable Name) 6 ...Using Index - 1 for the length 7 RETURN Name Note: Mark points may be combined for equivalent marks e.g a suitable <u>structured English description</u> of the pseudocode statement below satisfies MP 5, 6 and 7: RETURN LEFT(ParamString, Index - 1)	7
------	---	----------

84. M/J 20/P23/Q1/a,b,c

1(a)	When: when designing a <u>solution</u> to a problem Purpose: to describe the solution as a <u>sequence of steps / actions</u> One mark per answer Accept equivalent phrase for 'purpose' but reject specific programming references	2
1(b)	Testing <u>a condition</u> <u>to determine the sequence</u> of execution One mark per underlined phrase (or equivalent)	2
1(c)	Explanation: Breaking a problem down into sub tasks Reason: Make the problem easier to solve // to make the solution easier to implement / test / maintain	2

85. M/J 20/P23/Q2/c

2(c)	<pre> DECLARE Index, Count : INTEGER Count ← 0 FOR Index ← 1 TO 50 IF Item[Index, 1] = SearchString THEN Count ← Count + 1 ENDIF IF Item[Index, 2] = SearchString THEN Count ← Count + 1 ENDIF ENDFOR OUTPUT "The number of times SearchString found: ", Count Alternative DECLARE I, J, Count : INTEGER Count ← 0 FOR I ← 1 TO 50 FOR J ← 1 TO 2 IF Item[I, J] = SearchString THEN Count ← Count + 1 ENDIF ENDFOR ENDFOR OUTPUT "The number of times SearchString found: ", Count One mark for each of the following: 1 Initialisation of Count 2 FOR loop 3 Check column 1 element and increment count 4 Check column 2 element and increment count // nested loops 5 OUTPUT Count together with suitable mesage </pre>	5
------	---	----------

86. M/J 20/P23/Q3/a

3(a)	<pre> graph TD START([START]) --> SET_ERROR[SET Error TO value of Delta(Plan, Actual)] SET_ERROR --> CASE_ERROR{CASE OF Error} CASE_ERROR -- OTHERWISE --> OUTPUT[OUTPUT "Unexpected Error"] CASE_ERROR -- "> 10" --> SET_STEER_MINUS[SET Steer TO Steer - 10] CASE_ERROR -- "0" --> SET_ZCOUNT[SET ZCount TO ZCount + 1] CASE_ERROR -- "< -10" --> SET_STEER_PLUS[SET Steer TO Steer + 10] SET_STEER_MINUS --> MERGE(()) SET_ZCOUNT --> MERGE SET_STEER_PLUS --> MERGE MERGE --> END([END]) OUTPUT --> END </pre> Mark as follows: • One mark for START and END • One mark per area outlined All outputs from CASE must be labelled	5
------	---	---

87. O/N 19/P21/Q1/a

1(a)(i)	One mark for each feature: 1. meaningful / sensible identifier names // use of Camel case for identifier names // use of constants 2. blank lines / white space 3. comments	3
---------	--	---

1(a)(ii)	Mark as follows: • One mark for START and END • One mark per area outlined At least one decision box label (YES/NO) must be present <pre> graph TD START([START]) --> V1[V ← GetLevel()] V1 --> L[L ← V * 1.34] L --> C[C ← 0] C --> N1[N ← 1] N1 --> D1{Is N > 10?} D1 -- NO --> N2[N ← N + 1] D1 -- YES --> O1[/OUTPUT "Process complete"/] O1 --> R1[RETURN C] R1 --> END([END]) N2 --> V2[V ← GetLevel()] V2 --> D2{Is V > L?} D2 -- YES --> C2[C ← C + 1] D2 -- NO --> N2 C2 --> N2 </pre>	5
----------	--	---

88. O/N 19/P21/Q4/a

4(a)	One mark for process name, max 3 for structured English. Process: • Stepwise Refinement / Top-down design Structured English: • Check that character is between 'A' and 'Z' • Produce unique array index for this character • Increment this array element	4
------	--	---

89. O/N 19/P22/Q1/a

1(a)(i)	Any three from: 1. Indentation 2. Blank lines / white space 3. Sensible identifier names / use of Camel Case for identifier names 4. Capitalised keywords	3
1(a)(ii)	Mark as follows: • One mark for START and END • One mark for each of the 4 areas outlined Loop must be included for this area mark <pre> graph TD START([START]) --> Tries1[Tries ← 1] Tries1 --> ReadSensor1[Full ← ReadSensor("F1")] ReadSensor1 --> IsFull{Is Full = TRUE?} IsFull -- YES --> Output1[/Output "Already Full"/] IsFull -- NO --> LoopStart(()) subgraph LoopArea [] LoopStart --> IsNotFull{Is NOT Full AND Tries < 4} IsNotFull -- YES --> TopUp[Call TopUp()] TopUp --> ReadSensor2[ReadSensor("F1")] ReadSensor2 --> TriesInc[Tries ← Tries + 1] TriesInc --> IsNotFull end IsNotFull -- NO --> IsTries3{Is Tries > 3?} IsTries3 -- YES --> Output2[/Output "Too many attempts"/] IsTries3 -- NO --> Output3[/Output "Tank now full"/] Output1 --> End([END]) Output2 --> End Output3 --> End </pre>	5

90. O/N 19/P23/Q1/b,c,d

1(b)	Two from (max 2): • A list of identifier / variable names • Explanations/descriptions (of what they are used for) • Data types	2								
1(c)(i)	One mark for each stage (Input, Output) One mark for each correct example <table><tr><td>Stage</td><td>Example statement</td></tr><tr><td>Input</td><td><u>Next = Console.ReadLine()</u></td></tr><tr><td>Process</td><td><u>x = INT(y/3)</u></td></tr><tr><td>Output</td><td><u>Console.WriteLine("Goodbye")</u></td></tr></table>	Stage	Example statement	Input	<u>Next = Console.ReadLine()</u>	Process	<u>x = INT(y/3)</u>	Output	<u>Console.WriteLine("Goodbye")</u>	5
Stage	Example statement									
Input	<u>Next = Console.ReadLine()</u>									
Process	<u>x = INT(y/3)</u>									
Output	<u>Console.WriteLine("Goodbye")</u>									
1(c)(ii)	One mark for statement in program code that includes (at least) two 'stages' Example correct answers: • <code>Next = LEN(Console.Input())</code> • <code>Console.WriteLine(Name & Address)</code> • <code>Console.WriteLine(Console.ReadLine() & " is what you entered")</code>	1								
1(d)	Three from the following (max 3): • Blank lines • Capitalisation of Keywords • Sensible variable names • Use of (library/built-in) functions • Comments • PrettyPrint / keywords coloured	3								

91. O/N 19/P23/Q2/a(ii)

2(a)(ii)	<pre> REPEAT CALL AlarmReset() Status1 ← GetStatus(Sys_A) Status2 ← GetStatus(Sys_B) UNTIL (Status1 = TRUE AND Status2 = TRUE) </pre> One mark for each of: 1 REPEAT ... UNTIL 2 Call to AlarmReset() 3 Assignment of Status1 and Status2 4 correct logical test	4
----------	---	----------

92. M/J 19/P21/Q1/a

1(a)(i)	Construct: Assignment Pseudocode example: Answer ← "YES" Construct: Selection Pseudocode example: IF X = 3 THEN OUTPUT "HELLO" Construct: Repetition / Iteration Pseudocode example: FOR N ← 1 to 100 One mark for construct One mark for pseudocode example Maximum 4 marks	4																				
1(a)(ii)	<table><tr><th>Pseudocode statement</th><th>Input</th><th>Process</th><th>Output</th></tr><tr><td>Temp ← SensorValue * Factor</td><td></td><td>✓</td><td></td></tr><tr><td>WRITEFILE "LogFile.txt", TextLine</td><td></td><td></td><td>✓</td></tr><tr><td>WRITEFILE "LogFile.txt", MyName & MyIDNumber</td><td></td><td>✓</td><td>✓</td></tr><tr><td>READFILE "AddressBook.txt", NextLine</td><td>✓</td><td>(✓)</td><td></td></tr></table> One mark per correct row	Pseudocode statement	Input	Process	Output	Temp ← SensorValue * Factor		✓		WRITEFILE "LogFile.txt", TextLine			✓	WRITEFILE "LogFile.txt", MyName & MyIDNumber		✓	✓	READFILE "AddressBook.txt", NextLine	✓	(✓)		4
Pseudocode statement	Input	Process	Output																			
Temp ← SensorValue * Factor		✓																				
WRITEFILE "LogFile.txt", TextLine			✓																			
WRITEFILE "LogFile.txt", MyName & MyIDNumber		✓	✓																			
READFILE "AddressBook.txt", NextLine	✓	(✓)																				

93. M/J 19/P21/Q3/a

3(a)	<pre> TotalValue ← 0 ZeroCount ← 0 FOR Index ← 1 TO 100 TotalValue ← TotalValue + Result[Index] IF Result[Index] = 0.0 THEN ZeroCount ← ZeroCount + 1 ENDIF ENDFOR OUTPUT "The average is ", (TotalValue / 100) OUTPUT "The number of elements with a zero value is ", ZeroCount One mark for each of the following: 1 Both initialisations 2 Loop 100 times 3 Adding individual element to TotalValue in a loop 4 Check if element value is zero in a loop 5 If so increment ZeroCount in a loop 6 Average is calculated after the loop 7 Both OUTPUT statements, including message and variables </pre>	7
------	--	----------

94. M/J 19/P22/Q1/a

1(a)(i)	- a sequence of steps / stages / instructions - to implement a task // solution to a problem Allow alternatives to sequence providing meaning is clear.	2
1(a)(ii)	Input: - e.g. INPUT MyVar // READFILE MyFile, MyString Process: - e.g. NextChar ← 'X' // Count ← Count + 1 Output:: - e.g. OUTPUT "Hello World" // WRITEFILE "LogFile.txt", SomeData Mark as follows: One mark for each stage (Input and Process) One mark for each pseudocode example	5

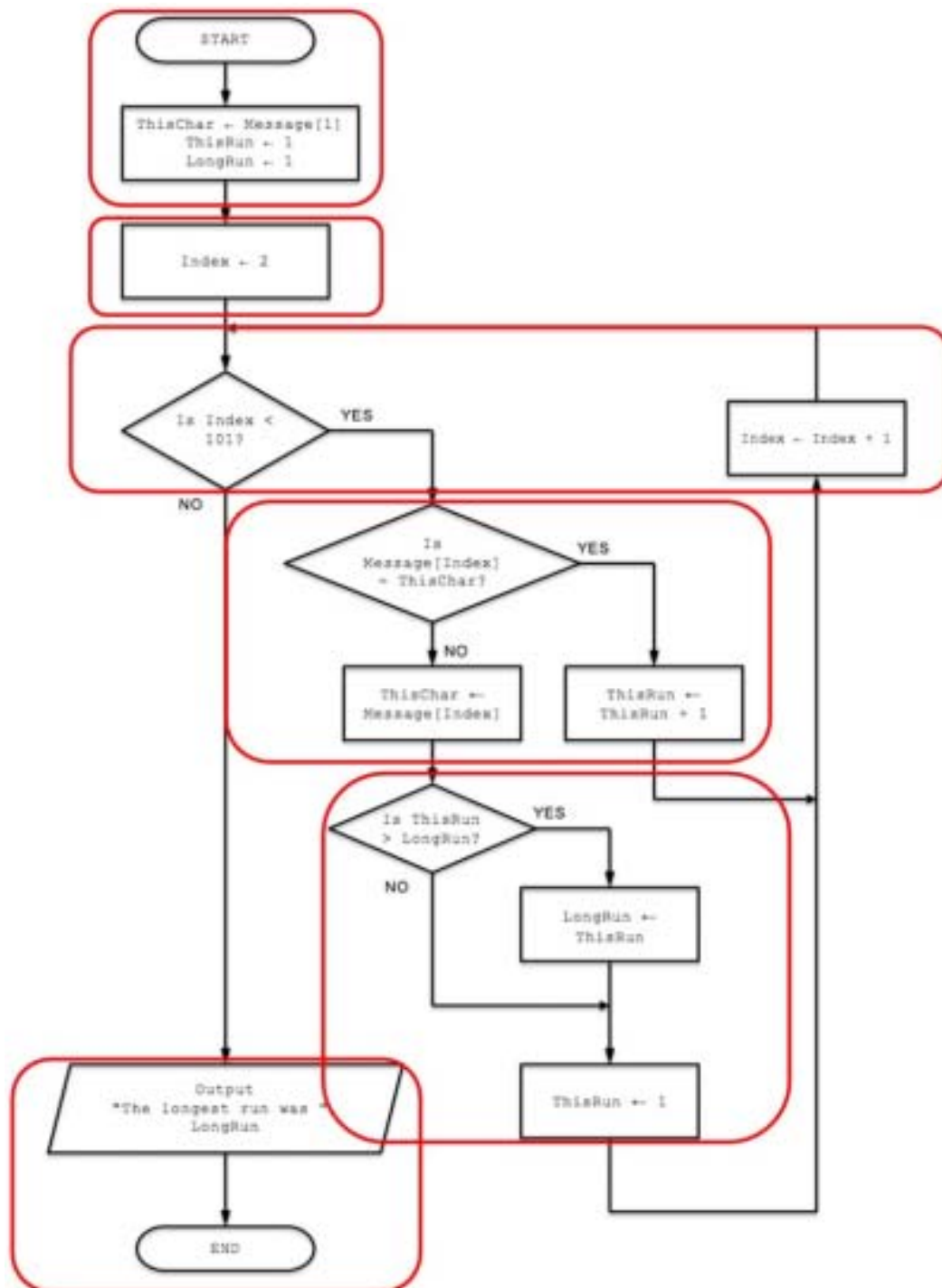
95. M/J 19/P22/Q5/a

5(a)	One mark for each of: • Open the file • Set a count to zero • Loop until end of file // no more lines to read • Increment the count each time a line is read in a loop Maximum 3 marks	3
------	--	----------

96. M/J 19/P23/Q1/a

1(a)

6



Mark as follows:

- One mark per area outlined, in correct place
- Decision must be diamond symbol and have two outputs with at least one label (YES / NO)

97. M/J 19/P23/Q2/a,b

2(a)	One mark for each point: • Initialise a count to zero • loop 100 times // loop through all of the array • compare an element with "Empty" in a loop • increment the count if equal in a loop • Output a message together with the count not inside a loop	5
2(b)	One mark for each point: • The breaking down of an algorithm / task / problem • to a level of (sufficient) detail // into smaller parts / sub-tasks • from which it can be programmed // which are easier to program	3

98. O/N 18/P22/Q1/a

1(a)(i)

Statement	Selection	Repetition (Iteration)	Assignment
Index ← Index + 5			✓
FOR Count ← 1 TO 100		✓	(✓)
TempValue[Index] ← ReadValue (SensorID)			✓
IF Index < 30	✓		
UNTIL DayNumber > 7		✓	
OTHERWISE OUTPUT "ERROR"	✓		

1 mark per correct row

Ignore any tick in assignment column for second statement

6

99. O/N 18/P22/Q2/a(i)

2(a)(i)	<pre> FUNCTION CalcPoints(CardNum: STRING, Total: REAL) RETURNS INTEGER DECLARE OldPoints : INTEGER DECLARE NewPoints : INTEGER IF Total > 100 THEN OldPoints ← GetPoints(CardNum) IF OldPoints > 2000 THEN NewPoints ← INT(Total * 1.2) ELSE NewPoints ← INT(Total * 1.1) ENDIF ELSE NewPoints ← 0 ENDIF RETURN NewPoints ENDFUNCTION </pre> 1 mark for each of the following: 1 Correct FUNCTION heading (as given) and end 2 Declaring local variables for OldPoints and NewPoints 3 IF...THEN...ELSE...ENDIF with Total > 100 4 ...Newpoints set to zero if Total <= 100 5 Nested IF...THEN...ELSE...ENDIF with OldPoints > 2000 6 ... with correct assignments of NewPoints 7 Return NewPoints for all cases	7
---------	--	----------

100. O/N 18/P22/Q3/b,c

3(b)	Key points: • to increase the level of detail of the algorithm // break the problem into smaller steps • until steps are easier to solve // to be directly translated into lines of code	2
3(c)	Key points: • A <u>loop / repetition / iteration</u> to check every element • Compare the array element with the value being searched • Exit loop / stop search when value found or end of array reached • If value is found then output the index position, otherwise output "Not found"	4

101. O/N 18/P23/Q1/a

1(a)(i)	Statement	Assignment	Selection	Iteration	6
	CASE OF TempSensor1		✓		
	ELSE		✓		
	REPEAT			✓	
	ENDFOR			✓	
	DayNumber ← DayNumber + 1	✓			
	Error ← TRUE	✓			
One mark per row					

102. O/N 18/P23/Q2/a(i)

2(a)(i)	FUNCTION CalcBonus(CardNum: STRING) RETURNS INTEGER DECLARE Points : INTEGER DECLARE Bonus : INTEGER DECLARE Spend : REAL Points ← GetPoints(CardNum) Spend ← GetSpend(CardNum) IF Points > 2000 THEN Bonus ← 100 ELSE IF Spend > 1000 THEN Bonus ← 50 ELSE Bonus ← 10 ENDIF ENDIF RETURN Bonus ENDFUNCTION 1 mark for each of the following up to max 5 marks: 1 Function heading (inc parameters) and ending 2 Declaring local variables and two function calls as above 3 IF...THEN...ELSE...ENDIF with Points > 2000 4 (Nested) IF...THEN...ELSE with Spend > 1000 5 ... assignment of Bonus to 10, 50 100 6 Return parameter	5
---------	--	----------

103. O/N 18/P23/Q3/b

3(b)	Steps: • Declare (and initialise values to first array element) for min and max as integers • A loop / iteration / repetition to check every element • Compare each array element with max variable and min variable • Update max variable if bigger and min variable if smaller 1 mark per bullet point Alternative steps: • Apply a sort routine to the values in the array • Swapping consecutive elements (as necessary) // until no more swaps • Min will be the first / last element and max will be the last / first element 1 mark per bullet point Max 3 marks	3
------	---	----------

104. M/J 18/P21/Q1/a

1(a)

Description of Data Item	Suitable Identifier Name
The temperature of the patient	PatientTemperature
The temperature of the room	RoomTemperature
The patient identification number	PatientID
The name of the nurse taking the measurement	NurseName

The above are examples only.
Names must be meaningful and unambiguous

Items 1 and 2 must have suitable prefix / suffix (i.e. not just 'temp')

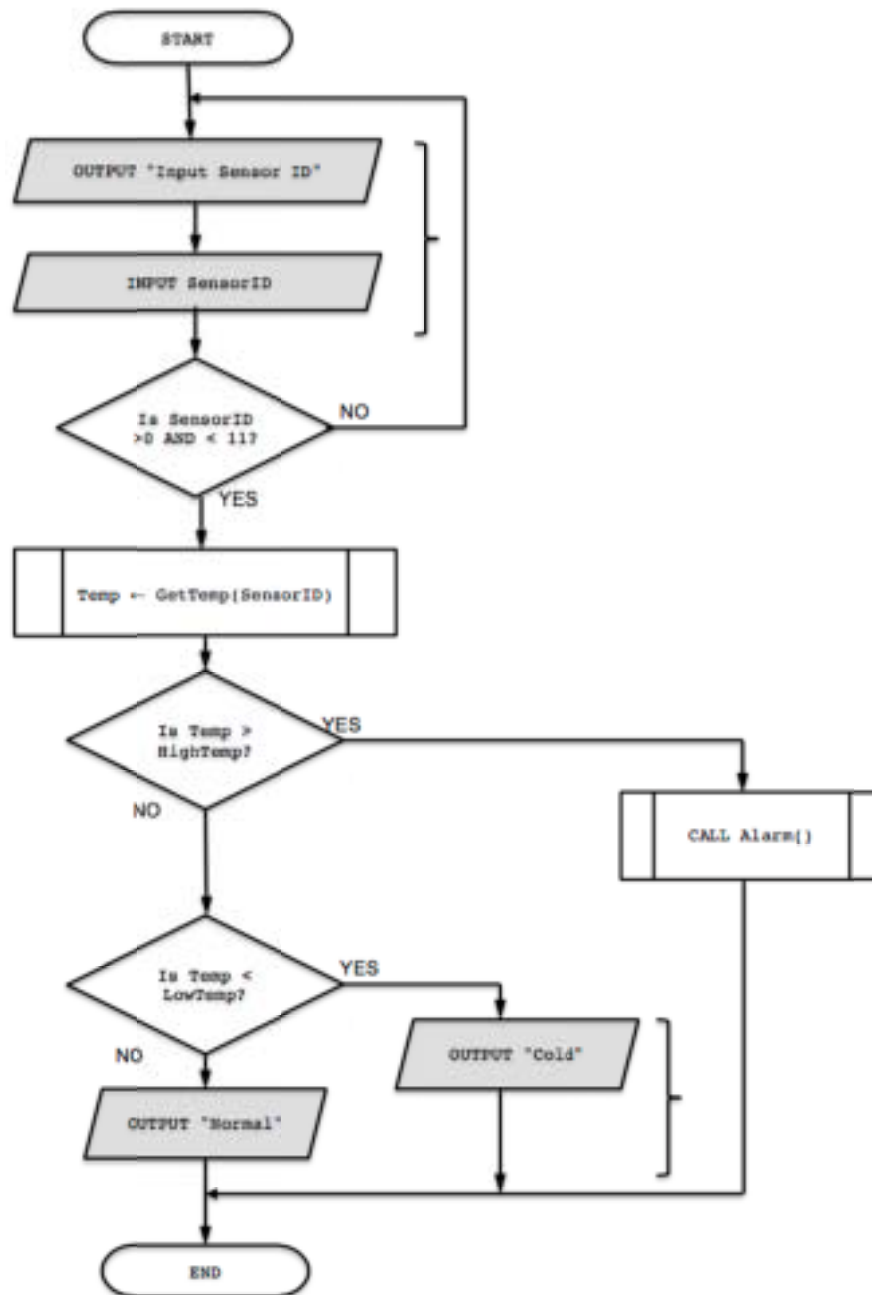
4

105. M/J 18/P21/Q2/a

2(a)(i)	• Indentation • Blank lines / white space • Capitalisation of keywords • Meaningful identifier names	4
2(a)(ii)	• Comments	1

106. M/J 18/P21/Q3

3



8

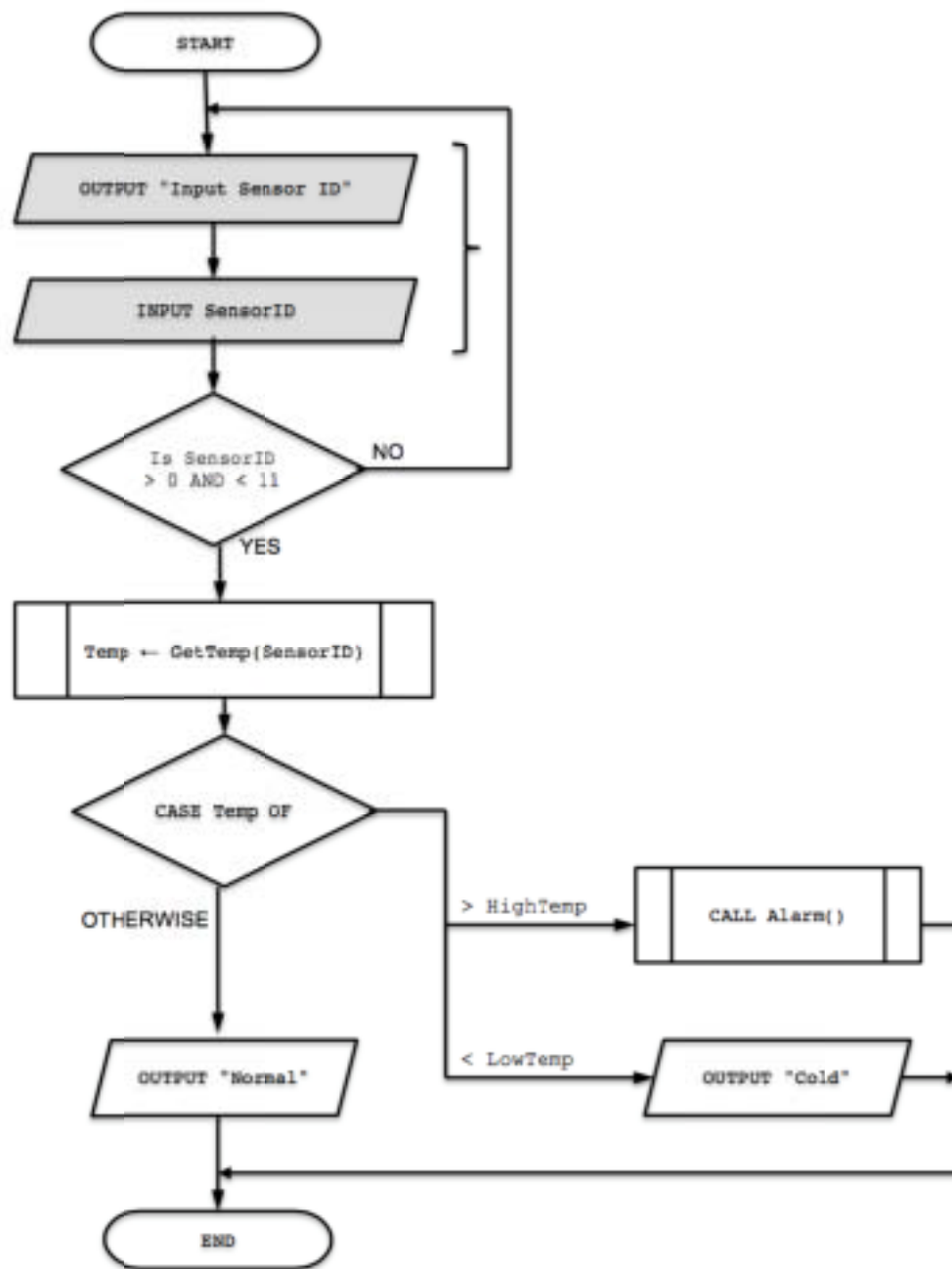
One mark for:

- 1 START and END / STOP
- 2 prompt and input of SensorID (allow alt name)
- 3 decision box checking that SensorID is between 1 & 10
- 4 calling GetTemp with SensorID as parameter
- 5 decision box comparing Temp > HighTemp
- 6 calling Alarm()
- 7 decision box comparing Temp < LowTemp
- 8 both output messages

3

ALTERNATIVE SOLUTION USING 'CASE'

8



Mark as follows:

- 1 One mark for START and END / STOP
- 2 One mark for prompt and input of SensorID (allow alt name)
- 3 One mark for decision box checking that SensorID is between 1 & 10
- 4 One mark for calling GetTemp with SensorID as parameter
- 5 One mark for decision box CASE Temp
- 6 One mark for calling Alarm()
- 7 One mark for two correct CASE conditions
- 8 One mark for both output messages

107. M/J 18/P22/Q1/a

1(a)

Description of data item	Suitable identifier name
The temperature inside the greenhouse	GreenhouseTemperature
The temperature outside the greenhouse	OutsideTemperature
The greenhouse identification number	GreenhouseID
The time the temperature was measured	SampleTime

The above are examples only.
Names must be meaningful and unambiguous

4

108. M/J 18/P22/Q2/a

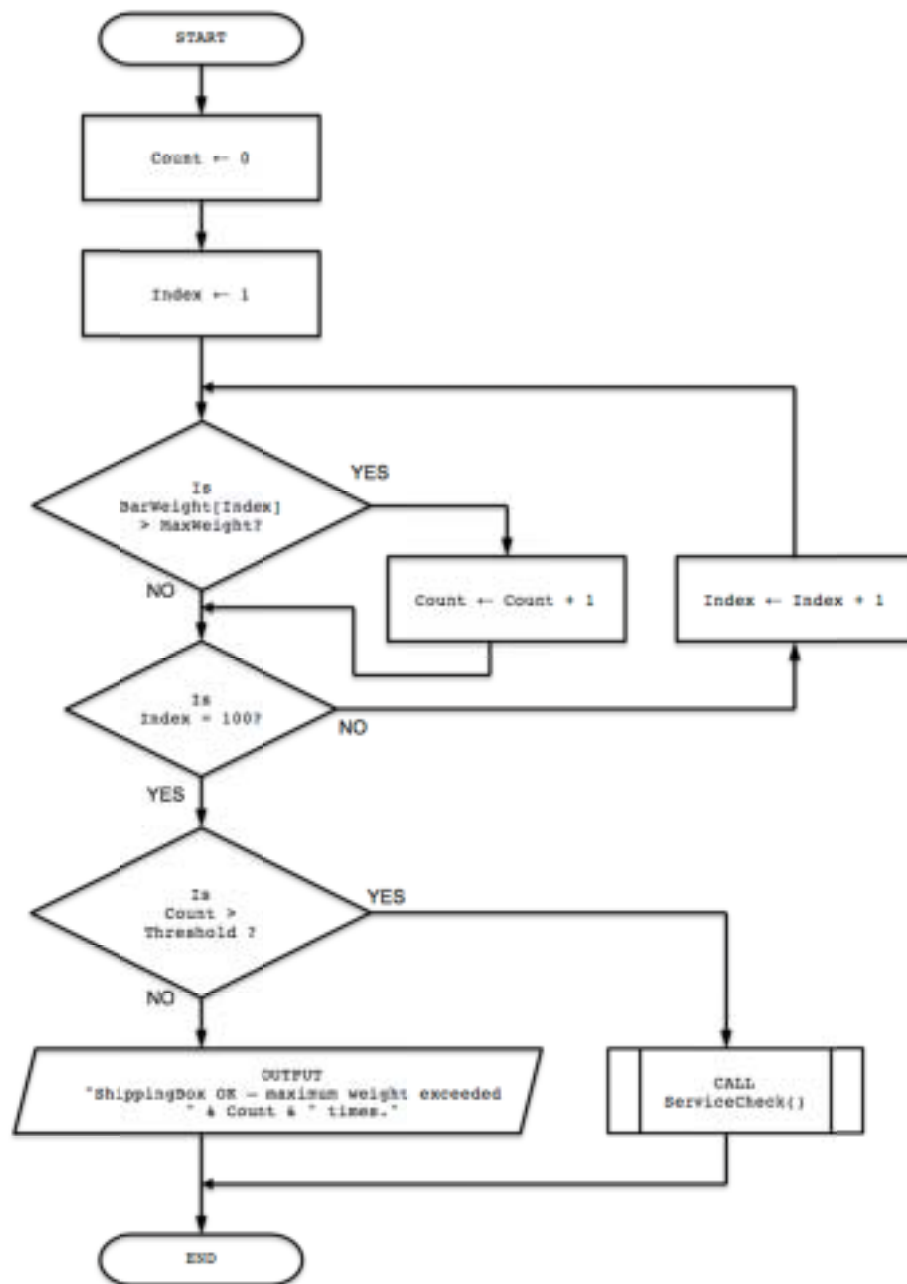
2(a)(i)	• Keywords in capitals • White Space / blank lines / grouping • Comments • Sensible function names	Max 2
2(a)(ii)	• Indentation • Meaningful identifier names	2

109. M/J 18/P22/Q3

3

Example Program Flowchart

10



3	One mark for each of: 1 START and END / STOP 2 Initialising <code>Count</code> to 0 3 Initialising <code>Index</code> to 1 or 0 4 Decision box comparing <code>BarWeight[Index] > MaxWeight</code> 5 Decision box comparing <code>Index</code> to 100 6 Decision box comparing <code>Count > Threshold</code> 7 Correct increment of <code>Index</code> 8 Correct increment of <code>Count</code> 9 Output message (concatenation of text and value) if threshold not exceeded 10 Calling <code>ServiceCheck()</code> if <code>Threshold</code> exceeded (without text message)	10
----------	---	-----------

110. M/J 18/P23/Q1/a

1(a)

Description of data item	Suitable identifier name
The temperature inside the house	InsideTemperature
The temperature outside the house	OutsideTemperature
The wind speed	WindSpeed
Whether it was raining or not	WasRaining

The above are examples only.
Names must be meaningful and unambiguous

Items 1 and 2 must have suitable prefix/suffix (i.e. not just 'temperature')

Reject single letter names

4

111. M/J 18/P23/Q2/a

2(a)	Comments: Explain the functionality of the code // Easier for other people to understand // Easier to maintain / debug / modify Indentation: Easier to identify <u>structure</u> / <u>blocks</u> // identify <u>blocks</u> of code	2
------	--	----------

112. M/J 18/P23/Q4

4

9



This is one possible solution – selection structure may differ

One mark for:

- 1 START **and** END // STOP
- 2 Initialisation of an `Index` variable **and** initialisation of a `Count` variable
- 3 Decision box / boxes to check temperature within acceptable range
- 4 Correct increment of `Count` variable
- 5 Decision box comparing `Index` to 100
- 6 Correct increment of `Index`
- 7 Decision box comparing `Count` > 20
- 8 Assigning both `TRUE` **and** `FALSE`
- 9 Returning the Boolean value

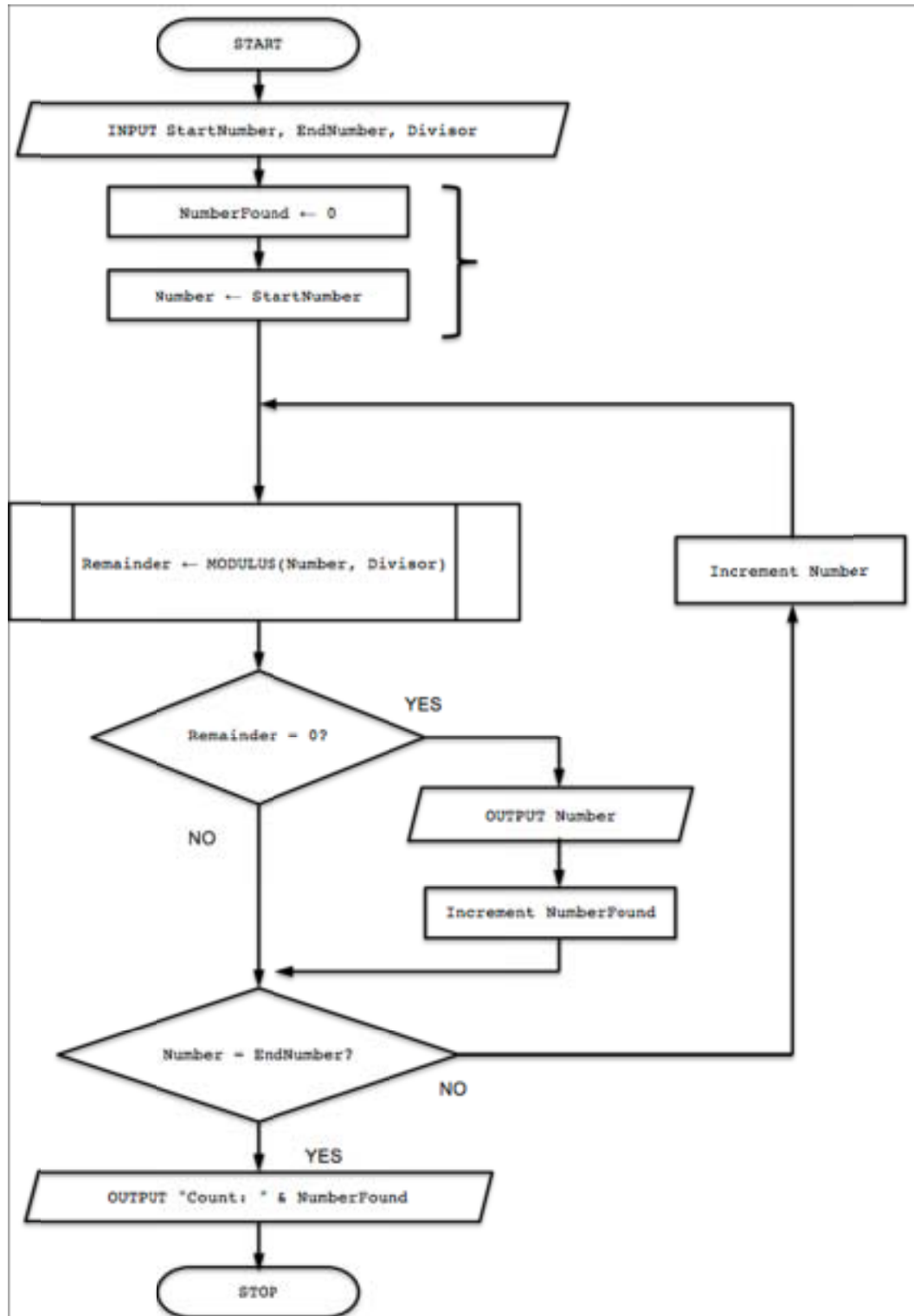
For solutions where Boolean variable not used:

- 8 Return `TRUE`
- 9 Return `FALSE`

113. O/N 17/P21/Q2/c

2(c)

10



2(c)	Mark as follows: • One mark for <code>START</code> and <code>STOP / END</code> • One mark for bracketed pair • One mark for each of other labelled boxes (shape must be correct for decision box) Decision box outputs must have two outputs and at least one label (<code>Yes / No</code>) Different statement categories should not appear in the same symbol (e.g. assignment and I/O) No mark for symbol (or pair) if parent missing or logically incorrect (except for <code>START/END</code>) Full marks should be awarded for functionally equivalent solutions.	
------	---	--

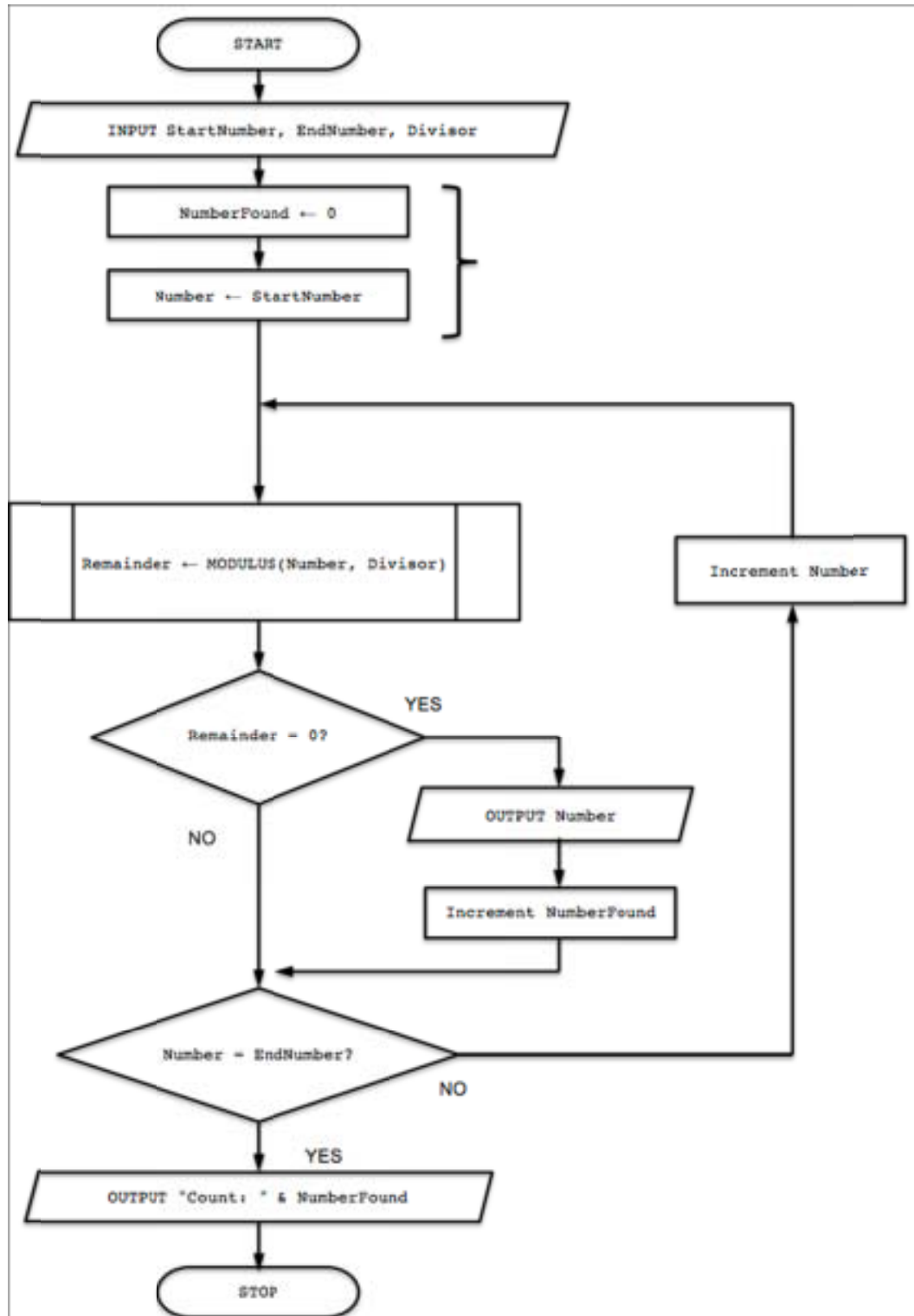
114. O/N 17/P22/Q2/b

2(b)	<pre> graph TD START([START]) --> Input[/Input SearchValue/] Input --> FoundFlag[FoundFlag ← FALSE] FoundFlag --> Index[Index ← 1] Index --> LoopCond{Index < 101 AND FoundFlag = FALSE?} LoopCond -- YES --> ArrayCheck{ClassName[Index] = SearchVal?} ArrayCheck -- YES --> OutputIndex[/OUTPUT Index/] OutputIndex --> FoundFlagTrue[FoundFlag ← TRUE] FoundFlagTrue --> IncrementIndex[Increment Index] ArrayCheck -- NO --> IncrementIndex LoopCond -- NO --> FoundFlagFalse{FoundFlag = FALSE?} FoundFlagFalse -- YES --> OutputNotFound[/OUTPUT "Not found"/] FoundFlagFalse -- NO --> STOP([STOP]) IncrementIndex --> LoopCond </pre>	9
2(b)	Mark as follows: • One mark for START and STOP / END • One mark for each bracketed pair • One mark for each of other labelled symbol (decision box shape must be correct) • Allow F/T from incorrect decision symbol Full marks should be awarded for functionally equivalent solutions.	

115. O/N 17/P23/Q2/c

2(c)

10



2(c)	Mark as follows: • One mark for START and STOP / END • One mark for bracketed pair • One mark for each of other labelled boxes (shape must be correct for decision box) Decision box outputs must have two outputs and at least one label (Yes / No) Different statement categories should not appear in the same symbol (e.g. assignment and I/O) No mark for symbol (or pair) if parent missing or logically incorrect (except for START/END) Full marks should be awarded for functionally equivalent solutions.	
------	---	--

116. M/J 17/P21/Q1/a

1(a)	Input: • Enter data into the system // get / receive / read data • <code>INPUT MyVar // READFILE MyFile, MyString</code> Process: • Manipulate / change data in some way // perform a calculation / find a result • <code>MyChar ← 'X' // MyNum ← MyNum + 1</code> Output: • Send data out from the system // display / print / transmit / show data • <code>OUTPUT "Hello World" // WRITEFILE MyFile, MyString</code> Mark as follows: 1 mark for each type (in bold) 1 mark for each description and pseudocode example	7
------	---	----------

117. M/J 17/P21/Q2/a,b,c

2(a)	Stepwise refinement	1
2(b)	1 mark for first 2 data types – String 1 mark for last 2 data types – Boolean 1 mark for each description: PasswordInput Stores password <u>entered</u> UserIDFound True if user ID found in the <u>file</u> PasswordValid True if password entered matches password from <u>file</u> //Input password matches stored password	5

2(c)	1. LOOP through the file until EOF... 2. ...OR UserIdInput is found 3. READ text line from Password.txt file in a loop 4. SPLIT into UserID and password in a loop 5. IF UserIdInput matches UserID from file THEN in a loop 6. SET UserIDFound to TRUE in a loop 7. IF UserIDFound = TRUE AND PasswordInput matches value from file THEN 8. Set PasswordValid to TRUE Mark as follows: 1 mark per functional equivalent of each numbered statement.	8
------	---	---

118. M/J 17/P22/Q1/a

1(a)

Item	Statement	Input	Process	Output
1	SomeChars = "Hello World"		✓	
2	OUTPUT RIGHT(String1,5)		✓	✓
3	READFILE (MyFile, String2)	✓		
4	WRITEFILE (MyFile, "Data is " & String2)		✓	✓

6

Mark as follows:

Row 1 as shown

Row 2 no marks if tick in Input column, otherwise 1 mark per tick

Row 3 as shown

Row 4 no marks if tick in Input column, otherwise 1 mark per tick

119. M/J 17/P22/Q2/a,b,c

2(a)	- to <u>increase the level of detail of an algorithm / design...</u> // breaking down a <u>problem / module / task</u> into smaller parts... ...from which the task may be <u>programmed</u> 1 mark per underlined phrase or equivalent	2
2(b)	1 mark for first 3 data types – String 1 mark for last data type – Boolean 1 mark for each description: FileUserID Stores (User) ID from <u>file</u> FilePreferredName Stores (preferred) name from <u>file</u> IDFoundFlag True if (User) ID found in <u>file</u> // False if (User) ID not found in <u>file</u> // If SearchUserID matches FileUserID	5

2(c)	1. LOOP through the file until EOF()... 2. OR SearchUserId is found 3. READ text line from UserNames.txt file in a loop 4. EXTRACT FileUserID in a loop 5. IF SearchUserId matches FileUserID THEN in a loop 6. SET FilePreferredName to the name from the file 7. Check if User ID found not in a loop 8. OUTPUT appropriate message for both conditions 1 mark per functional equivalent of each numbered statement.	Max 8
------	---	--------------

120. M/J 17/P23/Q1/a

1(a)	Input: • Enter data into the system // get / receive / read data • INPUT MyVar // READFILE MyFile, MyString Process: • Manipulate / change data in some way // perform a calculation / find a result • MyChar ← 'X' // MyNum ← MyNum + 1 Output: • Send data out from the system // display / print / transmit / show data • OUTPUT "Hello World" // WRITEFILE MyFile, MyString Mark as follows: 1 mark for each type (in bold) 1 mark for each description and pseudocode example	7
------	--	----------

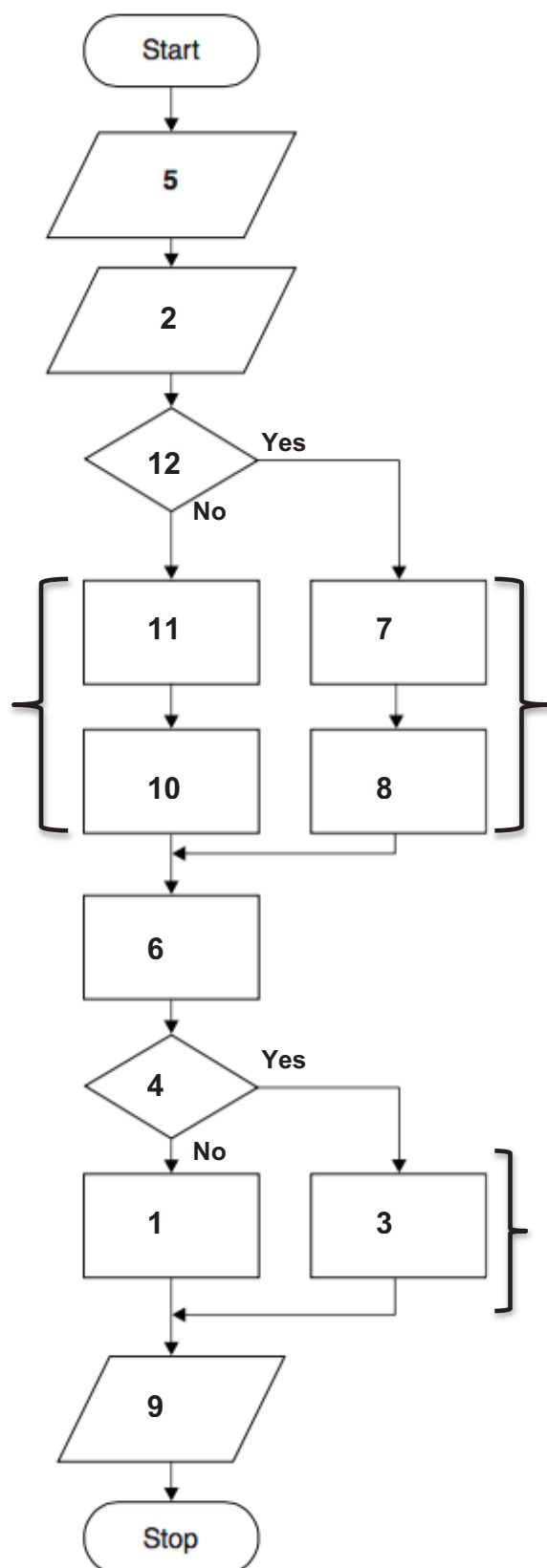
121. M/J 17/P23/Q2/a,b,c

2(a)	Stepwise refinement	1
2(b)	1 mark for first 2 data types – String 1 mark for last 2 data types – Boolean 1 mark for each description: PasswordInput Stores password <u>entered</u> UserIDFound True if user ID found in the <u>file</u> PasswordValid True if password entered matches password from <u>file</u> //Input password matches stored password	5

2(c)	1. LOOP through the file until EOF... 2. ...OR UserIdInput is found 3. READ text line from Password.txt file in a loop 4. SPLIT into UserID and password in a loop 5. IF UserIdInput matches UserID from file THEN in a loop 6. SET UserIDFound to TRUE in a loop 7. IF UserIDFound = TRUE AND PasswordInput matches value from file THEN 8. Set PasswordValid to TRUE Mark as follows: 1 mark per functional equivalent of each numbered statement.	8
------	--	----------

122. O/N 16/P21/Q1/a,b,c

1 (a)



Note: Order of 11, 10
and 7,8 may be
reversed.

One mark for each of the following symbols / symbol combinations:

- 2
- 7 and 8 from YES
- 10 and 11
- 6
- 1 and 3 (1 from NO, 3 from YES)
- 9
- 12 and 4

Max [6]

(b) Rows 2 to 7 are examples only

TicketType	BaggageWeight	Explanation	Expected output
E	15	Under the allowance	0
E	> 16	Under the allowance	<i>Charge</i>
S	<= 20	Under the allowance	0
S	> 20	Under the allowance	<i>Charge</i>
E	16	Boundary weight for a type E ticket	0
S	20	Boundary weight for a type S ticket	0
E or S	negative or non-numeric	Invalid weight	Error message

Ticket type	Baggage allowance (kg)	Charge rate per additional kg (\$)
'E'	16	3.50
'S'	20	5.75

One mark for each different test (examples above)

Max [5]

(c) INPUT TicketType
WHILE NOT (TicketType = 'E') OR (TicketType = 'S')
 INPUT TicketType
ENDWHILE

One mark for each of:

- WHILE ... ENDWHILE
- Correct condition in a loop
- INPUT within loop plus one before loop // alternative arrangement leading to correct exit from loop

[3]

123. O/N 16/P21/Q2/b

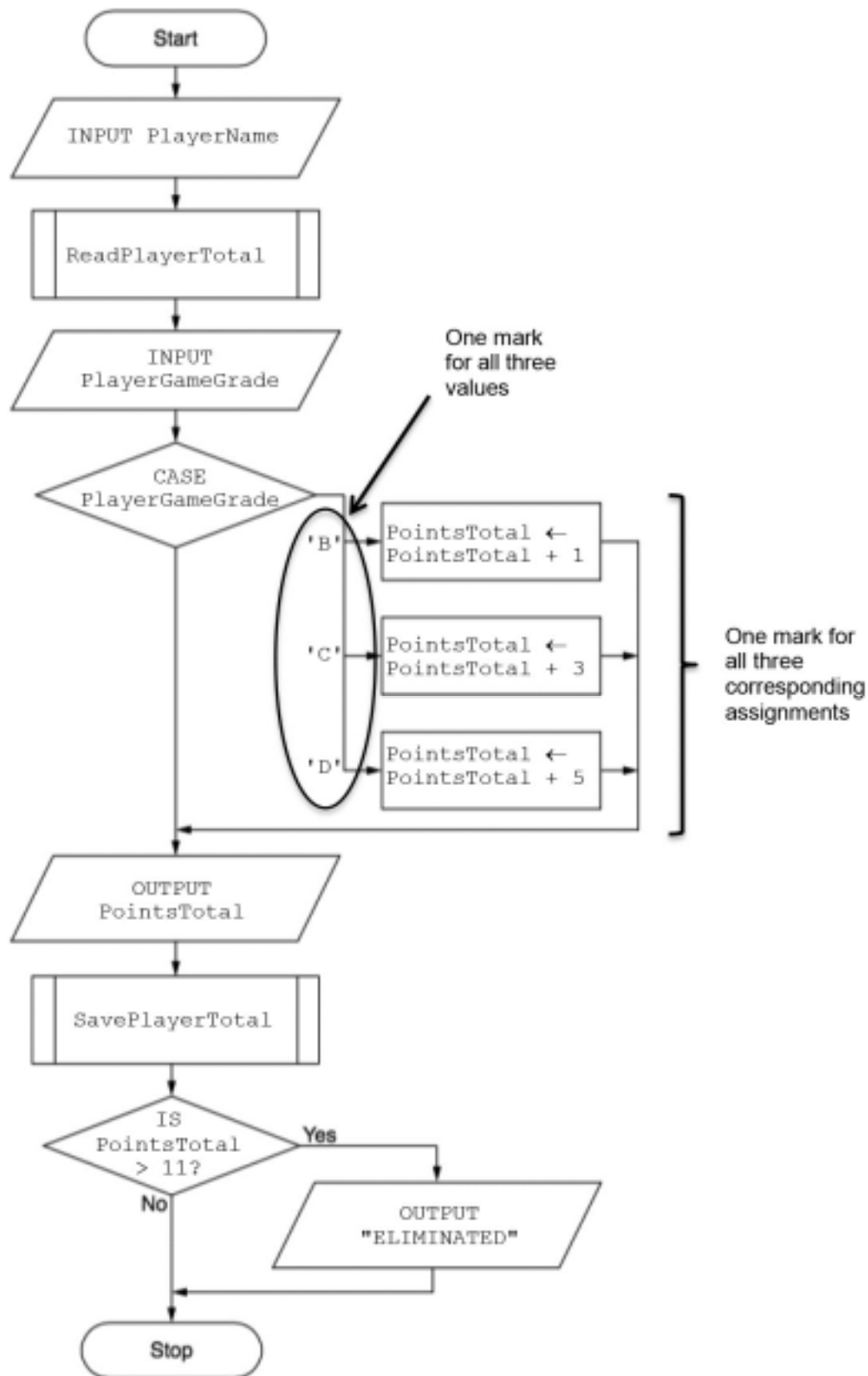
(b) One mark for each of:

- Assignment: 01 // 02 // 06 // 09 // 14 // 18
- Selection: 07 // 11
- Iteration: 03 // 05

[3]

124. O/N 16/P22/Q1/a,b,c

1 (a)



Mark as follows:

- One mark per shape, correctly labelled (except for three assignments as noted above)
- One mark for three selection values ('B', 'C' and 'D')

[9]

(b)

PointsTotal	PlayerGameGrade	Updated	Output
n	A	n	n
n	B	n + 1	n + 1
n	C	n + 3	n + 3
n	D	n + 5	n + 5
e.g. 10	e.g. C	13	13 ELIMINATED

One mark per complete row testing **different** routes through the algorithm.

[5]

(c) `INPUT PlayerGameGrade`
`WHILE NOT (PlayerGameGrade = 'A' OR`
`PlayerGameGrade = 'B' OR`
`PlayerGameGrade = 'C' OR`
`PlayerGameGrade = 'D')`
`OUTPUT "Invalid - Re-enter"`
`INPUT PlayerGameGrade`
`ENDWHILE`

One mark for each of:

- `WHILE ... ENDWHILE`
- Correct condition in a loop
- `INPUT` within loop plus one before loop // alternative arrangement leading to correct exit from loop

[3]

125. O/N 16/P22/Q5/b(i)

(b) (i) Stepwise refinement // Top-Down Design

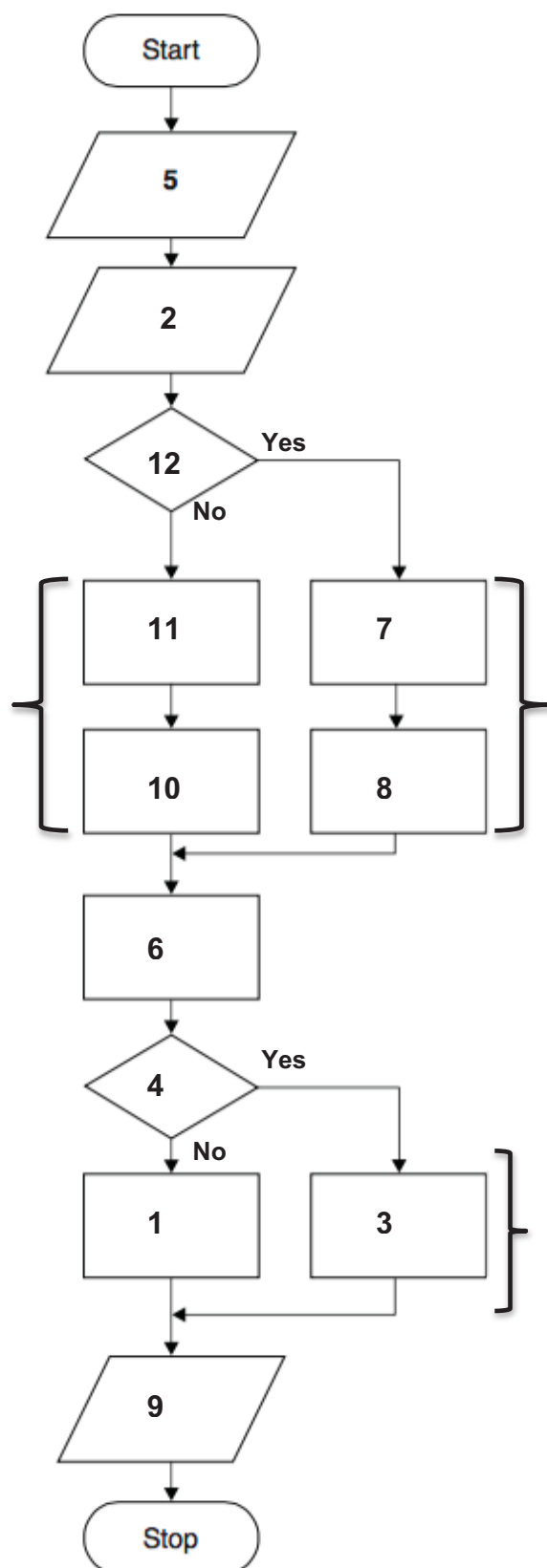
[1]

- 5 (a) • 2D array
 • of type integer
 • with identifier `PlayerScore`

[Max. 2]

126. O/N 16/P23/Q1/a,b,c

1 (a)



Note: Order of 11, 10
and 7,8 may be
reversed.

One mark for each of the following symbols / symbol combinations:

- 2
- 7 and 8 from YES
- 10 and 11
- 6
- 1 and 3 (1 from NO, 3 from YES)
- 9
- 12 and 4

Max [6]

(b) Rows 2 to 7 are examples only

TicketType	BaggageWeight	Explanation	Expected output
E	15	Under the allowance	0
E	> 16	Under the allowance	<i>Charge</i>
S	<= 20	Under the allowance	0
S	> 20	Under the allowance	<i>Charge</i>
E	16	Boundary weight for a type E ticket	0
S	20	Boundary weight for a type S ticket	0
E or S	negative or non-numeric	Invalid weight	Error message

Ticket type	Baggage allowance (kg)	Charge rate per additional kg (\$)
'E'	16	3.50
'S'	20	5.75

One mark for each different test (examples above)

Max [5]

```
(c) INPUT TicketType
WHILE NOT (TicketType = 'E') OR (TicketType = 'S')
    INPUT TicketType
ENDWHILE
```

One mark for each of:

- WHILE ... ENDWHILE
- Correct condition in a loop
- INPUT within loop plus one before loop // alternative arrangement leading to correct exit from loop

[3]

127. O/N 16/P23/Q2/b

(b) One mark for each of:

- Assignment: 01 // 02 // 06 // 09 // 14 // 18
- Selection: 07 // 11
- Iteration: 03 // 05

[3]

128. M/J 16/P21/Q2/a,b

2 (a)	Identifier	Data Type	Description	4
	AlarmState	BOOLEAN	Alarm is set to ON or OFF	
	SensorValue	INTEGER	Value / number from sensor / as input by user // used in calculation of Temperature	
	ThresholdValue	REAL / FLOAT / SINGLE / DOUBLE	Threshold value for comparison	
	Temperature	REAL / FLOAT / SINGLE / DOUBLE	Temperature value calculated from sensor value	
One mark per row Data types as shown Descriptions given above are examples only				

(b)	<pre> AlarmState ← FALSE ① INPUT ThresholdValue INPUT SensorValue } ② Temperature ← SensorValue * 1.135 ③ ④ { IF Temperature > ThresholdValue THEN AlarmState ← TRUE OUTPUT "Temperature Alarm" } ⑤ ELSE OUTPUT "Temperature OK" ⑥ AlarmState ← FALSE ⑦ ENDIF </pre> Mark points as circled, descriptions as below: 1. Setting AlarmState to FALSE (Cond. check not essential but must be correct if present) 2. Inputting SensorVal and ThresholdVal 3. Correct value assigned to Temperature (must be * not x) 4. IF..THEN..ELSE..ENDIF structure with correct condition (or two separate IF clauses) 5. Correct THEN statements as shown 6. Correct ELSE statement as shown 7. Setting AlarmState to FALSE within ELSE clause only if mark point 1 not given	Max 6
-----	--	-------

129. M/J 16/P21/Q4/a

4 (a)	• Program code is <u>easier</u> to implement / manage • Modules may be given to different people to develop // given to program specialists • Program code is <u>easier</u> to test / debug / maintain • Encourages the re-usability of program code	Max 2
-------	---	-------

130. M/J 16/P22/Q2/a,b

2 (a)	<table border="1"> <thead> <tr> <th>Identifier</th><th>Data Type</th><th>Description</th></tr> </thead> <tbody> <tr> <td>AlarmState</td><td>BOOLEAN</td><td>Alarm is set to ON or OFF</td></tr> <tr> <td>SensorValue</td><td>INTEGER</td><td>Value / number from sensor / as input by user // used in calculation of Temperature</td></tr> <tr> <td>ThresholdValue</td><td>REAL / FLOAT / SINGLE / DOUBLE</td><td>Threshold value for comparison</td></tr> <tr> <td>Temperature</td><td>REAL / FLOAT / SINGLE / DOUBLE</td><td>Temperature value calculated from sensor value</td></tr> </tbody> </table> One mark per row Data types as shown Descriptions given above are examples only	Identifier	Data Type	Description	AlarmState	BOOLEAN	Alarm is set to ON or OFF	SensorValue	INTEGER	Value / number from sensor / as input by user // used in calculation of Temperature	ThresholdValue	REAL / FLOAT / SINGLE / DOUBLE	Threshold value for comparison	Temperature	REAL / FLOAT / SINGLE / DOUBLE	Temperature value calculated from sensor value	4
Identifier	Data Type	Description															
AlarmState	BOOLEAN	Alarm is set to ON or OFF															
SensorValue	INTEGER	Value / number from sensor / as input by user // used in calculation of Temperature															
ThresholdValue	REAL / FLOAT / SINGLE / DOUBLE	Threshold value for comparison															
Temperature	REAL / FLOAT / SINGLE / DOUBLE	Temperature value calculated from sensor value															
(b)	<pre> AlarmState ← FALSE ① INPUT ThresholdValue INPUT SensorValue } ② Temperature ← SensorValue * 1.135 ③ IF Temperature > ThresholdValue THEN AlarmState ← TRUE OUTPUT "Temperature Alarm" } ⑤ ELSE OUTPUT "Temperature OK" ⑥ AlarmState ← FALSE ⑦ ENDIF ④ </pre> Mark points as circled, descriptions as below: - Setting AlarmState to FALSE (Cond. check not essential but must be correct if present) - Inputting SensorVal and ThresholdVal - Correct value assigned to Temperature (must be * not x) - IF...THEN...ELSE...ENDIF structure with correct condition (or two separate IF clauses) - Correct THEN statements as shown - Correct ELSE statement as shown - Setting AlarmState to FALSE within ELSE clause only if mark point 1 not given	Max 6															

131. M/J 16/P22/Q4/a

4 (a)	• Program code is <u>easier</u> to implement / manage • Modules may be given to different people to develop // given to program specialists • Program code is <u>easier</u> to test / debug / maintain • Encourages the re-usability of program code	Max 2

132. M/J 16/P23/Q2/a,b**2 (a)****[5]**

Identifier	Data Type	Description
Accelerator	INTEGER	Accelerator position Values: 0 to 100 in steps of 1 Meaning: 0 : none (not pressed) 100: maximum (fully pressed)
EngineTemp	REAL / FLOAT / SINGLE / DOUBLE	Engine temperature in °C (–50 to +150 correct to 1 decimal place)
NormalTemp	INTEGER	Normal engine temperature in °C Whole number; typical value 90
Speed	INTEGER	Road speed of car (in km/hr) Values: 0 to 200 in steps of 1
EngineStop	BOOLEAN	Value used to signal engine must be stopped Possible values: TRUE: stop engine FALSE: run engine

One mark per row
Data types as shown

(b)

[6]

```
INPUT Accelerator  
INPUT EngineTemp  
INPUT NormalTemp  
INPUT Speed  
}  
  
IF Accelerator = 0 AND EngineTemp >= NormalTemp AND Speed = 0  
  THEN  
    EngineStop ← TRUE  
  ELSE  
    EngineStop ← FALSE  
ENDIF  
OUTPUT "Engine Stopped"
```

Diagram annotations: A bracket on the right groups the four INPUT statements, with a circled '1' next to it. A circled '2' is to the left of the IF statement. A circled '3' is above the condition of the IF statement. A circled '4' is to the right of the THEN statement. A circled '5' is to the right of the ELSE statement. A circled '6' is to the right of the OUTPUT statement.

Mark points as circled, descriptions as below:

1. Four INPUT statements (correct names and sequence)
2. Correct IF...THEN...ELSE...ENDIF including first condition (or equivalent nested IFs)
3. Correct second and third conditions
4. Correct THEN statement
5. Correct ELSE statement
6. Output indicating EngineStop
(Following ENDIF or as **two** separate statements within THEN and ELSE)

133. O/N 15/P21/Q3/b

```
(b) IF InA = TRUE AND InB = TRUE
    THEN
        OutZ ← FALSE
    ELSE
        OutZ ← TRUE
ENDIF
```

Mark as follows

Structure: IF - THEN - ELSE - ENDIF

[1]

Condition: InA = TRUE AND InB = TRUE

[1]

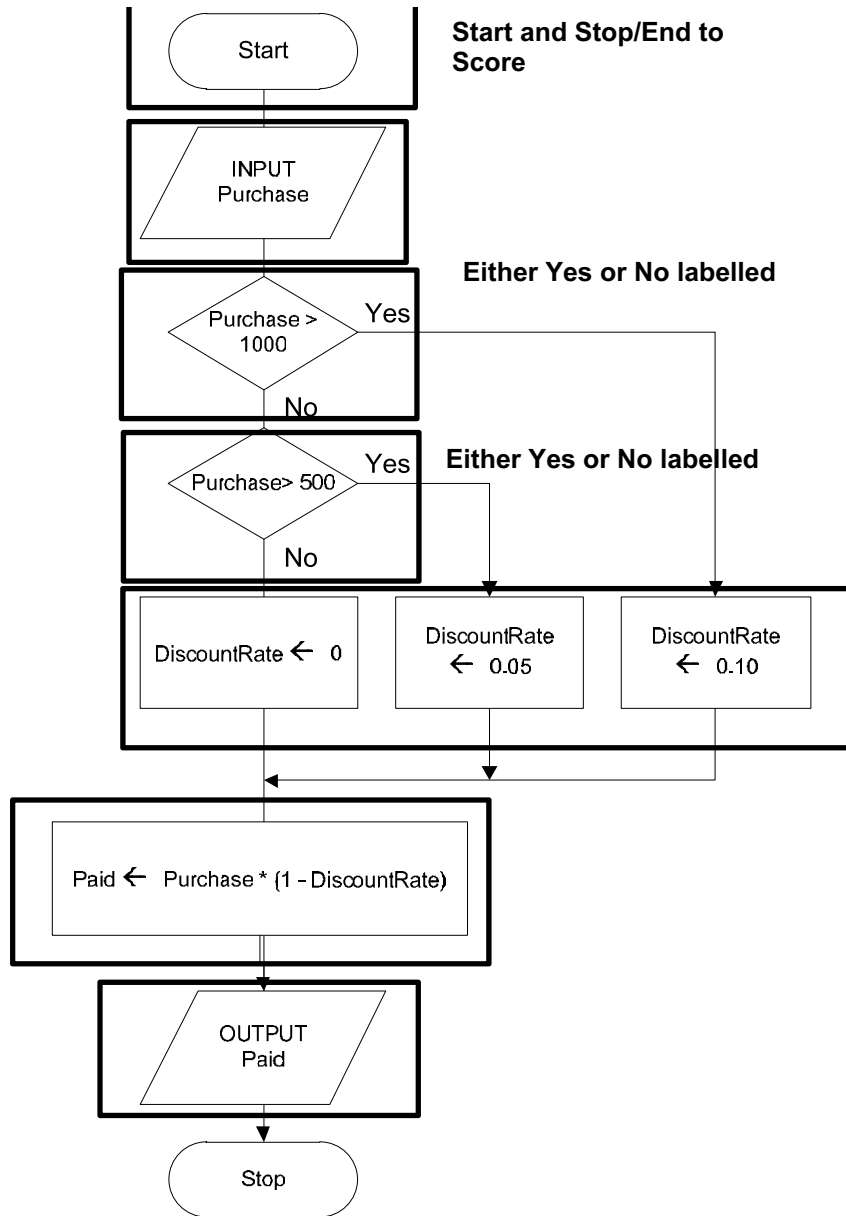
Logic: OutZ ← FALSE (when condition true)}
 OutZ ← TRUE (when condition false)}

[1]

Alternative answer (worth 3 marks):

OutZ ← NOT(InA AND InB)

OutZ ← NOT InA OR NOT InB

134. O/N 15/P21/Q4**4**

[MAX 6]

[Total: 6]

[MAX 6]

135. O/N 15/P21/Q5/a**5 (a)**

Identifier	Data type	Description
YearCount	INTEGER	Loop counter /// Age of the car
PurchasePrice	INTEGER	Purchase price of the car
CurrentValue	REAL // CURRENCY Allow: SINGLE Refuse: DOUBLE	The changing depreciated value

[1]

[1]

[1]

Must have correct identifier + Data type + Description to score

136. O/N 15/P22/Q2/b**(b)**

```

IF P = 1 AND Q = 1
  THEN
    X ← 1
  ELSE
    X ← 0
ENDIF

```

```

IF P = 0 OR Q = 0
  THEN
    X ← 0
  ELSE
    X ← 1
ENDIF

```

Mark as follows:**Structure:** IF - THEN - ELSE - ENDIF

[1]

Condition: P = 1 AND Q = 1

[1]

Allow &/&& for the operator

Logic:

X ← 1 (for TRUE)	}
X ← 0 (for FALSE)	

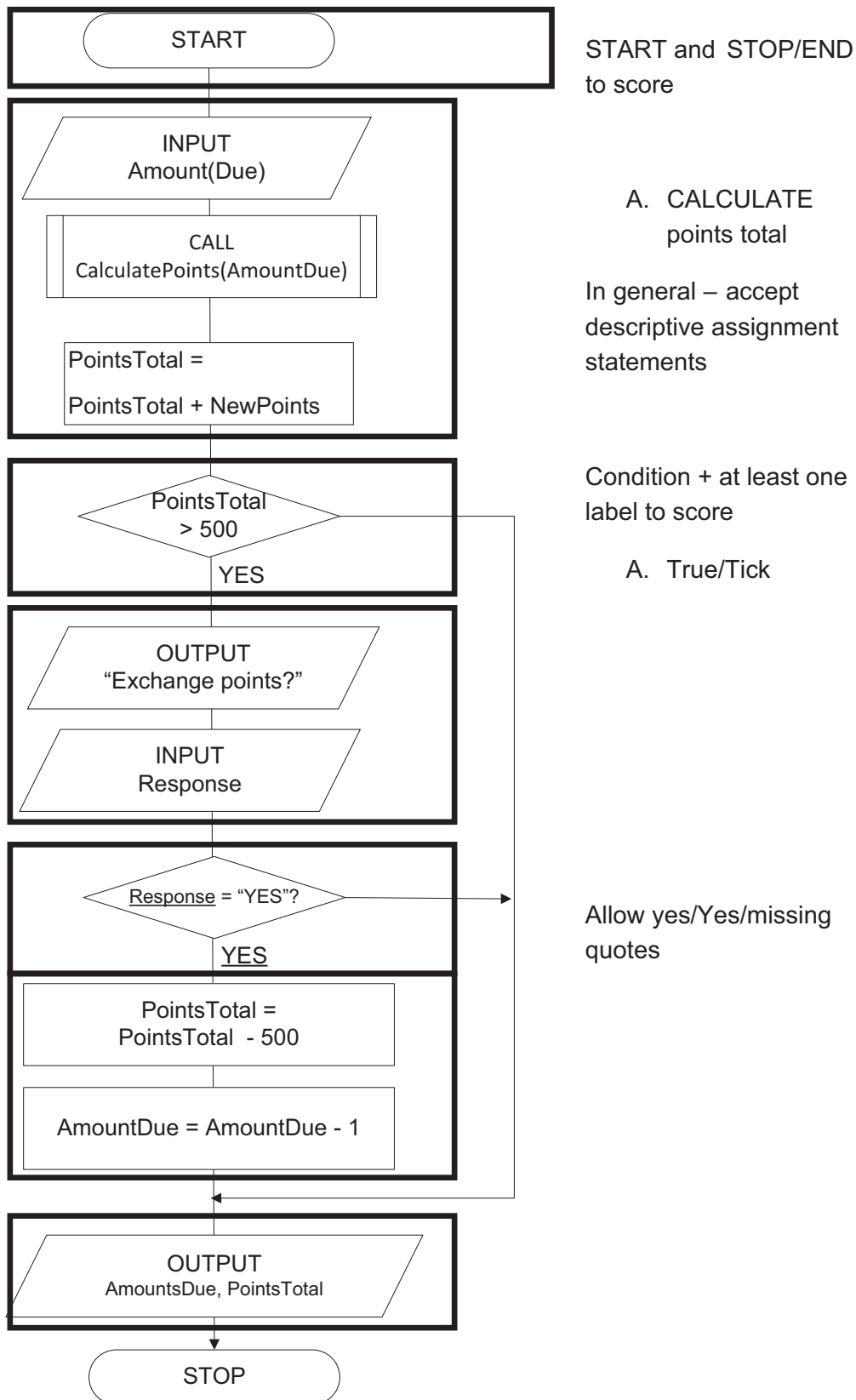
[1]

Check carefully for:

- other alternative correct algorithm
- a 'mirror copy' of the question paper algorithm – score 0

137. O/N 15/P22/Q3

3



[Max 6]

138. O/N 15/P22/Q5/b**(b)**

Identifier	Data Type	Description
Num		
N	INTEGER	The number of numbers in the list
i	INTEGER	Loop counter // The number of 'passes' up through the list
j	INTEGER	The <u>index</u> // position in the array
Temp	INTEGER	Description must imply/states the 'swapping' operation

Mark as follows:

INTEGER × 4

One mark per description

[1]

[4]

139. O/N 15/P22/Q6/b(i)**(b) (i)**

Identifier	Data Type	Description
InputString	STRING	The string value input by the user
i	INTEGER	<u>Loop</u> counter // (index) position of an individual character
j	INTEGER	Number of characters in / length of InputString
NextChar	CHAR//CHARACTER	(Single) character within InputString / from string input by the user
NewString	STRING	The string formed/made/created//output Allow: if "by the user" added

[1]

[1]

[1]

[1]

Note: Correct (identifier + the data type + description) needed to score

140. O/N 15/P22/Q7/b(ii)

- (ii) For each/every character [1]
A replacement character is 'calculated' from its ASCII value // or by example ... [1]

141. O/N 15/P23/Q3/b

```
(b) IF InA = TRUE AND InB = TRUE
    THEN
        OutZ ← FALSE
    ELSE
        OutZ ← TRUE
ENDIF
```

Mark as follows

Structure: IF - THEN - ELSE - ENDIF

[1]

Condition: InA = TRUE AND InB = TRUE

[1]

Logic: OutZ ← FALSE (when condition true)}
 OutZ ← TRUE (when condition false)}

[1]

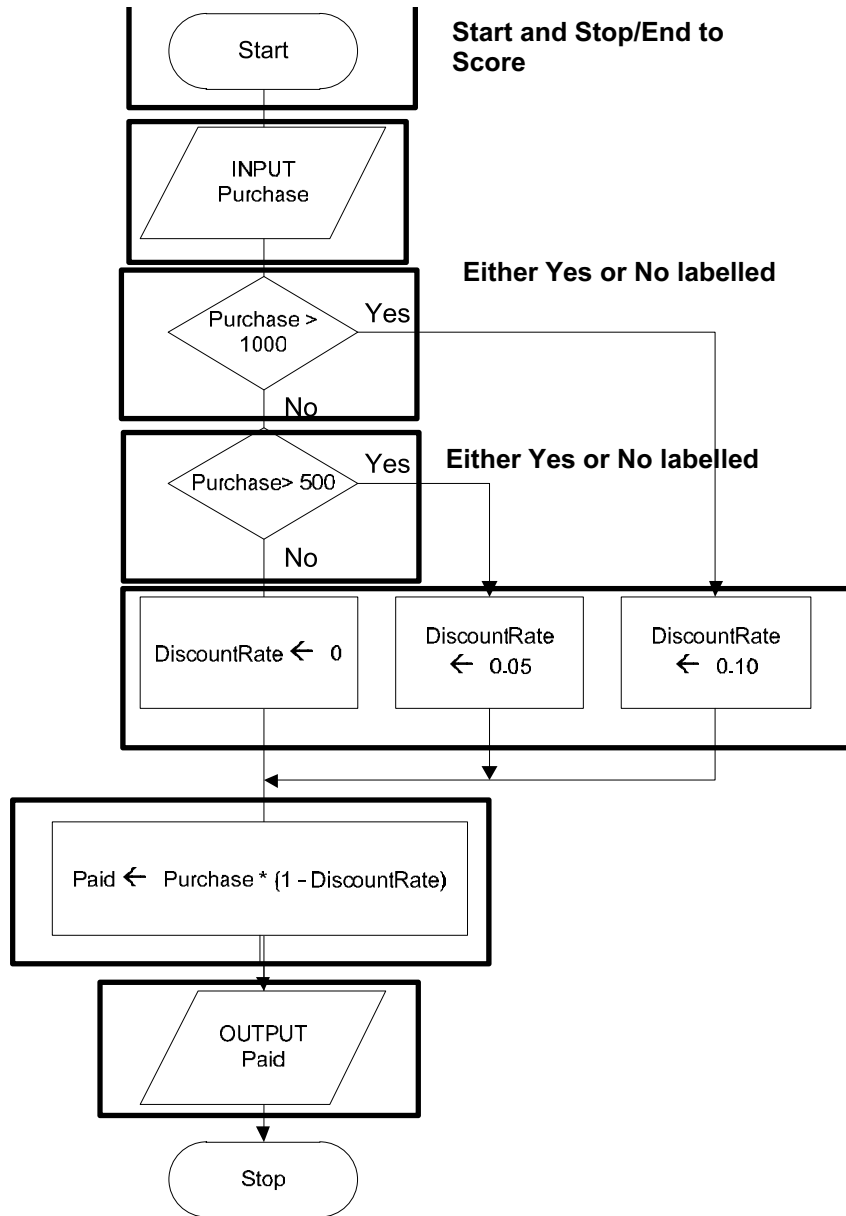
Alternative answer (worth 3 marks):

OutZ ← NOT(InA AND InB)

OutZ ← NOT InA OR NOT InB

142. O/N 15/P23/Q4

4



[MAX 6]

[Total: 6]

[MAX 6]

143. O/N 15/P23/Q5/a**5 (a)**

Identifier	Data type	Description
YearCount	INTEGER	Loop counter /// Age of the car
PurchasePrice	INTEGER	Purchase price of the car
CurrentValue	REAL // CURRENCY Allow: SINGLE Refuse: DOUBLE	The changing depreciated value

[1]

[1]

[1]

Must have correct identifier + Data type + Description to score

144. M/J 15/P21/Q1/a,b(i)**1 (a)**

Identifier	Data Type	Description
RaceHours	INTEGER	The hours part of the race time
RaceMinutes	INTEGER	the minute part of the race time
RaceSeconds	INTEGER // REAL	the seconds part of the race time
RaceTime	INTEGER // REAL	the race time in seconds

3 × (meaningful name + data type)

[3]

(b) (i)

Identifier	Data Type	Description
PersonalBestTime	INTEGER/REAL	Personal best time in seconds

meaningful name + data type

[1]

145. M/J 15/P22/Q1/a,b(i)**1 (a)**

Identifier	Data Type	Description
RaceHours	INTEGER	The hours part of the race time
RaceMinutes	INTEGER	the minute part of the race time
RaceSeconds	INTEGER // REAL	the seconds part of the race time
RaceTime	INTEGER // REAL	the race time in seconds

3 × (meaningful name + data type)

[3]

(b) (i)

Identifier	Data Type	Description
PersonalBestTime	INTEGER/REAL	Personal best time in seconds

meaningful name + data type

[1]